

**LAPORAN AKHIR PRAKTIKUM
PEMROGRAMAN MOBILE**



OLEH:

ZAHRA NABILA NIM 2310817320007

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2025**

LEMBAR PENGESAHAN
LAPORAN AKHIR PRAKTIKUM PEMROGRAMAN MOBILE

Laporan Akhir Praktikum Pemrograman Mobile ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Akhir Praktikum ini dikerjakan oleh:

Nama Praktikan : Zahra Nabila
NIM : 2310817320007

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar
NIM 2210817210026

Muti'a Maulida, S.Kom., M.T.I.
NIP. 198810272019032013

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR GAMBAR	5
DAFTAR TABEL.....	7
MODUL 1: Android Basic with Kotlin	10
SOAL 1	10
A. Source Code	12
B. Output Program.....	20
C. Pembahasan.....	20
MODUL 2: Android Layout	26
SOAL 1	26
A. Source Code	27
B. Output Program.....	38
C. Pembahasan.....	42
MODUL 3: Build a Scrollable List.....	49
SOAL 1	49
A. Source Code	51
B. Output Program.....	75
C. Pembahasan.....	76
SOAL 2	86
A. Pembahasan.....	86
MODUL 4: ViewModel and Debugging	87
SOAL 1	87
A. Source Code	88

B. Output Program.....	115
C. Pembahasan.....	119
SOAL 2	131
A. Pembahasan.....	131
MODUL 5: Connect to the Internet	133
SOAL 1	133
A. Source Code	133
B. Output Program.....	174
C. Pembahasan.....	176
LINK GITHUB	195

DAFTAR GAMBAR

MODUL 1: Android Basic with Kotlin

Gambar 1. Tampilan Awal Aplikasi Soal 1 Modul 1.....	10
Gambar 2. Tampilan Dadu Setelah di Roll Soal 1 Modul 1	11
Gambar 3. Tampilan Roll Dadu Double Soal 1 Modul 1	12
Gambar 4. Screenshot Hasil Tampilan Soal 1 Modul 1	20
Gambar 5. Screenshot Hasil Tampilan Soal 1 Modul 1	20
Gambar 6. Screenshot Hasil Tampilan Soal 1 Modul 1	20

MODUL 2: Android Layout

Gambar 7. Tampilan Awal Aplikasi Soal 1 Modul 2.....	26
Gambar 8. Tampilan Aplikasi Setelah Dijalankan Soal 1 Modul 2	26
Gambar 9. Screenshot Hasil Tampilan Soal 1 Modul 2.....	38
Gambar 10. Screenshot Hasil Tampilan Soal 1 Modul 2.....	38
Gambar 11. Screenshot Hasil Tampilan Soal 1 Modul 2.....	39
Gambar 12. Screenshot Hasil Tampilan Soal 1 Modul 2.....	39
Gambar 13. Screenshot Hasil Tampilan Soal 1 Modul 2.....	40
Gambar 14. Screenshot Hasil Tampilan Soal 1 Modul 2.....	40
Gambar 15. Screenshot Hasil Tampilan Soal 1 Modul 2.....	41
Gambar 16. Screenshot Hasil Tampilan Soal 1 Modul 2.....	41
Gambar 17. Screenshot Hasil Tampilan Soal 1 Modul 2.....	41

MODUL 3: Build a Scrollable List

Gambar 18. Contoh UI List Soal 1 Modul 3	50
Gambar 19. Contoh UI Detail Soal 1 Modul 3	50
Gambar 20. Screenshot Hasil Tampilan Soal 1 Modul 3	75
Gambar 21. Screenshot Hasil Tampilan Soal 1 Modul 3	75
Gambar 22. Screenshot Hasil Tampilan Soal 1 Modul 3	76

MODUL 4: ViewModel and Debugging

Gambar 23. Contoh Penggunaan Debugger Soal 1 Modul 4	87
Gambar 24. Screenshot Hasil Tampilan Soal 1 Modul 4	115
Gambar 25. Screenshot Hasil Tampilan Soal 1 Modul 4	115
Gambar 26. Screenshot Hasil Tampilan Soal 1 Modul 4	116
Gambar 27. Screenshot Hasil Tampilan Debugging Soal 1 Modul 4	117
Gambar 28. Screenshot Hasil Tampilan Debugging Step Into Soal 1 Modul 4	117
Gambar 29. Screenshot Hasil Tampilan Debugging Step Over Soal 1 Modul 4	118
Gambar 30. Screenshot Hasil Tampilan Debugging Step Out Soal 1 Modul 4	118

MODUL 5: Connect to the Internet

Gambar 31. Screenshot Hasil Tampilan Soal 1 Modul 5	175
Gambar 32. Screenshot Hasil Tampilan Soal 1 Modul 5	175
Gambar 33. Screenshot Hasil Tampilan Soal 1 Modul 5	176
Gambar 34. Screenshot Hasil Tampilan Soal 1 Modul 5	176

DAFTAR TABEL

MODUL 1: Android Basic with Kotlin

Tabel 1. Source Code Soal 1 Modul 1	12
Tabel 2. Source Code Soal 1 Modul 1	15
Tabel 3. Source Code Soal 1 Modul 1	17

MODUL 2: Android Layout

Tabel 4. Source Code Soal 1 Modul 2	27
Tabel 5. Source Code Soal 1 Modul 2	29
Tabel 6. Source Code Soal 1 Modul 2	34
Tabel 7. Source Code Soal 1 Modul 2	36
Tabel 8. Source Code Soal 1 Modul 2	37
Tabel 9. Source Code Soal 1 Modul 2	37

MODUL 3: Build a Scrollable List

Tabel 10. Source Code Soal 1 Modul 3	51
Tabel 11. Source Code Soal 1 Modul 3	52
Tabel 12. Source Code Soal 1 Modul 3	53
Tabel 13. Source Code Soal 1 Modul 3	57
Tabel 14. Source Code Soal 1 Modul 3	59
Tabel 15. Source Code Soal 1 Modul 3	59
Tabel 16. Source Code Soal 1 Modul 3	60
Tabel 17. Source Code Soal 1 Modul 3	63
Tabel 18. Source Code Soal 1 Modul 3	64
Tabel 19. Source Code Soal 1 Modul 3	67
Tabel 20. Source Code Soal 1 Modul 3	68
Tabel 21. Source Code Soal 1 Modul 3	72

MODUL 4: ViewModel and Debugging

Tabel 22. Source Code Soal 1 Modul 4	88
Tabel 23. Source Code Soal 1 Modul 4	89
Tabel 24. Source Code Soal 1 Modul 4	91
Tabel 25. Source Code Soal 1 Modul 4	95
Tabel 26. Source Code Soal 1 Modul 4	97
Tabel 27. Source Code Soal 1 Modul 4	98
Tabel 28. Source Code Soal 1 Modul 4	99
Tabel 29. Source Code Soal 1 Modul 4	99
Tabel 30. Source Code Soal 1 Modul 4	100
Tabel 31. Source Code Soal 1 Modul 4	103
Tabel 32. Source Code Soal 1 Modul 4	104
Tabel 33. Source Code Soal 1 Modul 4	107
Tabel 34. Source Code Soal 1 Modul 4	107
Tabel 35. Source Code Soal 1 Modul 4	112

MODUL 5: Connect to the Internet

Tabel 36. Source Code Soal 1 Modul 5	133
Tabel 37. Source Code Soal 1 Modul 5	135
Tabel 38. Source Code Soal 1 Modul 5	141
Tabel 39. Source Code Soal 1 Modul 5	142
Tabel 40. Source Code Soal 1 Modul 5	142
Tabel 41. Source Code Soal 1 Modul 5	144
Tabel 42. Source Code Soal 1 Modul 5	145
Tabel 43. Source Code Soal 1 Modul 5	147
Tabel 44. Source Code Soal 1 Modul 5	147
Tabel 45. Source Code Soal 1 Modul 5	149
Tabel 46. Source Code Soal 1 Modul 5	150
Tabel 47. Source Code Soal 1 Modul 5	150

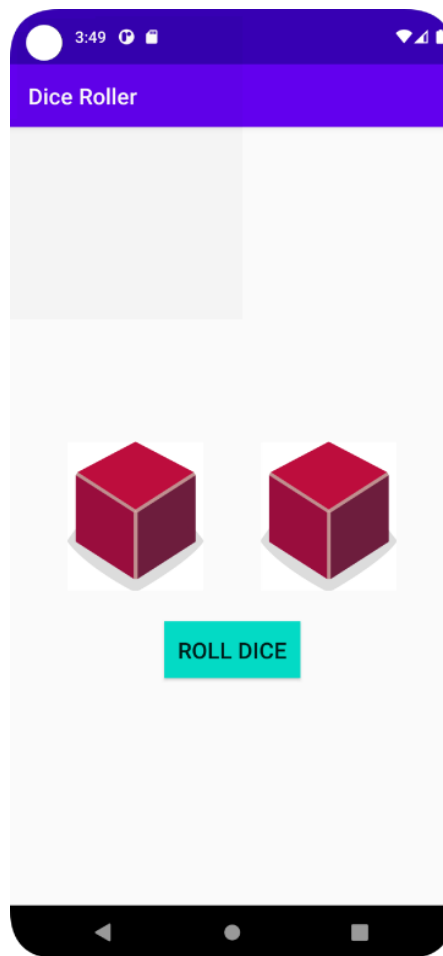
Tabel 48. Source Code Soal 1 Modul 5	153
Tabel 49. Source Code Soal 1 Modul 5	153
Tabel 50. Source Code Soal 1 Modul 5	154
Tabel 51. Source Code Soal 1 Modul 5	156
Tabel 52. Source Code Soal 1 Modul 5	159
Tabel 53. Source Code Soal 1 Modul 5	160
Tabel 54. Source Code Soal 1 Modul 5	161
Tabel 55. Source Code Soal 1 Modul 5	162
Tabel 56. Source Code Soal 1 Modul 5	165
Tabel 57. Source Code Soal 1 Modul 5	168
Tabel 58. Source Code Soal 1 Modul 5	171

MODUL 1: Android Basic with Kotlin

SOAL 1

Buatlah sebuah aplikasi yang dapat menampilkan 2 (dua) buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll Dice”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

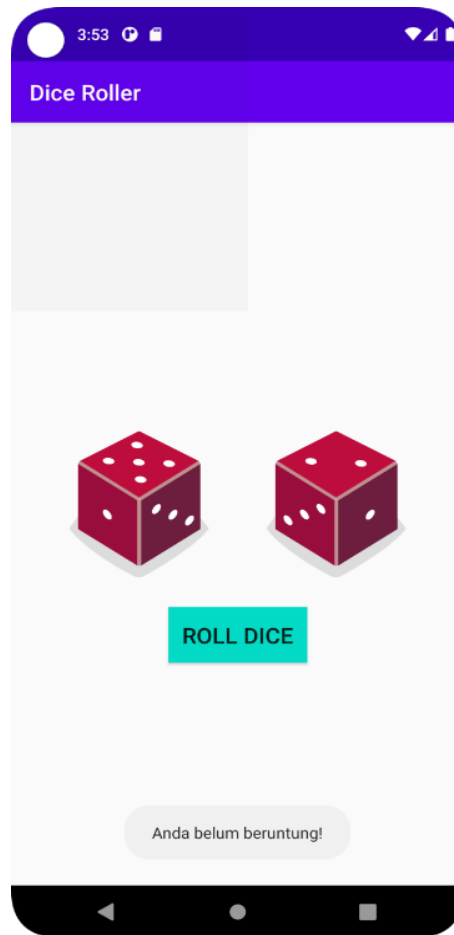
1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



Gambar 1. Tampilan Awal Aplikasi Soal 1 Modul 1

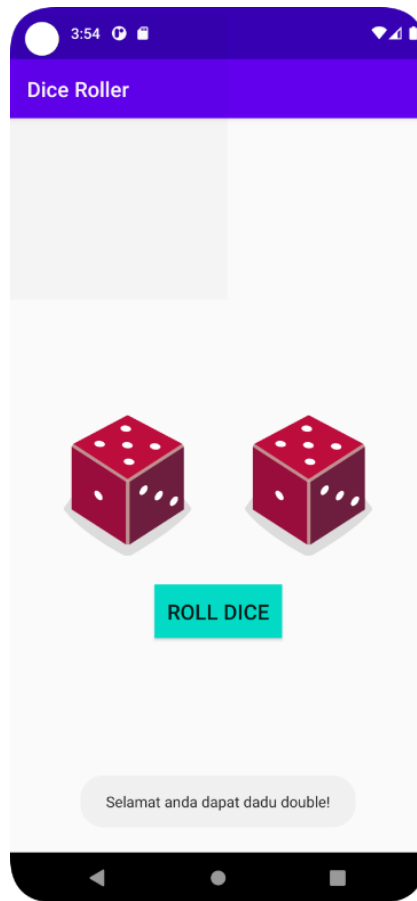
2. Setelah user menekan tombol “Roll Dice” maka masing-masing dadu akan memunculkan sisi dadu masing-masing dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2 maka akan

menampilkan pesan “Anda belum beruntung!” seperti dapat dilihat pada Gambar 2.



Gambar 2. Tampilan Dadu Setelah di Roll Soal 1 Modul 1

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat anda dapat dadu double!” seperti dapat dilihat pada Gambar 3
4. Upload aplikasi yang telah anda buat kedalam repository github ke dalam **folder Module 2 dalam bentuk project**. Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repo
5. Untuk gambar dadu dapat didownload pada link berikut:
https://drive.google.com/u/0/uc?id=147HT2IIH5qin3z5ta7H9y2N_5OMW81Ll&export=download



Gambar 3. Tampilan Roll Dadu Double Soal 1 Modul 1

A. Source Code

Tabel 1. Source Code Soal 1 Modul 1

File MainActivity.kt	
1	package com.example.dice_roll
2	
3	import android.os.Bundle
4	import android.widget.Button
5	import android.widget.ImageView
6	import android.widget.Toast
7	import androidx.activity.enableEdgeToEdge
8	import androidx.appcompat.app.AppCompatActivity

```

9  import androidx.core.view.ViewCompat
10 import androidx.core.view.WindowInsetsCompat
11
12 class MainActivity : AppCompatActivity() {
13     override fun onCreate(savedInstanceState:
Bundle?) {
14         super.onCreate(savedInstanceState)
15         enableEdgeToEdge()
16         setContentView(R.layout.activity_main)
17
18         val rollBtn: Button =
findViewById(R.id.button)
19         val diceImgFirst: ImageView =
findViewById(R.id.dice1)
20         val diceImgSecond: ImageView =
findViewById(R.id.dice2)
21
22 diceImgFirst.setImageResource(R.drawable.empty_dice)
23 diceImgSecond.setImageResource(R.drawable.empty_dice)
24
25         rollBtn.setOnClickListener {
26             rollDice()
27         }
28     }
29
30     class Dice(private val jumlahSisi: Int) {
31         fun roll(): Int {
32             return (1..jumlahSisi).random()
33         }
34     }

```

```

35
36     private fun rollDice() {
37         val dice1 = Dice(6)
38         val dice2 = Dice(6)
39         val diceRoll1 = dice1.roll()
40         val diceRoll2 = dice2.roll()
41
42         val diceImgFirst: ImageView =
43             findViewById(R.id.dice1)
44         val diceImgSecond: ImageView =
45             findViewById(R.id.dice2)
46
47         val drawableResourceFirst = when (diceRoll1) {
48             1 -> R.drawable.dice_1
49             2 -> R.drawable.dice_2
50             3 -> R.drawable.dice_3
51             4 -> R.drawable.dice_4
52             5 -> R.drawable.dice_5
53             else -> R.drawable.dice_6
54         }
55
56         val drawableResourceSecond = when (diceRoll2)
57         {
58             1 -> R.drawable.dice_1
59             2 -> R.drawable.dice_2
60             3 -> R.drawable.dice_3
61             4 -> R.drawable.dice_4
62             5 -> R.drawable.dice_5
63             else -> R.drawable.dice_6
64         }

```

62	
63	<code>diceImgFirst.setImageResource(drawableResourceFirst)</code>
64	<code>diceImgSecond.setImageResource(drawableResourceSecond)</code>
65	
66	<code> diceImgFirst.contentDescription =</code>
	<code>diceRoll1.toString()</code>
67	<code> diceImgSecond.contentDescription =</code>
	<code>diceRoll2.toString()</code>
68	
69	<code> if (diceRoll1 != diceRoll2) {</code>
70	<code> Toast.makeText(this, "Anda belum</code>
	<code>beruntung", Toast.LENGTH_SHORT).show()</code>
71	<code> } else {</code>
72	<code> Toast.makeText(this, "Selamat anda dapat</code>
	<code>dadu double", Toast.LENGTH_SHORT).show()</code>
73	<code> }</code>
74	<code> }</code>
75	<code>}</code>

Tabel 2. Source Code Soal 1 Modul 1

File activity_main.xml	
1	<code><?xml version="1.0" encoding="utf-8"?></code>
2	<code><androidx.constraintlayout.widget.ConstraintLayout</code>
	<code>xmlns:android="http://schemas.android.com/apk/res/andr</code>
	<code>oid"</code>
3	<code> xmlns:app="http://schemas.android.com/apk/res-</code>
	<code>auto"</code>
4	<code> xmlns:tools="http://schemas.android.com/tools"</code>
5	<code> android:id="@+id/main"</code>
6	<code> android:layout_width="match_parent"</code>

7	android:layout_height="match_parent"
8	tools:context=".MainActivity">
9	
10	<TextView
11	android:layout_width="match_parent"
12	android:layout_height="wrap_content"
13	android:text="Dice Roller"
14	android:textSize="30sp"
15	app:layout_constraintEnd_toStartOf="parent"
16	app:layout_constraintTop_toTopOf="parent"
17	android:padding="10dp"
18	android:background="#B43757"/>
19	
20	<ImageView
21	android:id="@+id/dice1"
22	android:layout_width="150dp"
23	android:layout_height="150dp"
24	app:layout_constraintBottom_toBottomOf="parent"
25	app:layout_constraintTop_toTopOf="parent"
26	app:layout_constraintStart_toEndOf="@+id/dice2"
27	app:layout_constraintEnd_toEndOf="parent"
28	app:layout_constraintHorizontal_chainStyle=
	"packed"
29	app:srcCompat="@drawable/empty_dice" />
30	
31	<ImageView
32	android:id="@+id/dice2"
33	android:layout_width="150dp"
34	android:layout_height="150dp"
35	app:layout_constraintBottom_toBottomOf="parent"

36	app:layout_constraintTop_toTopOf="parent"
37	app:layout_constraintStart_toStartOf="parent"
38	app:layout_constraintEnd_toStartOf="@+id/dice1"
39	app:srcCompat="@drawable/empty_dice" />
40	
41	<Button
42	android:id="@+id/button"
43	android:layout_width="wrap_content"
44	android:layout_height="wrap_content"
45	android:text="Roll Dice"
46	app:layout_constraintEnd_toEndOf="parent"
47	app:layout_constraintStart_toStartOf="parent"
48	app:layout_constraintTop_toBottomOf="@+id/dice1"
49	/>
50	</androidx.constraintlayout.widget.ConstraintLayout>

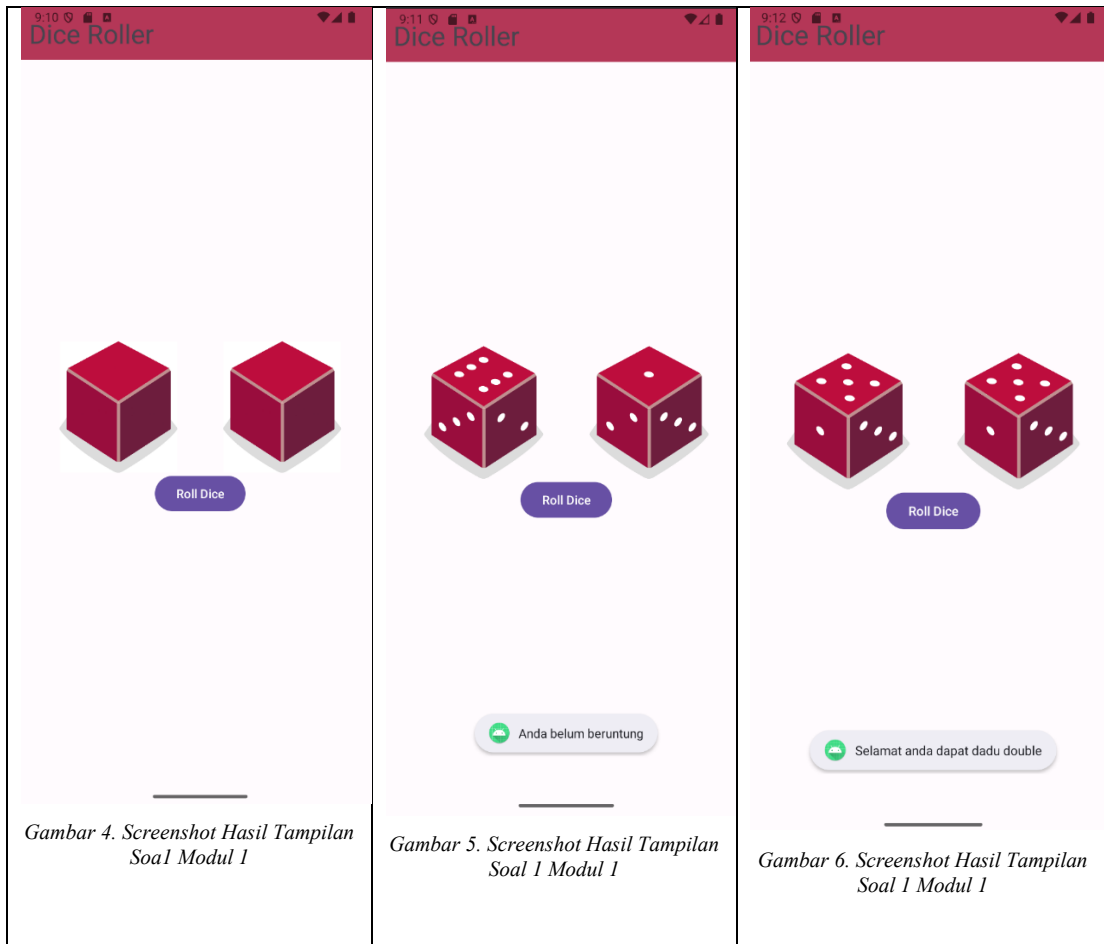
Tabel 3. Source Code Soal 1 Modul 1

File build.gradle.kts	
1	plugins {
2	alias(libs.plugins.android.application)
3	alias(libs.plugins.kotlin.android)
4	}
5	
6	android {
7	namespace = "com.example.dice_roll"
8	compileSdk = 35
9	
10	defaultConfig {
11	applicationId = "com.example.dice_roll"
12	minSdk = 24

13	targetSdk = 35
14	versionCode = 1
15	versionName = "1.0"
16	
17	testInstrumentationRunner =
	"androidx.test.runner.AndroidJUnitRunner"
18	}
19	
20	buildFeatures{
21	viewBinding = true
22	}
23	
24	buildTypes {
25	release {
26	isMinifyEnabled = false
27	proguardFiles(
28	getDefaultProguardFile("proguard-
	android-optimize.txt"),
29	"proguard-rules.pro"
30	)
31	}
32	}
33	compileOptions {
34	sourceCompatibility = JavaVersion.VERSION_11
35	targetCompatibility = JavaVersion.VERSION_11
36	}
37	kotlinOptions {
38	jvmTarget = "11"
39	}
40	}

41	
42	<code>dependencies {</code>
43	
44	<code> implementation(libs.androidx.core.ktx)</code>
45	<code> implementation(libs.androidx.appcompat)</code>
46	<code> implementation(libs.material)</code>
47	<code> implementation(libs.androidx.activity)</code>
48	<code> implementation(libs.androidx.constraintlayout)</code>
49	<code> testImplementation(libs.junit)</code>
50	<code> androidTestImplementation(libs.androidx.junit)</code>
51	<code> androidTestImplementation(libs.androidx.espresso.core)</code>
52	<code>}</code>

B. Output Program



C. Pembahasan

File MainActivity.kt

Kelas utama bernama **MainActivity** yang mewarisi dari **AppCompatActivity**. Di dalam fungsi **onCreate()**, sistem Android akan menyiapkan tampilan dengan memuat layout dari file **activity_main.xml** melalui **setContentView()**. Selain itu, fungsi **enableEdgeToEdge()** digunakan untuk membuat tampilan aplikasi menyesuaikan layar penuh. Selanjutnya, program mengambil referensi dari elemen UI yang ada pada layout, yaitu tombol roll **R.id.button** serta dua gambar dadu **R.id.dice1** dan **R.id.dice2**. Kedua gambar dadu tersebut awalnya diatur untuk menampilkan

gambar dadu kosong **R.drawable.empty_dice**, sebagai tampilan awal sebelum tombol ditekan.

Tombol roll diatur agar memiliki sebuah listener, yaitu dengan menggunakan **rollBtn.setOnClickListener**. Listener ini adalah bagian dari program yang akan menunggu dan merespon saat tombol ditekan oleh pengguna. Ketika tombol tersebut ditekan, listener akan memicu fungsi **rollDice()**. Di dalam fungsi **rollDice()**, pertama-tama dibuat dua objek dari kelas **Dice** yang masing-masing mewakili sebuah dadu dengan enam sisi. Kelas **Dice** sendiri adalah kelas khusus yang dideklarasikan di dalam **MainActivity** dan memiliki fungsi **roll()** untuk menghasilkan angka acak dari 1 sampai 6. Nilai acak dari masing-masing dadu disimpan dalam variabel **diceRoll1** dan **diceRoll2**.

Setelah angka acak didapatkan, langkah berikutnya adalah menyesuaikan gambar dadu yang akan ditampilkan di layar. Hal ini dilakukan dengan struktur kontrol **when**, yang berfungsi mirip seperti **if-else** tetapi lebih ringkas. Jika nilai acak adalah 1, maka akan digunakan gambar **R.drawable.dice_1**, jika nilainya 2 maka **dice_2**, dan seterusnya hingga nilai 6. Gambar ini kemudian ditampilkan dengan **setImageResource()** pada kedua **ImageView**, **diceImgFirst** dan **diceImgSecond**. Tak hanya itu, program juga mengatur **contentDescription** dari masing-masing gambar sesuai dengan angka dadu yang muncul.

Setelah kedua dadu ditampilkan, aplikasi akan memberikan umpan balik kepada pengguna dalam bentuk notifikasi kecil bernama **Toast**. Jika angka dari kedua dadu berbeda, maka aplikasi akan menampilkan pesan “**Anda belum beruntung**”. Namun, jika kedua dadu menunjukkan angka yang sama, maka akan muncul pesan “**Selamat anda dapat dadu double**”.

File activity_main.xml

File layout untuk tampilan utama aplikasi *Dice Roller* yang dibuat menggunakan **ConstraintLayout** sebagai *root layout*. **ConstraintLayout**

adalah jenis layout fleksibel di Android yang memungkinkan pengaturan posisi elemen UI berdasarkan hubungan (*constraint*) antar elemen, baik terhadap elemen lain maupun terhadap parent-nya. File ini menentukan bagaimana komponen visual seperti teks, gambar dadu, dan tombol akan ditampilkan dan diposisikan pada layar pengguna.

Di bagian atas layout terdapat komponen **TextView** yang berfungsi sebagai judul aplikasi. **TextView** ini memiliki **match_parent** (lebar penuh), **wrap_content** (tinggi sesuai konten), dan teks bertuliskan "**Dice Roller**" dengan ukuran huruf **30sp**. Komponen ini diberi padding sebesar **10dp** agar teks tidak terlalu mepet ke pinggir, serta latar belakang berwarna merah muda gelap (**#B43757**).

Menggunakan **app:layout_constraintEnd_toEndOf="parent"** agar teks bisa benar-benar terpusat secara horizontal.

Berikutnya terdapat dua komponen **ImageView** yang masing-masing digunakan untuk menampilkan gambar dadu. **ImageView** dengan ID **dice1** dan **dice2** masing-masing diberi ukuran tetap sebesar 150dp x 150dp. Keduanya diatur agar berada di tengah layar secara vertikal dengan menggunakan **app:layout_constraintTop_toTopOf="parent"** dan **Bottom_toBottomOf="parent"**). Posisi horizontal kedua gambar dadu ini diatur saling berdampingan menggunakan constraint: **dice2** berada di kiri **Start_toStartOf="parent"**, **End_toStartOf="@+id/dice1"**, sedangkan **dice1** di sebelah kanannya **Start_toEndOf="@+id/dice2"**, **End_toEndOf="parent"**. Mereka juga berada dalam sebuah *horizontal chain* dengan gaya **app:layout_constraintHorizontal_chainStyle="packed"**, yang membuat keduanya saling rapat dan terpusat di tengah layar.

Di bawah kedua gambar dadu, terdapat komponen **Button** dengan ID **button**. Tombol ini memiliki ukuran lebar dan tinggi yang mengikuti isi teks **wrap_content** dan berfungsi sebagai pemicu untuk melempar dadu. Tombol

diberi teks “Roll Dice” dan diposisikan di bawah **dice1** menggunakan constraint **app:layout_constraintTop_toBottomOf="@+id/dice1"**, serta disejajarkan secara horizontal di tengah layar dengan menggunakan **Start_toStartOf="parent"** dan **End_toEndOf="parent"**.

Kombinasi antara **ConstraintLayout**, **ImageView**, **TextView**, dan **Button** ini menyusun struktur visual dasar aplikasi pelempar dadu secara efisien dan responsif.

File **build.gradle.kts**

Pada file **build.gradle.kts** yang ditampilkan adalah berkas konfigurasi utama untuk proyek Android berbasis Kotlin menggunakan Gradle dengan DSL (*Domain Specific Language*) Kotlin. File ini digunakan untuk mendefinisikan bagaimana proyek akan dibangun, termasuk plugin yang digunakan, versi SDK Android, fitur *build*, dan dependensi *library* yang dibutuhkan dalam pengembangan aplikasi.

Bagian pertama dari file ini adalah deklarasi plugin, di mana proyek ini menggunakan dua plugin utama: **android.application** dan **kotlin.android**. Plugin **android.application** digunakan untuk menandai bahwa proyek ini adalah sebuah aplikasi Android, bukan *library*. Sementara itu, plugin **kotlin.android** menambahkan dukungan Kotlin ke dalam proyek Android, sehingga pengembangan dapat dilakukan menggunakan bahasa Kotlin. Penulisan **alias(libs.plugins...)** menunjukkan bahwa proyek ini menggunakan fitur *Version Catalog*, yang memungkinkan pengelolaan versi plugin dan dependensi secara terpusat melalui file **libs.versions.toml**.

Selanjutnya, blok **android** berisi konfigurasi utama proyek. Properti **namespace** menentukan *namespace* atau ruang nama dasar yang akan digunakan oleh aplikasi, umumnya mengikuti format nama paket. Kemudian **compileSdk** diatur ke **versi 35**, yang berarti aplikasi ini dikompilasi menggunakan **Android API**

level 35, memberikan akses terhadap fitur-fitur terbaru dari Android versi tersebut. Di dalam **defaultConfig**, terdapat beberapa pengaturan penting seperti **applicationId** yang menjadi identitas unik aplikasi di perangkat dan di *Google Play Store*. **minSdk** diatur ke 24, artinya aplikasi ini hanya dapat diinstal di perangkat dengan **Android Nougat (7.0) ke atas**. **targetSdk** ditetapkan ke 35 agar aplikasi ini dioptimalkan untuk versi Android terbaru. Selain itu, **versionCode** dan **versionName** digunakan untuk pengelolaan versi aplikasi, di mana **versionCode** bersifat **numerik** dan **dibutuhkan saat proses update di Google Play**, sedangkan **versionName** bersifat **deskriptif** dan **ditampilkan kepada pengguna**. **testInstrumentationRunner** menentukan runner untuk pengujian instrumentasi Android.

Pada bagian **buildFeatures**, fitur **viewBinding** diaktifkan. **ViewBinding** adalah fitur yang memungkinkan pengembang untuk lebih mudah mengakses elemen antarmuka pengguna (UI) tanpa harus menggunakan **findViewById**, sehingga lebih aman dari kesalahan *runtime* dan meningkatkan produktivitas. Kemudian di blok **buildTypes**, terdapat konfigurasi untuk tipe build **release**, yang biasanya digunakan saat aplikasi akan dipublikasikan. Di sini, **isMinifyEnabled** diatur ke **false**, artinya proses minifikasi kode tidak diaktifkan, yang umum dilakukan pada tahap awal pengembangan atau untuk aplikasi kecil. File **proguardFiles** menunjuk ke konfigurasi *default ProGuard* dan file tambahan **proguard-rules.pro** untuk aturan khusus.

Bagian **compileOptions** dan **kotlinOptions** memastikan bahwa proyek menggunakan Java 11 untuk kompatibilitas sumber dan target **bytecode**, dan juga mengarahkan Kotlin untuk menggunakan JVM target versi 11.

Terakhir, bagian **dependencies** mencantumkan semua *library* yang digunakan dalam aplikasi. Di antaranya adalah **androidx.core.ktx**, **appcompat**, **material**, **activity**, dan **constraintlayout**, yang merupakan *library* penting untuk pengembangan antarmuka dan fungsi dasar aplikasi Android. Untuk pengujian, digunakan **junit** untuk unit *testing* standar,

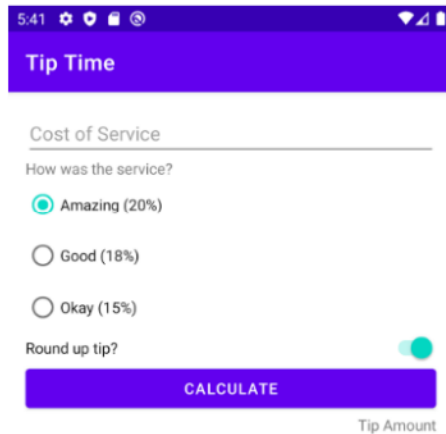
serta **androidx.junit** dan **espresso.core** untuk *instrumented testing* yang berjalan langsung di perangkat atau emulator. Semua dependensi ini juga ditulis menggunakan **libs.xxx**, menunjukkan bahwa proyek mengadopsi pendekatan *Version Catalog* yang lebih modern dan terstruktur dalam pengelolaan dependensi.

MODUL 2: Android Layout

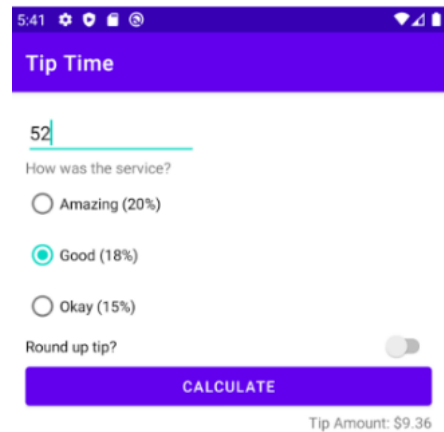
SOAL 1

Buatlah sebuah aplikasi kalkulator tip yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

1. Input Biaya Layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
2. Pilihan Persentase Tip: Pengguna dapat memilih persentase tip yang diinginkan dari opsi yang disediakan, yaitu 15%, 18%, dan 20%.
3. Pengaturan Pembulatan Tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
4. Tampilan Hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input:



Gambar 7. Tampilan Awal Aplikasi Soal 1 Modul 2



Gambar 8. Tampilan Aplikasi Setelah Dijalankan Soal 1 Modul 2

A. Source Code

Tabel 4. Source Code Soal 1 Modul 2

File MainActivity.kt	
1	package com.example.tip_time
2	
3	import android.os.Bundle
4	import android.widget.Toast
5	import androidx.appcompat.app.AppCompatActivity
6	import androidx.core.view.WindowCompat
7	import com.example.tip_time.databinding.ActivityMainBinding
8	import java.text.NumberFormat
9	import kotlin.math.ceil
10	
11	class MainActivity : AppCompatActivity() {
12	
13	private lateinit var binding: ActivityMainBinding
14	
15	override fun onCreate(savedInstanceState: Bundle?) {
16	super.onCreate(savedInstanceState)
17	
18	WindowCompat.setDecorFitsSystemWindows(window, false)
19	
20	binding = ActivityMainBinding.inflate(layoutInflater)
21	setContentView(binding.root)
22	
23	binding.btnCalculate.setOnClickListener { calculateTip() }

```

24     }
25
26     private fun calculateTip() {
27         val stringInTextField =
binding.etCostOfService.text.toString()
28         val cost = stringInTextField.toDoubleOrNull()
29         if (cost == null || cost == 0.0 ) {
30             binding.etCostOfService.error = "Please
enter a valid amount"
31             Toast.makeText(this, "Invalid input.
Please enter a number.", Toast.LENGTH_SHORT).show()
32             binding.tvTipAmount.text = ""
33             return
34         }
35
36         val tipPercentage = when
(binding.rgTipOptions.checkedRadioButtonId) {
37             R.id.rb_amazing -> 0.20
38             R.id.rb_good -> 0.18
39             else -> 0.15
40         }
41
42         var tip = tipPercentage * cost
43         if (binding.switchRoundUp.isChecked) {
44             tip = ceil(tip)
45         }
46
47         val formattedTip =
NumberFormat.getCurrencyInstance().format(tip)
48         binding.tvTipAmount.text =

```

	getString(R.string.tip_amount, formattedTip)
49	}
50	}

Tabel 5. Source Code Soal 1 Modul 2

File activity_main.xml	
1	<?xml version="1.0" encoding="utf-8"?>
2	<ScrollView
	xmlns:android="http://schemas.android.com/apk/res/a
	ndroid"
3	xmlns:app="http://schemas.android.com/apk/res-
	auto"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	android:background="#EBE6F3"
7	android:paddingTop="?attr/actionBarSize"
8	android:padding="16dp">
9	
10	<androidx.constraintlayout.widget.ConstraintLayout
11	android:layout_width="match_parent"
12	android:layout_height="wrap_content">
13	
14	<TextView
15	android:id="@+id/titleText"
16	android:layout_width="match_parent"
17	android:layout_height="wrap_content"
18	android:background="#8000FF"
19	android:padding="16dp"
20	android:text="Tip Time"
21	android:textColor="#FFFFFF"

22	android:textSize="24sp"
23	android:textStyle="bold"
24	app:layout_constraintEnd_toEndOf="parent"
25	app:layout_constraintStart_toStartOf="parent"
26	app:layout_constraintTop_toTopOf="parent"
27	" />
28	<EditText
29	android:id="@+id/et_cost_of_service"
30	android:layout_width="0dp"
31	android:layout_height="wrap_content"
32	android:hint="Cost of Service"
33	android:inputType="numberDecimal"
34	android:textSize="18sp"
35	android:layout_marginTop="24dp"
36	android:background="@android:drawable/edit_text"
37	android:padding="8dp"
38	app:layout_constraintTop_toBottomOf="@id/titleText"
39	app:layout_constraintStart_toStartOf="parent"
40	app:layout_constraintEnd_toEndOf="parent"
41	/>
42	<TextView
43	android:id="@+id/textViewHowWasService"
44	android:layout_width="wrap_content"

45	android:layout_height="wrap_content"
46	android:text="How was the service?"
47	android:textSize="16sp"
48	android:textColor="#999999"
49	android:layout_marginTop="16dp"
50	app:layout_constraintTop_toBottomOf="@i
	d/et_cost_of_service"
51	app:layout_constraintStart_toStartOf="p
	arent" />
52	
53	<RadioGroup
54	android:id="@+id/rg_tip_options"
55	android:layout_width="0dp"
56	android:layout_height="wrap_content"
57	android:orientation="vertical"
58	app:layout_constraintTop_toBottomOf="@i
	d/textViewHowWasService"
59	app:layout_constraintStart_toStartOf="p
	arent"
60	app:layout_constraintEnd_toEndOf="parent">
61	
62	<RadioButton
63	android:id="@+id/rb_amazing"
64	android:layout_width="wrap_content"
65	android:layout_height="wrap_content"
66	android:text="Amazing (20%) "
67	android:textColor="@color/black"/>
68	
69	<RadioButton
70	android:id="@+id/rb_good"

71	android:layout_width="wrap_content"
72	android:layout_height="wrap_content"
73	android:text="Good (18%) "
74	android:textColor="@color/black"/>
75	
76	<RadioButton
77	android:id="@+id/rb_okay"
78	android:layout_width="wrap_content"
79	android:layout_height="wrap_content"
80	android:text="Okay (15%) "
81	android:textColor="@color/black"/>
82	</RadioGroup>
83	
84	<TextView
85	android:id="@+id/textViewRoundUp"
86	android:layout_width="0dp"
87	android:layout_height="wrap_content"
88	android:text="Round up tip?"
89	android:textSize="16sp"
90	app:layout_constraintTop_toBottomOf="@i
	d/rg_tip_options"
91	app:layout_constraintStart_toStartOf="p
	arent"
92	app:layout_constraintEnd_toStartOf="@id
	/switch_round_up"
93	app:layout_constraintHorizontal_bias="0"
94	android:layout_marginTop="8dp" />
95	
96	<Switch
97	android:id="@+id/switch_round_up"

98	android:layout_width="wrap_content"
99	android:layout_height="wrap_content"
100	android:checked="false"
101	android:thumbTint="@drawable/switch_thu
	mb_color"
102	android:trackTint="@drawable/switch_tra
	ck_color"
103	app:layout_constraintTop_toTopOf="@id/t
	extViewRoundUp"
104	app:layout_constraintBottom_toBottomOf=
	"@id/textViewRoundUp"
105	app:layout_constraintEnd_toEndOf="parent"
	/>
106	
107	<Button
108	android:id="@+id/btn_calculate"
109	android:layout_width="0dp"
110	android:layout_height="wrap_content"
111	android:text="CALCULATE"
112	android:textColor="#FFFFFF"
113	android:layout_marginTop="16dp"
114	app:layout_constraintTop_toBottomOf="@i
	d/textViewRoundUp"
115	app:layout_constraintStart_toStartOf="p
	arent"
116	app:layout_constraintEnd_toEndOf="parent"/>
117	
118	<TextView
119	android:id="@+id/tv_tip_amount"
120	android:layout_width="wrap_content"

121	android:layout_height="wrap_content"
122	android:text="Tip Amount: \$0.00"
123	android:textSize="16sp"
124	android:textColor="#888888"
125	android:layout_marginTop="16dp"
126	app:layout_constraintTop_toBottomOf="@i
	d/btn_calculate"
127	app:layout_constraintEnd_toEndOf="parent"
	/>
128	
129	</androidx.constraintlayout.widget.ConstraintLayout
	>
130	</ScrollView>

Tabel 6. Source Code Soal 1 Modul 2

File build.gradle.kts (:app)	
1	plugins {
2	alias(libs.plugins.android.application)
3	alias(libs.plugins.kotlin.android)
4	}
5	
6	android {
7	namespace = "com.example.tip_time"
8	compileSdk = 35
9	
10	defaultConfig {
11	applicationId = "com.example.dice_roll"
12	minSdk = 24
13	targetSdk = 35
14	versionCode = 1

```

15         versionName = "1.0"
16
17         testInstrumentationRunner =
18             "androidx.test.runner.AndroidJUnitRunner"
19     }
20     buildFeatures{
21         viewBinding = true
22     }
23
24     buildTypes {
25         release {
26             isMinifyEnabled = false
27             proguardFiles(
28                 getDefaultProguardFile("proguard-
29 android-optimize.txt"),
30                 "proguard-rules.pro"
31             )
32         }
33         compileOptions {
34             sourceCompatibility = JavaVersion.VERSION_11
35             targetCompatibility = JavaVersion.VERSION_11
36         }
37         kotlinOptions {
38             jvmTarget = "11"
39         }
40     }
41
42     dependencies {

```

43	<code>implementation(libs.androidx.core.ktx)</code>
44	<code>implementation(libs.androidx.appcompat)</code>
45	<code>implementation(libs.material)</code>
46	<code>implementation(libs.androidx.activity)</code>
47	<code>implementation(libs.androidx.constraintlayout)</code>
48	<code>testImplementation(libs.junit)</code>
49	<code>androidTestImplementation(libs.androidx.junit)</code>
50	<code>androidTestImplementation(libs.androidx.espresso.core</code>
	<code>)</code>
51	<code>}</code>

Tabel 7. Source Code Soal 1 Modul 2

File string.xml	
1	<code><resources></code>
2	<code> <string name="app_name">Tip_Time</string></code>
3	<code> <string name="cost_of_service">Cost of</code>
	<code>Service</string></code>
4	<code> <string name="how_was_service">How was the</code>
	<code>service?</string></code>
5	<code> <string name="amazing">Amazing (20%)</string></code>
6	<code> <string name="good">Good (18%)</string></code>
7	<code> <string name="okay">Okay (15%)</string></code>
8	<code> <string name="round_up_tip">Round up</code>
	<code>tip?</string></code>
9	<code> <string name="calculate">CALCULATE</string></code>
10	<code> <string name="tip_amount">Tip Amount: %s</string></code>
11	<code> <string name="default_tip_amount">Tip Amount:</code>
	<code>\$0.00</string></code>
12	<code></resources></code>

Tabel 8. Source Code Soal 1 Modul 2

File switch_track_color.xml	
1	<?xml version="1.0" encoding="utf-8"?>
2	<selector xmlns:android="http://schemas.android.com/apk/res/and roid">
3	<item android:color="#8000FF" android:state_checked="true"/>
4	<item android:color="#D1B3FF"/>
5	</selector>

Tabel 9. Source Code Soal 1 Modul 2

File switch_thumb_color.xml	
1	<?xml version="1.0" encoding="utf-8"?>
2	<selector xmlns:android="http://schemas.android.com/apk/res/and roid">
3	<item android:color="#62339C" android:state_checked="true"/>
4	<item android:color="#D1B3FF"/>
5	</selector>

B. Output Program

The screenshot shows the 'Tip Time' app interface. At the top is a purple header with the text 'Tip Time'. Below it is a white input field labeled 'Cost of Service'. Underneath the input field is the question 'How was the service?' followed by three radio button options: 'Amazing (20%)', 'Good (18%)', and 'Okay (15%)'. Below these options is a toggle switch for 'Round up tip?'. At the bottom of the form is a purple button labeled 'CALCULATE'. Below the button, the text 'Tip Amount: \$0.00' is displayed.

Gambar 9. Screenshot Hasil Tampilan Soal 1 Modul 2

This screenshot shows the 'Tip Time' app after an invalid input. The 'Cost of Service' input field is highlighted with a red border and a red exclamation mark icon. A black tooltip with the text 'Please enter a valid amount' is visible next to the input field. The 'Amazing (20%)' radio button is now selected. The 'CALCULATE' button remains at the bottom. At the very bottom of the screen, a green error message bubble says 'Invalid input. Please enter a number.'

Gambar 10. Screenshot Hasil Tampilan Soal 1 Modul 2

Tip Time

52

How was the service?

☒ Amazing (20%)

☐ Good (18%)

☐ Okay (15%)

Round up tip? ☐

CALCULATE

Tip Amount: \$10.40

Gambar 11. Screenshot Hasil Tampilan Soal 1 Modul 2

Tip Time

52

How was the service?

☒ Amazing (20%)

☐ Good (18%)

☐ Okay (15%)

Round up tip? ☒

CALCULATE

Tip Amount: \$11.00

Gambar 12. Screenshot Hasil Tampilan Soal 1 Modul 2

The screenshot shows the 'Tip Time' app interface. At the top, there is a purple header with the text 'Tip Time'. Below the header is a white input field with an orange border containing the number '52'. Underneath the input field is the text 'How was the service?'. There are three radio button options: 'Amazing (20%)', 'Good (18%)', and 'Okay (15%)'. The 'Good (18%)' option is selected. Below the radio buttons is a toggle switch for 'Round up tip?' which is currently turned off. At the bottom of the form is a purple button labeled 'CALCULATE'. Below the button, the text 'Tip Amount: \$9.36' is displayed.

Gambar 13. Screenshot Hasil Tampilan Soal 1 Modul 2

The screenshot shows the 'Tip Time' app interface. At the top, there is a purple header with the text 'Tip Time'. Below the header is a white input field with an orange border containing the number '52'. Underneath the input field is the text 'How was the service?'. There are three radio button options: 'Amazing (20%)', 'Good (18%)', and 'Okay (15%)'. The 'Good (18%)' option is selected. Below the radio buttons is a toggle switch for 'Round up tip?' which is currently turned on. At the bottom of the form is a purple button labeled 'CALCULATE'. Below the button, the text 'Tip Amount: \$10.00' is displayed.

Gambar 14. Screenshot Hasil Tampilan Soal 1 Modul 2

5:29

Tip Time

52

How was the service?

☐ Amazing (20%)

☐ Good (18%)

☒ Okay (15%)

Round up tip? ☐

CALCULATE

Tip Amount: \$7.80

Gambar 15. Screenshot Hasil Tampilan Soal 1 Modul 2

5:31

Tip Time

52

How was the service?

☐ Amazing (20%)

☐ Good (18%)

☒ Okay (15%)

Round up tip? ☒

CALCULATE

Tip Amount: \$8.00

Gambar 16. Screenshot Hasil Tampilan Soal 1 Modul 2

5:31

Tip Time

52

How was the service?

☒ Amazing (20%)

☐ Good (18%)

☐ Okay (15%)

Round up tip? ☒

CALCULATE

Tip Amount: \$10.40

Gambar 17. Screenshot Hasil Tampilan Soal 1 Modul 2

C. Pembahasan File MainActivity.kt

Kelas utama **MainActivity** mewarisi **AppCompatActivity**, yang merupakan kelas dasar untuk aktivitas yang mendukung fitur-fitur dari Android. Di dalam metode **onCreate()**, aktivitas menginisialisasi tampilan dengan memanfaatkan **ActivityMainBinding**, yaitu bagian dari mekanisme **View Binding**. Mekanisme ini memungkinkan kita untuk berinteraksi dengan elemen UI secara langsung tanpa perlu memanggil **findViewById**, sehingga membuat kode lebih aman dari kesalahan tipe dan lebih mudah dibaca.

Pemanggilan

WindowCompat.setDecorFitsSystemWindows(window, false) berfungsi untuk mengatur apakah tata letak aktivitas akan menyesuaikan dengan batas sistem. Jika mengatur ke **false**, konten aplikasi dapat meluas ke seluruh layar, yang biasanya digunakan untuk tampilan imersif atau mendukung desain penuh layar.

Setelah tampilan aktivitas diatur, sebuah **OnClickListener** dipasang ke tombol **btnCalculate** melalui **binding.btnCalculate.setOnClickListener { calculateTip() }**. Ini berarti ketika tombol ditekan, fungsi **calculateTip()** akan dijalankan. Fungsi inilah yang berisi logika utama dari aplikasi penghitung tip.

Di dalam **calculateTip()**, teks dari **EditText (etCostOfService)** diambil dan diubah menjadi **Double** menggunakan **toDoubleOrNull()**, yang akan menghasilkan **null** jika konversi gagal. Jika nilai input tidak valid, maka dua hal seperti **EditText** akan menampilkan error menggunakan **setError()** dengan pesan "**Please enter a valid amount**", dan sebuah **Toast** akan muncul, memberi tahu pengguna bahwa input tidak valid.

Selain itu, jika input tidak valid, tampilan **tvTipAmount** dikosongkan agar tidak menampilkan hasil yang menyesatkan dari input sebelumnya. Jika input valid, aplikasi menentukan persentase tip berdasarkan tombol radio yang dipilih yaitu,

R.id.rb_amazing untuk 20%, **R.id.rb_good** untuk 18%, dan **R.id.rb_okay** untuk 15%.

Setelah persentase tip ditentukan, aplikasi mengalikan nilai **cost** dengan persentase tip untuk mendapatkan jumlah tip. Jika pengguna mencentang atau menyalakan **switchRoundUp**, maka nilai tip dibulatkan ke atas menggunakan fungsi **ceil()** dari Kotlin yang digunakan untuk membulatkan angka desimal ke bilangan bulat terdekat ke atas.

Terakhir, hasil tip diformat ke dalam bentuk mata uang lokal menggunakan **NumberFormat.getCurrencyInstance().format(tip)**. Hasil ini kemudian ditampilkan pada **TextView (tvTipAmount)** menggunakan **getString()** dengan referensi ke string di **strings.xml** yang memiliki format seperti **"Tip amount: %s"**

File activity_main.xml

Layout aplikasi dibungkus oleh elemen **ScrollView** agar tampilan bisa digulir secara vertikal ketika konten melebihi tinggi layar. **ScrollView** ini memiliki lebar dan tinggi yang disesuaikan dengan ukuran layar **match_parent**, dengan latar belakang berwarna **#EBE6F3**. Padding atas ditentukan berdasarkan tinggi **ActionBar**, sedangkan padding umum diatur sebesar **16dp** untuk menjaga jarak antara konten dan tepi layar. Di dalam **ScrollView**, digunakan **ConstraintLayout** sebagai kontainer utama yang fleksibel dalam mengatur posisi elemen UI berdasarkan constraint antar elemen.

Komponen pertama adalah **TextView** dengan ID **titleText** yang berfungsi sebagai judul aplikasi. Komponen ini menampilkan teks **"Tip Time"** dengan gaya tebal, ukuran font **24sp**, latar belakang **#8000FF**, dan teks berwarna putih. Elemen ini direntangkan dari kiri ke kanan dan diletakkan di bagian atas layout. Di bawahnya terdapat **EditText** dengan ID **et_cost_of_service** sebagai tempat pengguna menginput biaya layanan. **EditText** ini memiliki *hint* **"Cost of Service"**, menerima input angka desimal, dan menggunakan latar belakang

bawaan Android dengan padding **8dp**. Komponen ini juga diatur agar lebarnya mengikuti constraint kiri dan kanan serta memiliki margin atas sebesar **24dp**.

Selanjutnya, terdapat **TextView** dengan ID **textViewHowWasService** yang memberikan petunjuk kepada pengguna dengan teks “**How was the service?**” berwarna abu-abu terang. Di bawah petunjuk ini terdapat **RadioGroup** dengan ID **rg_tip_options** yang berisi tiga **RadioButton**, yaitu **rb_amazing**, **rb_good**, dan **rb_okay**, yang masing-masing menampilkan label “**Amazing (20%)**”, “**Good (18%)**”, dan “**Okay (15%)**” dengan warna teks hitam. Komponen ini digunakan untuk memilih tingkat kepuasan pengguna terhadap layanan.

Setelah itu, terdapat pasangan **TextView** dan **Switch** yang digunakan untuk opsi pembulatan tip. **TextView** dengan ID **textViewRoundUp** menampilkan pertanyaan “**Round up tip?**”, sementara **Switch** dengan ID **switch_round_up** memungkinkan pengguna mengaktifkan atau menonaktifkan pembulatan ke atas. *Switch* ini menggunakan warna **thumb** dan **track** yang ditentukan oleh file **drawable** khusus **switch_thumb_color** dan **switch_track_color**. Kedua komponen ini dirancang agar tampil sejajar secara horizontal.

Berikutnya adalah **Button** dengan ID **btn_calculate** yang berfungsi untuk memicu perhitungan tip ketika ditekan. Tombol ini memiliki teks “**CALCULATE**” dengan warna putih, margin atas **16dp**, dan direntangkan secara horizontal di bawah elemen *switch*. Terakhir, **TextView** dengan ID **tv_tip_amount** digunakan untuk menampilkan hasil perhitungan tip dengan teks default “**Tip Amount: \$0.00**”, teks berwarna abu-abu, dan diletakkan di bawah tombol kalkulasi.

File build.gradle.kts (:app)

Pada file **build.gradle.kts (:app)** yang ditampilkan adalah berkas konfigurasi utama untuk proyek Android berbasis Kotlin menggunakan Gradle

dengan DSL (*Domain Specific Language*) Kotlin. File ini digunakan untuk mendefinisikan bagaimana proyek akan dibangun, termasuk plugin yang digunakan, versi SDK Android, fitur *build*, dan dependensi *library* yang dibutuhkan dalam pengembangan aplikasi.

Bagian pertama dari file ini adalah deklarasi plugin, di mana proyek ini menggunakan dua plugin utama: **android.application** dan **kotlin.android**. Plugin **android.application** digunakan untuk menandai bahwa proyek ini adalah sebuah aplikasi Android, bukan *library*. Sementara itu, plugin **kotlin.android** menambahkan dukungan Kotlin ke dalam proyek Android, sehingga pengembangan dapat dilakukan menggunakan bahasa Kotlin. Penulisan **alias(libs.plugins...)** menunjukkan bahwa proyek ini menggunakan fitur *Version Catalog*, yang memungkinkan pengelolaan versi plugin dan dependensi secara terpusat melalui file **libs.versions.toml**.

Selanjutnya, blok **android** berisi konfigurasi utama proyek. Properti **namespace** menentukan *namespace* atau ruang nama dasar yang akan digunakan oleh aplikasi, umumnya mengikuti format nama paket. Kemudian **compileSdk** diatur ke **versi 35**, yang berarti aplikasi ini dikompilasi menggunakan **Android API level 35**, memberikan akses terhadap fitur-fitur terbaru dari Android versi tersebut. Di dalam **defaultConfig**, terdapat beberapa pengaturan penting seperti **applicationId** yang menjadi identitas unik aplikasi di perangkat dan di *Google Play Store*. **minSdk** diatur ke 24, artinya aplikasi ini hanya dapat diinstal di perangkat dengan **Android Nougat (7.0) ke atas**. **targetSdk** ditetapkan ke 35 agar aplikasi ini dioptimalkan untuk versi Android terbaru. Selain itu, **versionCode** dan **versionName** digunakan untuk pengelolaan versi aplikasi, di mana **versionCode** bersifat **numerik** dan **dibutuhkan saat proses update di Google Play**, sedangkan **versionName** bersifat **deskriptif** dan **ditampilkan kepada pengguna**. **testInstrumentationRunner** menentukan runner untuk pengujian instrumentasi Android.

Pada bagian **buildFeatures**, fitur **viewBinding** diaktifkan. **ViewBinding** adalah fitur yang memungkinkan pengembang untuk lebih mudah mengakses elemen antarmuka pengguna (UI) tanpa harus menggunakan **findViewById**, sehingga lebih aman dari kesalahan *runtime* dan meningkatkan produktivitas. Kemudian di blok **buildTypes**, terdapat konfigurasi untuk tipe build **release**, yang biasanya digunakan saat aplikasi akan dipublikasikan. Di sini, **isMinifyEnabled** diatur ke **false**, artinya proses minifikasi kode tidak diaktifkan, yang umum dilakukan pada tahap awal pengembangan atau untuk aplikasi kecil. File **proguardFiles** menunjuk ke konfigurasi *default ProGuard* dan file tambahan **proguard-rules.pro** untuk aturan khusus.

Bagian **compileOptions** dan **kotlinOptions** memastikan bahwa proyek menggunakan Java 11 untuk kompatibilitas sumber dan target *bytecode*, dan juga mengarahkan Kotlin untuk menggunakan JVM target versi 11.

Terakhir, bagian **dependencies** mencantumkan semua *library* yang digunakan dalam aplikasi. Di antaranya adalah **androidx.core.ktx**, **appcompat**, **material**, **activity**, dan **constraintlayout**, yang merupakan *library* penting untuk pengembangan antarmuka dan fungsi dasar aplikasi Android. Untuk pengujian, digunakan **junit** untuk unit *testing* standar, serta **androidx.junit** dan **espresso.core** untuk *instrumented testing* yang berjalan langsung di perangkat atau emulator. Semua dependensi ini juga ditulis menggunakan **libs.xxx**, menunjukkan bahwa proyek mengadopsi pendekatan *Version Catalog* yang lebih modern dan terstruktur dalam pengelolaan dependensi.

File String.xml

Setiap elemen dalam file ini didefinisikan menggunakan tag **<string>** dengan atribut **name** yang unik, yang nantinya akan digunakan sebagai referensi dari file layout XML atau file Kotlin/Java. **<string**

`name="app_name">Tip_Time</string>` menetapkan nama aplikasi sebagai "Tip_Time".

String seperti `"cost_of_service"`, `"how_was_service"`, `"amazing"`, `"good"`, dan `"okay"` merupakan label teks yang digunakan dalam tampilan antarmuka pengguna (UI), yang masing-masing ditampilkan sebagai petunjuk input atau label untuk tombol radio.

Selanjutnya, string `"round_up_tip"` dan `"calculate"` masing-masing digunakan sebagai label untuk opsi *switch* dan tombol kalkulasi. String `"tip_amount"` menggunakan format string `%s` yang akan digantikan oleh nilai dinamis saat aplikasi berjalan, yaitu jumlah tip yang telah diformat sebagai mata uang. Ini memungkinkan penggunaan satu string *template* yang fleksibel dan dapat disesuaikan selama *runtime*. Adapun string `"default_tip_amount"` berisi teks default yang ditampilkan saat belum ada perhitungan dilakukan, yaitu **"Tip Amount: \$0.00"**.

File `switch_track_color.xml`

Elemen `<selector>`, memungkinkan pengembang untuk mendefinisikan warna yang berbeda tergantung pada kondisi tertentu dari elemen tersebut, misalnya saat elemen sedang dipilih (*checked*), difokuskan, ditekan, atau dalam keadaan normal.

Terdapat dua elemen `<item>`. Elemen pertama memiliki atribut `android:state_checked="true"`, artinya kondisi ini berlaku ketika elemen UI berada dalam keadaan terpilih (*checked*). Jika kondisi ini terpenuhi, maka warna yang digunakan adalah **#8000FF**, yaitu warna ungu tua. Warna ini akan terlihat, misalnya, pada teks atau ikon `RadioButton` yang dipilih, tergantung pada di mana *color selector* ini diterapkan.

Elemen kedua bertindak sebagai *default* atau *fallback*. Artinya, ketika elemen tidak dalam keadaan *checked*, maka warna yang digunakan adalah **#D1B3FF**, yaitu

ungu muda. Ini memberikan efek visual bahwa elemen yang tidak aktif tampil lebih lembut, sedangkan elemen aktif tampil lebih mencolok.

File switch_thumb_color.xml

Elemen **<selector>**, memungkinkan pengembang untuk mendefinisikan warna yang berbeda tergantung pada kondisi tertentu dari elemen tersebut, misalnya saat elemen sedang dipilih (*checked*), difokuskan, ditekan, atau dalam keadaan normal.

Terdapat dua elemen **<item>**. Elemen pertama memiliki atribut **android:state_checked="true"**, artinya kondisi ini berlaku ketika elemen UI berada dalam keadaan terpilih (*checked*). Jika kondisi ini terpenuhi, maka warna yang digunakan adalah **#62339C**, yaitu warna ungu gelap.

Elemen kedua tidak memiliki atribut **state**, bertindak sebagai *default*. Artinya, ketika elemen tidak dalam keadaan *checked*, maka warna yang digunakan adalah **#D1B3FF**, yaitu ungu muda. Sehingga pengguna dapat dengan mudah membedakan antara elemen yang aktif dan tidak aktif secara visual.

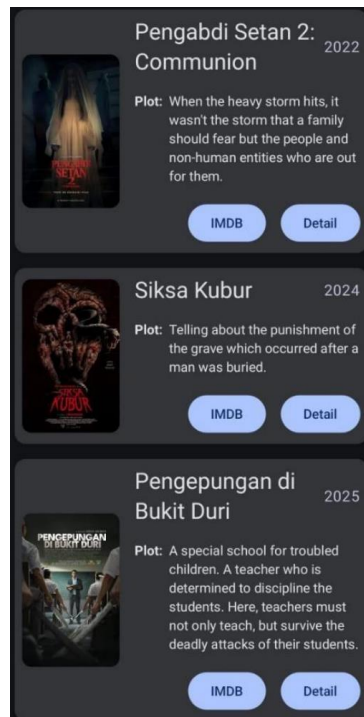
MODUL 3: Build a Scrollable List

SOAL 1

Buatlah sebuah aplikasi Android menggunakan XML atau Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) atau LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut:
 - a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
 - b. Button kedua menggunakan Navigation component/intent untuk membuka laman detail item
5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
8. Aplikasi berbasis XML harus menggunakan ViewBinding

UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dan setiap baris memiliki 2 teks yang berbeda. Diusahakan agar desain UI item list menyerupai UI berikut:



Gambar 18. Contoh UI List Soal 1 Modul 3

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut:



Gambar 19. Contoh UI Detail Soal 1 Modul 3

A. Source Code

Tabel 10. Source Code Soal 1 Modul 3

File MainActivity.kt	
1	package com.example.recycler_view
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	
6	class MainActivity : AppCompatActivity() {
7	override fun onCreate(savedInstanceState:
	Bundle?) {
8	super.onCreate(savedInstanceState)
9	setContentView(R.layout.activity_main)
10	
11	val fragmentManager = <i>supportFragmentManager</i>
12	val homeFragment = HomeFragment()
13	val fragment =
	fragmentManager.findFragmentByTag(HomeFragment::class.java.simpleName)
14	if (fragment !is HomeFragment) {
15	fragmentManager
16	.beginTransaction()
17	.add(R.id.frame_container,
	homeFragment, HomeFragment::class.java.simpleName)
18	.commit()
19	}
20	}
21	}

Tabel 11. Source Code Soal 1 Modul 3

File DetailFragment.kt	
1	package com.example.recycler_view
2	
3	import android.os.Bundle
4	import androidx.fragment.app.Fragment
5	import android.view.LayoutInflater
6	import android.view.View
7	import android.view.ViewGroup
8	import
	com.example.recycler_view.databinding.FragmentDetailB
	inding
9	
10	class DetailFragment : Fragment() {
11	
12	private var _binding: FragmentDetailBinding? =
	null
13	private val binding get() = _binding!!
14	
15	override fun onCreateView(
16	inflater: LayoutInflater, container:
	ViewGroup?,
17	savedInstanceState: Bundle?
18	): View? {
19	_binding =
	FragmentDetailBinding.inflate(inflater, container,
	false)
20	
21	val title =
	arguments?.getString("EXTRA_TITLE")

22	val photo = arguments?.getInt("EXTRA_PHOTO")
23	val plot = arguments?.getString("EXTRA_PLOT")
24	val year = arguments?.getString("EXTRA_YEAR")
25	val cast = arguments?.getString("EXTRA_CAST")
26	
27	binding.tvName.text = title
28	binding.tvPlot.text = plot
29	binding.tvYear.text = "\$year"
30	binding.tvCast.text = "\$cast"
31	photo?.let {
	binding.imgItemPhoto.setImageResource(it) }
32	
33	return binding.root
34	}
35	
36	override fun onDestroyView() {
37	super.onDestroyView()
38	_binding = null
39	}
40	}

Tabel 12. Source Code Soal 1 Modul 3

File HomeFragment.kt	
1	package com.example.recycler_view
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.os.Bundle
6	import android.view.LayoutInflater
7	import android.view.View

8	import android.view.ViewGroup
9	import androidx.fragment.app.Fragment
10	import androidx.recyclerview.widget.LinearLayoutManager
11	import com.example.recycler_view.databinding.FragmentHomeBi nding
12	
13	class HomeFragment : Fragment() {
14	
15	private var _binding: FragmentHomeBinding? = null
16	private val binding get() = _binding!!
17	
18	override fun onCreateView(19 inflater: LayoutInflater, container: ViewGroup?, 20 savedInstanceState: Bundle? 21): View? { 22 _binding = FragmentHomeBinding.inflate(inflater, container, false) 23 return binding.root 24 } 25
26	override fun onViewCreated(view: View, savedInstanceState: Bundle?) { 27 super.onViewCreated(view, savedInstanceState)

28	binding.rvDrama.layoutManager =
	LinearLayoutManager(requireContext())
29	binding.rvDrama.setHasFixedSize(true)
30	
31	val dramaAdapter =
	DramaAdapter(getDramaList(),
32	onWikiClick = { link ->
33	val uri = Uri.parse(link)
34	startActivity(Intent(Intent.ACTION_VIEW,
	uri))
35	},
36	onDetailClick = { title, photo, plot,
	year, cast ->
37	val detailFragment =
	DetailFragment().apply {
38	arguments = Bundle().apply {
39	putString("EXTRA_TITLE",
	title)
40	putInt("EXTRA_PHOTO", photo)
41	putString("EXTRA_PLOT",
	plot)
42	putString("EXTRA_YEAR",
	year)
43	putString("EXTRA_CAST",
	cast)
44	}
45	}
46	parentFragmentManager.beginTransaction()
47	.replace(R.id.frame_container,
	detailFragment)

48	.addToBackStack(null)
49	.commit()
50	}}
51	
52	binding.rvDrama.adapter = dramaAdapter
53	}
54	
55	private fun getDramaList(): ArrayList<Series> {
56	val dataTitle =
	resources.getStringArray(R.array.data_name)
57	val dataLink =
	resources.getStringArray(R.array.data_link)
58	val dataPhoto =
	resources.obtainTypedArray(R.array.data_photo)
59	val dataPlot =
	resources.getStringArray(R.array.data_plot)
60	val dataYear =
	resources.getStringArray(R.array.data_year)
61	val dataCast =
	resources.getStringArray(R.array.data_cast)
62	val listDrama = ArrayList<Series>()
63	for (i in dataTitle.indices) {
64	val drama = Series(
65	dataTitle[i],
66	dataLink[i],
67	dataPhoto.getResourceId(i, -1),
68	dataPlot[i],
69	dataYear[i],
70	dataCast[i]
71	)

72	listDrama.add(drama)
73	}
74	dataPhoto.recycle()
75	return listDrama
76	}
77	
78	override fun onDestroyView() {
79	super.onDestroyView()
80	_binding = null
81	}
82	}

Tabel 13. Source Code Soal 1 Modul 3

File ListSeriesAdapter.kt	
1	package com.example.recycler_view
2	
3	import android.view.LayoutInflater
4	import android.view.View
5	import android.view.ViewGroup
6	import android.widget.Button
7	import android.widget.ImageView
8	import android.widget.TextView
9	import androidx.recyclerview.widget.RecyclerView
10	
11	class DramaAdapter(
12	private val listDrama: ArrayList<Series>,
13	private val onWikiClick: (String) -> Unit,
14	private val onDetailClick: (String, Int, String,
	String, String) -> Unit
15	) : RecyclerView.Adapter<DramaAdapter.ViewHolder>() {

```

16
17     class ViewHolder(itemView: View) :
RecyclerView.ViewHolder(itemView) {
18         val imgPhoto: ImageView =
itemView.findViewById(R.id.img_item_photo)
19         val tvTitle: TextView =
itemView.findViewById(R.id.tv_item_name)
20         val tvPlot: TextView =
itemView.findViewById(R.id.tv_item_plot)
21         val btnWiki: Button =
itemView.findViewById(R.id.btn_wiki)
22         val btnDetail: Button =
itemView.findViewById(R.id.button_detail)
23     }
24
25     override fun onCreateViewHolder(parent:
ViewGroup, viewType: Int): ViewHolder {
26         val view: View =
LayoutInflater.from(parent.context).inflate(R.layout.
item_list, parent, false)
27         return ViewHolder(view)
28     }
29
30     override fun getItemCount(): Int = listDrama.size
31
32     override fun onBindViewHolder(holder: ViewHolder,
position: Int) {
33         val (title, link, photo, plot, year, cast) =
listDrama[position]
34         holder.tvTitle.text = title

```

35	holder.imgPhoto.setImageResource(photo)
36	holder.tvPlot.text = "\$cast"
37	holder.btnWiki.setOnClickListener {
	onWikiClick(link) }
38	holder.btnDetail.setOnClickListener {
	onDetailClick(title, photo, plot, year, cast) }
39	}
40	}

Tabel 14. Source Code Soal 1 Modul 3

File Series.kt	
1	package com.example.recycler_view
2	
3	import android.os.Parcelable
4	import kotlinx.parcelize.Parcelize
5	
6	@Parcelize
7	data class Series(
8	val title: String,
9	val link: String,
10	val photo: Int,
11	val plot: String,
12	val year: String,
13	val cast: String
14	) : Parcelable

Tabel 15. Source Code Soal 1 Modul 3

File activity_main.xml	
1	<androidx.constraintlayout.widget.ConstraintLayout
	xmlns:android="http://schemas.android.com/apk/res/a

	ndroid"
2	xmlns:app="http://schemas.android.com/apk/res-auto"
3	xmlns:tools="http://schemas.android.com/tools"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	tools:context=".MainActivity">
7	
8	<FrameLayout
9	android:id="@+id/frame_container"
10	android:layout_width="match_parent"
11	android:layout_height="match_parent" />
12	</androidx.constraintlayout.widget.ConstraintLayout>

Tabel 16. Source Code Soal 1 Modul 3

File fragment_detail.xml	
1	<?xml version="1.0" encoding="utf-8"?>
2	<ScrollView
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:fillViewport="true"
8	android:background="@drawable/bg"
9	tools:context=".DetailFragment">
10	

11	<androidx.constraintlayout.widget.ConstraintLayout
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"
14	android:paddingBottom="32dp">
15	
16	<ImageView
17	android:id="@+id/img_item_photo"
18	android:layout_width="341dp"
19	android:layout_height="458dp"
20	android:layout_marginTop="32dp"
21	android:scaleType="centerCrop"
22	app:layout_constraintEnd_toEndOf="parent"
23	app:layout_constraintStart_toStartOf="
	parent"
24	app:layout_constraintTop_toTopOf="parent"
25	tools:src="@drawable/antares" />
26	
27	<TextView
28	android:id="@+id/tv_name"
29	android:layout_width="wrap_content"
30	android:layout_height="wrap_content"
31	android:layout_marginTop="16dp"
32	android:fontFamily="serif"
33	android:textSize="25sp"
34	android:textStyle="bold"
35	app:layout_constraintEnd_toEndOf="parent"
36	app:layout_constraintStart_toStartOf="
	parent"
37	app:layout_constraintTop_toBottomOf="@id
	/img_item_photo"

```

38         tools:text="Judul Series" />
39
40     <TextView
41         android:id="@+id/tv_year"
42         android:layout_width="wrap_content"
43         android:layout_height="wrap_content"
44         android:text="Tahun: "
45         android:fontFamily="serif"
46         android:textSize="18sp"
47         android:layout_marginTop="8dp"
48         app:layout_constraintTop_toBottomOf="@id
/tv_name"
49         app:layout_constraintStart_toStartOf="parent"
50         app:layout_constraintEnd_toEndOf="parent"
/>
51
52     <TextView
53         android:id="@+id/tv_cast"
54         android:layout_width="0dp"
55         android:layout_height="wrap_content"
56         android:text="Pemeran: "
57         android:fontFamily="serif"
58         android:textSize="18sp"
59         android:layout_marginTop="8dp"
60         android:layout_marginHorizontal="16dp"
61         android:textAlignment="center"
62         app:layout_constraintTop_toBottomOf="@id
/tv_year"
63         app:layout_constraintStart_toStartOf="parent"
64         app:layout_constraintEnd_toEndOf="parent"

```

	/>
65	
66	<TextView
67	android:id="@+id/tv_plot"
68	android:layout_width="0dp"
69	android:layout_height="wrap_content"
70	android:layout_margin="16dp"
71	android:fontFamily="serif"
72	android:text="Sinopsis: "
73	android:textAlignment="center"
74	android:textSize="18sp"
75	app:layout_constraintBottom_toBottomOf="
	parent"
76	app:layout_constraintEnd_toEndOf="parent"
77	app:layout_constraintStart_toStartOf="
	parent"
78	app:layout_constraintTop_toBottomOf=
	"@+id/tv_cast" />
79	
80	</androidx.constraintlayout.widget.ConstraintLayout>
81	</ScrollView>

Tabel 17. Source Code Soal 1 Modul 3

File fragment_home.xml	
1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/and
	roid"
4	xmlns:app="http://schemas.android.com/apk/res-
	auto"

5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:background="@drawable/bg2">
8	
9	<androidx.recyclerview.widget.RecyclerView
10	android:id="@+id/rvDrama"
11	android:layout_width="0dp"
12	android:layout_height="0dp"
13	app:layout_constraintTop_toTopOf="parent"
14	app:layout_constraintBottom_toBottomOf="parent"
15	app:layout_constraintStart_toStartOf="parent"
16	app:layout_constraintEnd_toEndOf="parent" />
17	</androidx.constraintlayout.widget.ConstraintLayout>

Tabel 18. Source Code Soal 1 Modul 3

File item_list.xml	
1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.cardview.widget.CardView
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:layout_width="match_parent"
5	android:layout_height="wrap_content"
6	xmlns:tools="http://schemas.android.com/tools"
7	android:layout_margin="8dp"
8	app:cardCornerRadius="8dp"
9	app:cardElevation="4dp">
10	

11	<androidx.constraintlayout.widget.ConstraintLayout
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"
14	android:padding="8dp"
15	tools:src="@drawable/bg2">
16	
17	<ImageView
18	android:id="@+id/img_item_photo"
19	android:layout_width="70dp"
20	android:layout_height="100dp"
21	android:scaleType="centerCrop"
22	app:layout_constraintStart_toStartOf="
	parent"
23	app:layout_constraintTop_toTopOf="parent"
24	app:layout_constraintBottom_toBottomOf="
	parent"
25	tools:src="@drawable/antares" />
26	
27	<TextView
28	android:id="@+id/tv_item_name"
29	android:layout_width="0dp"
30	android:layout_height="wrap_content"
31	android:layout_marginStart="12dp"
32	android:textSize="18sp"
33	android:textStyle="bold"
34	android:fontFamily="serif"
35	android:textColor="#000000"
36	app:layout_constraintStart_toEndOf="@id/i
	mg_item_photo"
37	app:layout_constraintTop_toTopOf="parent"

38	app:layout_constraintEnd_toEndOf="parent"
39	tools:text="Antares" />
40	
41	<TextView
42	android:id="@+id/tv_item_plot"
43	android:layout_width="0dp"
44	android:layout_height="wrap_content"
45	android:layout_marginTop="4dp"
46	android:textSize="14sp"
47	android:fontFamily="serif"
48	app:layout_constraintTop_toBottomOf="@id/ tv_item_name"
49	app:layout_constraintStart_toStartOf="@id /tv_item_name"
50	app:layout_constraintEnd_toEndOf="parent"
51	tools:text="Cast: " />
52	
53	<Button
54	android:id="@+id/btn_wiki"
55	android:layout_width="wrap_content"
56	android:layout_height="wrap_content"
57	android:layout_marginStart="60dp"
58	android:layout_marginTop="12dp"
59	android:backgroundTint="#9bc6ec"
60	android:fontFamily="serif"
61	android:textStyle="bold"
62	android:text="WeTV"
63	app:layout_constraintStart_toStartOf="@id /tv_item_plot"

64	app:layout_constraintTop_toBottomOf="@id/ tv_item_plot" />
65	
66	<Button
67	android:id="@+id/button_detail"
68	android:layout_width="wrap_content"
69	android:layout_height="wrap_content"
70	android:layout_marginStart="40dp"
71	android:backgroundTint="#9bc6ec"
72	android:fontFamily="serif"
73	android:textStyle="bold"
74	android:text="Detail"
75	app:layout_constraintStart_toEndOf="@id/b tn_wiki"
76	app:layout_constraintTop_toTopOf="@id/btn _wiki" />
77	
78	</androidx.constraintlayout.widget.ConstraintLayout>
79	</androidx.cardview.widget.CardView>

Tabel 19. Source Code Soal 1 Modul 3

File colors.xml	
1	<?xml version="1.0" encoding="utf-8"?>
2	<resources>
3	<color name="black">#FF000000</color>
4	<color name="white">#FFFFFFFF</color>
5	<color name="blue">#9bc6ec</color>
6	</resources>

Tabel 20. Source Code Soal 1 Modul 3

File strings.xml	
1	<resources>
2	<string name="app_name">Rycycler View</string>
3	<string name="btn_detail">Detail</string>
4	<string name="btn_wiki">Wiki</string>
5	
6	<string-array name="data_name">
7	<item>Antares</item>
8	<item>Harus Kawin</item>
9	<item>Kaget Nikah</item>
10	<item>Jodoh atau Bukan</item>
11	<item>Kisah untuk Geri</item>
12	<item>Mozachiko</item>
13	<item>5 Detik dan Rasa Rindu</item>
14	<item>17 Selamanya</item>
15	<item>Teluk Alaska</item>
16	<item>2 Wajah Arjuna</item>
17	</string-array>
18	
19	<string-array name="data_photo">
20	<item>@drawable/antares</item>
21	<item>@drawable/harus_kawin</item>
22	<item>@drawable/kaget_nikah</item>
23	<item>@drawable/jodoh_bukan</item>
24	<item>@drawable/kisah_geri</item>
25	<item>@drawable/mozachiko</item>
26	<item>@drawable/rasa_rindu</item>
27	<item>@drawable/selamanya</item>
28	<item>@drawable/teluk_alaska</item>

29	<item>@drawable/wajah_arjuna</item>
30	</string-array>
31	
32	<string-array name="data_link">
33	<item>https://wetv.vip/id/play/a41kjezlclczg0i/y0040sv1hu9-EP01A%3A_Antares</item>
34	<item>https://wetv.vip/id/play/hm0734w9etht4vn-Harus%20Kawin/w0047tjoai-EP01A%3A%20Harus%20Kawin</item>
35	<item>https://wetv.vip/id/play/yweofusr9c2jpy2/z0041b49t19-EP01A%3A_Kaget_Nikah</item>
36	<item>https://wetv.vip/id/play/k2cb8gl9t27lpbq-Jodoh%20atau%20Bukan/h0045cixsi-EP01A%3A%20Jodoh%20atau%20Bukan</item>
37	<item>https://wetv.vip/id/play/ns0pvt3754wv3il-Kisah%20Untuk%20Geri/p0036f3bfbj-EP01%3A%20Kisah%20Untuk%20Geri</item>
38	<item>https://wetv.vip/id/play/3qoivvzv2pbi6wt-Mozachiko/w0046wfrhc5-EP01A%3A%20Mozachiko</item>
39	<item>https://wetv.vip/id/play/yfxtx6zpngrg25j-5%20Detik%20%26%20Rasa%20Rindu/b0047d2dh1h-EP01A%3A%205%20Detik%20%26%20Rasa%20Rindu</item>
40	<item>https://wetv.vip/id/play/2s6dyeoedxd6xc-17%20Selamanya/o00429wd0s1-EP01A%3A%2017%20Selamanya</item>
41	<item>https://wetv.vip/id/play/akwckof2lj9wjmn-Teluk%20Alaska/i0041ztcg2t-EP01A%3A%20Teluk%20Alaska</item>
42	<item>https://wetv.vip/id/play/ff3culwgrhb8q63-Dua%20Wajah%20Arjuna/k0047b3g69u-

	EP01A%3A%20Dua%20Wajah%20Arjuna</item>
43	</string-array>
44	
45	<string-array name="data_plot">
46	<item>Series ini mengisahkan Zea, siswi baru yang berusaha mengungkap kebenaran di balik kecelakaan kakaknya yang melibatkan geng motor bernama Calderioz, yang dipimpin oleh Antares.</item>
47	<item>Mengisahkan lima sahabat yang dipaksa menikah oleh keluarga mereka. Serial ini menyoroti dilema dan tekanan sosial yang mereka hadapi dalam menghadapi pernikahan yang tidak diinginkan.</item>
48	<item>Lalita mengalami kecelakaan yang menyebabkan kehilangan keperawanannya dan terpaksa menikah dengan Andre, pemuda yang menolongnya. Pernikahan ini dijalani tanpa cinta karena desakan orang tua.</item>
49	<item>Natalie, seorang wanita stylish, hidupnya berubah saat ibunya menjodohkannya dengan anak partner bisnisnya. Serial ini mengeksplorasi dilema antara cinta dan perjodohan.</item>
50	<item>Setelah keluarganya bangkrut, Dinda, yang dulunya populer, harus menghadapi realitas baru dan menjalin hubungan dengan Geri, mantan musuhnya, dalam sebuah kontrak palsu yang berubah menjadi cinta.</item>
51	<item>Moza, siswi culun, bertekad membuat Chiko, siswa populer, jatuh cinta padanya dalam 100 hari.</item>

52	<code><item>Via, seorang karyawan muda, mencari artis baru untuk duet dengan Drupadi. Gana pun ditunjuk sebagai teman duet, dan mereka merilis singel yang diadaptasi dari puisi karya Via.</item></code>
53	<code><item>Dawai, gadis polos yang disukai Putra, menyimpan rahasia besar terkait masa lalu mereka yang memengaruhi hubungan mereka.</item></code>
54	<code><item>Anastasia, gadis pendiam, bertemu kembali dengan Alister, teman masa kecilnya yang kini menjadi siswa populer.</item></code>
55	<code><item>Arjuna, pria dengan wajah biasa, menemukan spray misterius yang mengubah wajahnya menjadi tampan, mengubah kehidupannya secara drastis.</item></code>
56	<code></string-array></code>
57	
58	<code><string-array name="data_year"></code>
59	<code><item>30 Juli 2021</item></code>
60	<code><item>5 Januari 2024</item></code>
61	<code><item>23 Desember 2021</item></code>
62	<code><item>13 Januari 2023</item></code>
63	<code><item>5 Maret 2021</item></code>
64	<code><item>2 Juni 2023</item></code>
65	<code><item>8 September 2023</item></code>
66	<code><item>10 Maret 2022</item></code>
67	<code><item>5 November 2021</item></code>
68	<code><item>30 Oktober 2023</item></code>
69	<code></string-array></code>
70	
71	<code><string-array name="data_cast"></code>

72	<code><item>Angga Yunanda, Beby Tsabina, Irzan Faiq</item></code>
73	<code><item>Kevin Julio, Haico Van der Veken</item></code>
74	<code><item>Aurora Ribero, Kevin Julio, Steffi Zamora, Fero Walandouw</item></code>
75	<code><item>Megan Domani, Rayn Wijaya, Salshabilla Adriani</item></code>
76	<code><item>Angga Yunanda, Syifa Hadju </item></code>
77	<code><item>Rebecca Kloppe, Junior Roberts, Yesaya Abraham</item></code>
78	<code><item>Prilly Latuconsina, Bryan Domani, Salshabilla Adriani</item></code>
79	<code><item>Syifa Hadju, Rizky Nazar, Steffi Zamora</item></code>
80	<code><item>Syifa Hadju, Devano Danendra, Kaneishia Yusuf</item></code>
81	<code><item>Yesaya Abraham, Beby Tsabina, Mikha Herman</item></code>
82	<code></string-array></code>
83	
84	<code><string name="hello_blank_fragment">Hello blank fragment</string></code>
85	<code></resources></code>

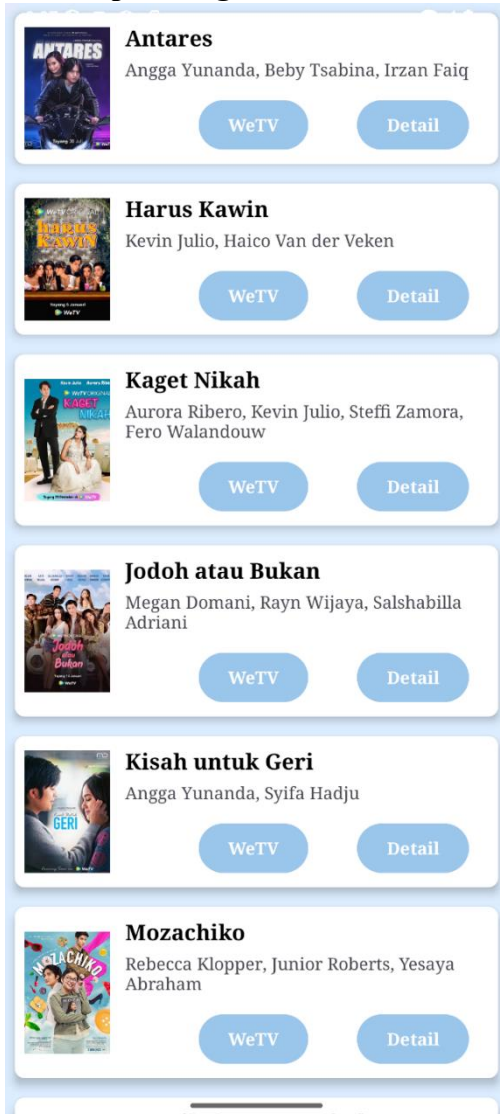
Tabel 21. Source Code Soal 1 Modul 3

File build.gradle.kts (:app)	
1	<code>plugins {</code>
2	<code>id("com.android.application")</code>
3	<code>id("org.jetbrains.kotlin.android")</code>

4	id("org.jetbrains.kotlin.plugin.parcelize")
5	}
6	
7	android {
8	namespace = "com.example.recycler_view"
9	compileSdk = 35
10	
11	defaultConfig {
12	applicationId = "com.example.recycler_view"
13	minSdk = 30
14	targetSdk = 35
15	versionCode = 1
16	versionName = "1.0"
17	
18	testInstrumentationRunner =
	"androidx.test.runner.AndroidJUnitRunner"
19	}
20	
21	buildTypes {
22	release {
23	isMinifyEnabled = false
24	proguardFiles(
25	getDefaultProguardFile("proguard-
	android-optimize.txt"),
26	"proguard-rules.pro"
27	)
28	}
29	}
30	compileOptions {
31	sourceCompatibility = JavaVersion.VERSION_11

32	targetCompatibility = JavaVersion.VERSION_11
33	}
34	kotlinOptions {
35	jvmTarget = "11"
36	}
37	buildFeatures {
38	viewBinding = true
39	}
40	}
41	
42	dependencies {
43	
44	implementation(libs.androidx.core.ktx)
45	implementation(libs.androidx.appcompat)
46	implementation(libs.material)
47	implementation(libs.androidx.activity)
48	implementation("androidx.recyclerview:recyclerview:1.3.1")
49	implementation
	("androidx.constraintlayout:constraintlayout:2.1.4")
50	testImplementation(libs.junit)
51	androidTestImplementation(libs.androidx.junit)
52	androidTestImplementation(libs.androidx.espresso.core)
53	}

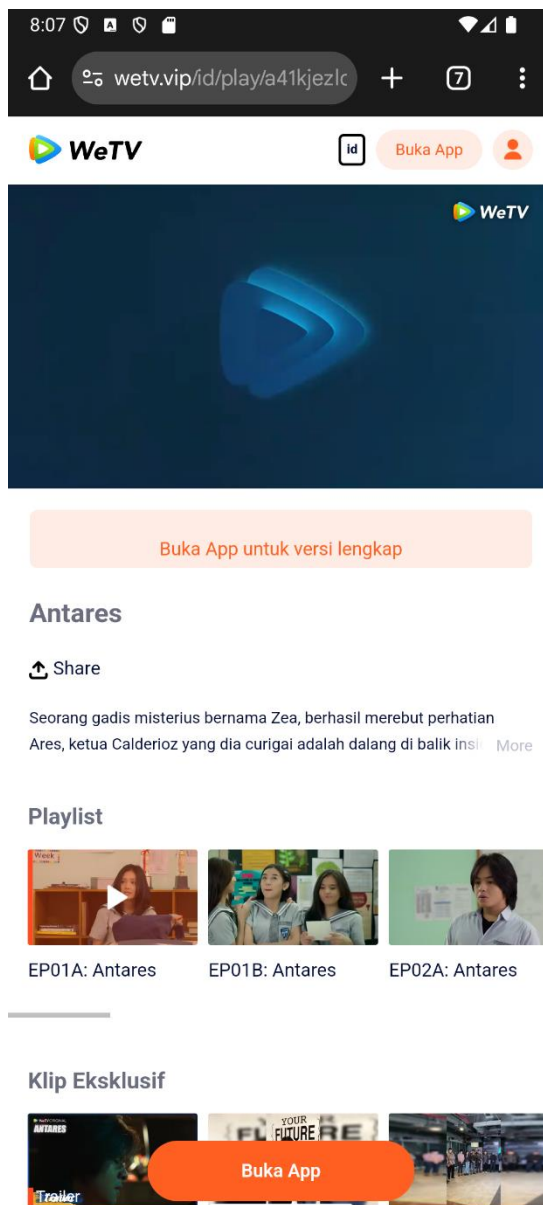
B. Output Program



Gambar 20. Screenshot Hasil Tampilan Soal 1 Modul 3



Gambar 21. Screenshot Hasil Tampilan Soal 1 Modul 3



Gambar 22. Screenshot Hasil Tampilan Soal 1 Modul 3

C. Pembahasan File MainActivity.kt

Kelas **MainActivity** mewarisi dari **AppCompatActivity**, yang merupakan superclass umum untuk aktivitas yang menggunakan fitur-fitur modern seperti **ActionBar**. Di dalam metode **onCreate()**, pertama-tama dipanggil

super.onCreate(savedInstanceState) untuk memastikan bahwa logika inisialisasi dari superclass berjalan sebagaimana mestinya. Selanjutnya, **setContentView(R.layout.activity_main)** digunakan untuk menentukan layout XML yang akan digunakan sebagai tampilan utama dari aktivitas ini, dalam hal ini adalah **activity_main.xml**.

Setelah layout diatur, kode melanjutkan dengan inisialisasi **FragmentManager** melalui **supportFragmentManager**. **FragmentManager** ini digunakan untuk mengelola transaksi fragment seperti penambahan, penggantian, atau penghapusan fragment. Kemudian, dibuat sebuah *instance* dari **HomeFragment**, yaitu fragment yang akan ditampilkan dalam activity. Baris **val fragment = fragmentManager.findFragmentByTag(HomeFragment::class.java.simpleName)** digunakan untuk memeriksa apakah fragment dengan tag tertentu yang berdasarkan nama kelas **HomeFragment** sudah ada atau belum dalam manajer fragment.

Jika fragment yang ditemukan bukan *instance* dari **HomeFragment** (**fragment !is HomeFragment**), maka dilakukan transaksi untuk menambahkan fragment ke dalam container dengan ID **R.id.frame_container**. Metode **beginTransaction()** digunakan untuk memulai transaksi, kemudian **add()** menambahkan fragment ke dalam layout. Parameter **HomeFragment::class.java.simpleName** sebagai tag digunakan agar fragment ini bisa dikenali atau dicari kembali di kemudian hari. Terakhir, transaksi ini dikonfirmasi dan dijalankan dengan **commit()**.

File DetailFragment.kt

DetailFragment bertanggung jawab untuk menampilkan detail dari suatu series. **DetailFragment** mewarisi dari kelas **Fragment**, dan menggunakan *view binding* untuk mengakses elemen-elemen UI secara aman dan efisien tanpa perlu menggunakan **findViewById**.

Dalam fragment ini, terdapat properti `_binding` bertipe *nullable* **FragmentDetailBinding**, yang menyimpan referensi ke objek binding yang dihasilkan dari file layout **fragment_detail.xml**. Akses terhadap binding dilakukan melalui properti **binding**, yang menggunakan operator **!!** untuk memastikan binding tidak null sehingga, hanya bisa diakses ketika view sudah dibuat.

Metode **onCreateView()** digunakan untuk meng-*inflate* layout fragment dan mengisi data ke dalam elemen UI. Pertama, objek `_binding` diinisialisasi menggunakan **FragmentDetailBinding.inflate(inflater, container, false)**, yang berarti layout dari fragment di-*inflate* dan dikaitkan dengan *binding*. Kemudian, data diambil dari *argument bundle* yang diberikan ke fragment melalui **arguments?.getString(...)** dan **arguments?.getInt(...)**. Nilai-nilai ini meliputi **EXTRA_TITLE**, **EXTRA_PHOTO**, **EXTRA_PLOT**, **EXTRA_YEAR**, dan **EXTRA_CAST**.

Setelah data diambil, fragment mengisi tampilan menggunakan binding, misalnya dengan **binding.tvName.text = title** dan **binding.imgItemPhoto.setImageResource(it)** untuk menampilkan gambar jika nilai **photo** tidak null. Metode ini mengembalikan **binding.root**, yaitu *root view* dari layout yang telah di-*inflate*. Untuk menghindari memory leak, **onDestroyView()** digunakan untuk menyetel `_binding` ke null saat view fragment dihancurkan.

File ListSeriesAdapter.kt

Kelas ini menampilkan daftar objek **Series** dalam bentuk daftar item. Adapter ini menerima tiga parameter utama dalam konstruktor sebuah **ArrayList<Series>** yang berisi data drama, sebuah lambda **onWikiClick** yang menangani aksi saat pengguna ingin membuka tautan WeTV, dan **onDetailClick** yang menangani aksi ketika pengguna ingin melihat detail drama.

Di dalam adapter terdapat kelas **ViewHolder**, yang bertanggung jawab untuk menampung dan menginisialisasi elemen-elemen UI dari setiap item, yaitu **ImageView** untuk **imgPhoto**, dua **TextView** untuk **tvTitle**, dan **tvPlot**, serta dua **Button** masing-masing untuk membuka **btnWiki** dan **btnDetail**. Layout setiap item dibuat dengan menggunakan file **item_list.xml** yang di-*inflate* di dalam metode **onCreateViewHolder()**. Metode ini mengembalikan objek **ViewHolder** yang siap digunakan.

Metode **getItemCount()** akan mengembalikan jumlah total item dalam daftar **listDrama**, yang menentukan seberapa banyak item yang akan ditampilkan di layar. Kemudian, pada **onBindViewHolder()**, adapter akan mengikat data ke elemen UI berdasarkan posisi item dalam daftar. Di sini, elemen-elemen seperti **title**, **photo**, dan **cast** dimasukkan ke tampilan masing-masing, dan tombol-tombol diatur dengan listener yang memanggil fungsi callback yang telah didefinisikan. Misalnya, **btnWiki** akan membuka link WeTV menggunakan fungsi **onWikiClick**, sedangkan **btnDetail** akan memicu **onDetailClick** yang akan membawa data drama ke tampilan detail.

File Series.kt

Kelas ini menggunakan anotasi **@Parcelize** dan mengimplementasikan antarmuka **Parcelable**, yang memungkinkan objek **Series** untuk dikirim antar komponen Android (seperti antar **Activity** atau **Fragment**) melalui **Bundle**.

Setiap instance dari **Series** memiliki enam property yaitu, **title** bertipe **String** untuk menyimpan judul serial, **link** untuk menyimpan URL untuk mengarah ke web WeTV, **photo** bertipe **Int** yang merupakan resource ID dari drawable, **plot** yang berisi sinopsis cerita, **year** untuk tahun rilis, dan **cast** yang memuat daftar pemeran.

Dengan penggunaan **@Parcelize**, proses pembuatan parcel (objek yang bisa dipaketkan) menjadi otomatis, sehingga tidak perlu menuliskan kode boilerplate seperti **writeToParcel()** atau **describeContents()**.

File activity_main.xml

Kode XML tersebut merupakan layout utama dari **MainActivity** dalam aplikasi Android yang menggunakan **ConstraintLayout** sebagai root layout. Di dalamnya terdapat satu komponen penting, yaitu **FrameLayout** dengan ID **@+id/frame_container**. Elemen ini berfungsi sebagai **wadah** untuk menampung dan menampilkan **Fragment**, misalnya **HomeFragment** atau **DetailFragment**, yang dimuat secara dinamis melalui transaksi fragment dari kode Kotlin.

Tag **ConstraintLayout** menyediakan kemampuan untuk menempatkan dan mengatur posisi elemen-elemen UI secara fleksibel menggunakan constraint antar komponen atau ke parent layout. Namun dalam kasus ini, hanya ada satu elemen di dalamnya, yaitu **FrameLayout**. Meskipun begitu, karena ukuran **FrameLayout** diatur ke **match_parent** baik untuk lebar maupun tinggi, maka elemen ini akan mengisi seluruh ruang yang tersedia dalam layar.

Atribut **tools:context=".MainActivity"** digunakan oleh Android Studio saat preview layout di editor, untuk memberi tahu bahwa layout ini digunakan dalam **MainActivity**.

File fragment_detail.xml

Layout ini menggunakan **ScrollView** sebagai root. Di dalam **ScrollView**, terdapat **ConstraintLayout** sebagai container utama untuk menata elemen-elemen UI secara fleksibel. Elemen pertama adalah **ImageView** dengan ID **img_item_photo**, yang menampilkan gambar serial dengan ukuran tetap (**341dp x 458dp**) dan **scaleType centerCrop** agar gambar terisi penuh namun tetap proporsional. Gambar ini ditampilkan di bagian atas layout dan diberi

margin atas **32dp**. Terdapat empat buah **TextView** yaitu, **tv_name** untuk menampilkan nama atau judul serial. ukuran teks **25sp, bold**, dan menggunakan font **serif** agar tampak khas seperti judul buku atau film, **tv_year** untuk menampilkan tahun rilis dengan label “Tahun:”, **tv_cast** untuk menampilkan pemeran dari serial, diletakkan tepat di bawah tahun dan dipusatkan, **tv_plot** untuk menampilkan sinopsis atau alur cerita dari serial tersebut, juga dipusatkan dan diberi margin agar tampak rapi.

Semua elemen diposisikan menggunakan **ConstraintLayout**, yaitu dengan menentukan hubungan seperti **app:layout_constraintTop_toBottomOf,**
layout_constraintStart_toStartOf, dan
layout_constraintEnd_toEndOf agar konten tersusun secara vertikal dan simetris di tengah layar.

Atribut **android:background="@drawable/bg"** pada **ScrollView** menunjukkan bahwa latar belakang layar menggunakan gambar atau bentuk dari resource drawable dengan nama **bg**.

File fragment_home.xml

Kelas ini root layout-nya menggunakan **ConstraintLayout**, yang memberi fleksibilitas tinggi dalam menyusun elemen-elemen UI. Layout ini juga memiliki latar belakang bergambar melalui atribut **android:background="@drawable/bg2"**, yang memberikan estetika visual yang lebih menarik dibandingkan latar belakang polos.

Satu-satunya elemen yang dimuat dalam **ConstraintLayout** ini adalah **RecyclerView** dengan ID **rvDrama**. Komponen ini digunakan untuk menampilkan daftar series secara efisien, karena hanya memuat tampilan yang terlihat di layar dan mendaur ulang elemen lainnya. **RecyclerView** diatur agar mengisi seluruh area layar dengan menggunakan constraint ke setiap **Top, Bottom,**

Start, **End**, serta lebar dan tinggi diset ke **0dp** yang berarti akan mengikuti constraint yang telah didefinisikan.

Layout ini bekerja bersama dengan **DramaAdapter** dan data dari **Series** untuk menampilkan daftar item yang interaktif, di mana pengguna bisa menekan tombol untuk membuka halaman detail atau tautan WeTV dari serial tersebut.

File `item_list.xml`

Root elemen dari layout ini adalah **CardView**, yang memberikan tampilan kartu dengan sudut melengkung dengan menggunakan **cardCornerRadius="8dp"** dan bayangan dengan menggunakan **cardElevation="4dp"**, menciptakan tampilan modern dan terangkat dari latar belakang.

Di dalam **CardView**, terdapat **ConstraintLayout** sebagai wadah utama untuk mengatur tata letak elemen UI seperti gambar, teks, dan tombol dengan posisi yang fleksibel dan presisi. Bagian kiri layout menampilkan **ImageView** (`@+id/img_item_photo`) berukuran **70x100dp** yang menampilkan gambar sampul serial dengan skala **centerCrop**, sehingga gambar akan memenuhi area tampilan secara proporsional.

Di samping gambar, terdapat dua **TextView**: satu untuk nama series dengan menggunakan `@+id/tv_item_name` dengan ukuran teks besar dan gaya tebal untuk menonjolkan judul, dan satu lagi di bawahnya yaitu `@+id/tv_item_plot` yang menampilkan informasi singkat seperti nama pemeran. Kedua teks ini menggunakan font **serif**, memberi nuansa klasik dan rapi. Di bagian bawah, terdapat dua tombol **Button** yang berfungsi sebagai aksi untuk pengguna. `btn_wiki` diberi label "WeTV" dan kemungkinan digunakan untuk membuka link eksternal yaitu WeTV. Tombol kedua yaitu `button_detail` membuka halaman detail serial. Kedua tombol ini didesain dengan **backgroundTint** berwarna biru muda dengan kode warna **#9bc6ec**, sehingga tampak serasi.

File colors.xml

Dalam kelas ini terdapat beberapa warna yang didefinisikan untuk digunakan secara konsisten di seluruh aplikasi, yaitu:

- **black** – Diberi nilai **#FF000000**, yang merupakan warna hitam sepenuhnya.
- **white** – Diberi nilai **#FFFFFFFF**, warna putih dengan opasitas penuh.
- **blue** – Diberi nilai **#9bc6ec**, warna biru muda yang sering dipakai dalam elemen UI seperti latar belakang tombol (**backgroundTint**) agar terlihat segar dan menarik secara visual.

File strings.xml

Terdapat tiga string tunggal yaitu **app_name**, **btn_detail**, dan **btn_wiki** yang masing-masing digunakan sebagai nama aplikasi, label tombol detail, dan tombol WeTV. Selanjutnya, file ini berisi beberapa **string-array** yang digunakan untuk menampilkan data dinamis dalam daftar seperti **RecyclerView**. **data_name** berisi daftar judul drama, **data_plot** berisi sinopsis singkat tiap drama, **data_year** mencatat tanggal rilisnya, **data_cast** menampilkan nama-nama pemeran utama, dan **data_photo** untuk menampilkan poster tiap series. Masing-masing elemen dalam array ini disusun secara paralel, artinya urutannya harus konsisten satu sama lain agar data bisa ditampilkan dengan benar.

File build.gradle.kts (:app)

Pada file **build.gradle.kts** (:app) yang ditampilkan merupakan berkas konfigurasi utama untuk aplikasi Android berbasis Kotlin yang menggunakan DSL (*Domain Specific Language*) Kotlin. File ini berperan dalam mendefinisikan proses build aplikasi, termasuk plugin yang digunakan, konfigurasi SDK Android, fitur build, serta daftar dependensi. Di bagian atas file, terdapat deklarasi tiga plugin utama:

com.android.application,
org.jetbrains.kotlin.android, dan

`org.jetbrains.kotlin.plugin.parcelize.` Plugin `com.android.application` digunakan untuk menyatakan bahwa modul ini adalah aplikasi Android. Plugin `kotlin.android` memungkinkan penggunaan bahasa Kotlin dalam pengembangan Android, sementara plugin `kotlin-parcelize` memungkinkan penggunaan anotasi `@Parcelize` untuk mempermudah implementasi `Parcelable`.

Blok `android` memuat konfigurasi inti proyek. Properti `namespace` diatur ke `"com.example.recycler_view"`, yang berfungsi sebagai ruang nama unik aplikasi. Versi `compileSdk`, `targetSdk`, dan `minSdk` masing-masing ditetapkan ke 35, 35, dan 30. Ini menandakan bahwa aplikasi dikembangkan menggunakan API level 35 dan dapat dijalankan pada perangkat Android dengan API level 30 ke atas (Android 11+). Bagian `defaultConfig` juga mendefinisikan `applicationId`, `versionCode`, dan `versionName`, yang diperlukan dalam distribusi dan pembaruan aplikasi di Google Play. `testInstrumentationRunner` disetel ke `androidx.test.runner.AndroidJUnitRunner` untuk keperluan pengujian instrumentasi.

Dalam blok `buildTypes`, terdapat konfigurasi `release` yang menonaktifkan proses `minifyEnabled`, artinya kode tidak akan dimampatkan saat proses rilis, dan `proguardFiles` menunjukkan penggunaan konfigurasi default ProGuard serta file `proguard-rules.pro` untuk aturan kustom. Kemudian, `compileOptions` dan `kotlinOptions` menetapkan bahwa proyek ini menggunakan Java 11 baik sebagai target maupun sumber kompatibilitas, serta mengarahkan Kotlin untuk menargetkan JVM versi 11.

Selanjutnya, fitur `viewBinding` diaktifkan melalui `buildFeatures`, yang memungkinkan pengaksesan langsung ke elemen UI tanpa menggunakan `findViewById`, sehingga meningkatkan keamanan dan efisiensi pengembangan.

Di bagian `dependencies`, proyek ini sudah menggunakan Version Catalog, namun hanya pada bagian library, bukan plugin. Hal ini terlihat dari penggunaan

`libs.xxx` seperti `libs.androidx.core.ktx`,
`libs.androidx.appcompat`, `libs.material`,
`libs.androidx.activity`, serta dependensi eksplisit untuk
`recyclerview` dan `constraintlayout`. Untuk pengujian, digunakan `junit`
untuk unit test, serta `androidx.junit` dan `espresso.core` untuk
instrumented test. Penggunaan Version Catalog pada library memungkinkan
pengelolaan versi yang lebih terpusat dan konsisten melalui file
`libs.versions.toml`.

SOAL 2

Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

A. Pembahasan

RecyclerView masih banyak digunakan karena stabilitas, fleksibilitas, dan dukungan ekosistemnya yang sangat matang. Sejak diperkenalkan sebagai pengganti ListView, RecyclerView telah menjadi standar de facto untuk menampilkan daftar data dalam Android UI berbasis View System. Meskipun memiliki kode yang lebih panjang dan memerlukan boilerplate seperti adapter, view holder, dan layout manager, RecyclerView menawarkan kontrol penuh atas perilaku tampilan daftar, seperti pengaturan item grid, drag & drop, swipe, animasi kompleks, dan integrasi dengan berbagai library eksternal. Banyak proyek lama juga masih menggunakan View System, sehingga RecyclerView tetap relevan demi kompatibilitas dan konsistensi.

Sebaliknya, LazyColumn adalah bagian dari Jetpack Compose, toolkit UI modern yang berbasis deklaratif dan memungkinkan penulisan UI yang lebih ringkas, reaktif, dan modular. LazyColumn mengurangi kebutuhan akan boilerplate karena tidak memerlukan adapter atau view holder, sehingga kode menjadi lebih bersih dan cepat ditulis. Namun, Compose masih tergolong baru, dan meskipun sudah stabil, adopsinya masih dalam transisi. Beberapa tim pengembang memilih tetap menggunakan RecyclerView karena mereka memiliki kode warisan (legacy code), kendala integrasi, atau belum sepenuhnya beralih ke Compose karena berbagai alasan seperti resource, kompatibilitas library pihak ketiga, atau preferensi tim.

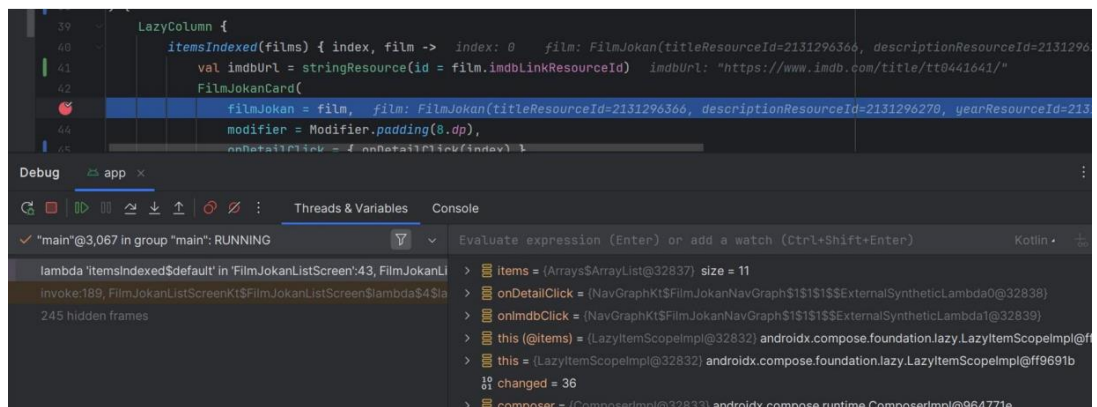
MODUL 4: ViewModel and Debugging

SOAL 1

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- Gunakan ViewModelFactory dalam pembuatan ViewModel
- Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment.
- Gunakan logging untuk event berikut:
 - Log saat data item masuk ke dalam list
 - Log saat tombol Detail dan tombol Explicit Intent ditekan
 - Log data dari list yang dipilih ketika berpindah ke halaman Detail
- Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out.

Aplikasi harus dapat mempertahankan fitur-fitur yang sudah dibuat pada modul sebelumnya. Berikut adalah contoh debugging dalam Android Studio.



Gambar 23. Contoh Penggunaan Debugger Soal 1 Modul 4

A. Source Code

Tabel 22. Source Code Soal 1 Modul 4

File MainActivity.kt	
1	package com.example.recycler_view
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	
6	class MainActivity : AppCompatActivity() {
7	override fun onCreate(savedInstanceState:
	Bundle?) {
8	super.onCreate(savedInstanceState)
9	setContentView(R.layout.activity_main)
10	
11	val fragmentManager = <i>supportFragmentManager</i>
12	val homeFragment = HomeFragment()
13	val fragment =
	fragmentManager.findFragmentByTag(HomeFragment::class
	. <i>java.simpleName</i>)
14	if (fragment !is HomeFragment) {
15	fragmentManager
16	.beginTransaction()
17	.add(R.id.frame_container,
	homeFragment, HomeFragment::class. <i>java.simpleName</i>)
18	.commit()
19	}
20	}
21	}

Tabel 23. Source Code Soal 1 Modul 4

File DetailFragment.kt	
1	package com.example.recycler_view
2	
3	import android.os.Bundle
4	import android.util.Log
5	import android.view.LayoutInflater
6	import android.view.View
7	import android.view.ViewGroup
8	import androidx.fragment.app.Fragment
9	import androidx.fragment.app.viewModels
10	import androidx.lifecycle.lifecycleScope
11	import
	com.example.recycler_view.databinding.FragmentDetailB
	inding
12	import kotlinx.coroutines.flow.collectLatest
13	import kotlinx.coroutines.launch
14	
15	class DetailFragment : Fragment() {
16	
17	private var _binding: FragmentDetailBinding? =
	null
18	private val binding get() = _binding!!
19	private val viewModel: SeriesViewModel by
	viewModels { SeriesViewModelFactory() }
20	
21	override fun onCreateView(
22	inflater: LayoutInflater, container:
	ViewGroup?,
23	savedInstanceState: Bundle?

```

24         ): View? {
25             _binding =
FragmentDetailBinding.inflate(inflater, container,
false)
26             return binding.root
27         }
28
29         override fun onCreateView(view: View,
savedInstanceState: Bundle?) {
30             super.onCreateView(view, savedInstanceState)
31
32             lifecycleScope.launch {
33                 viewModel.selectedSeries.collectLatest {
series ->
34                     series?.let {
35                         Log.d("DetailFragment",
"Menampilkan detail untuk: ${series.title}")
36                         binding.tvName.text =
series.title
37                         binding.tvPlot.text = series.plot
38                         binding.tvYear.text = series.year
39                         binding.tvCast.text = series.cast
40                     binding.imgItemPhoto.setImageResource(series.photo)
41                 }
42             }
43         }
44     }
45
46     override fun onDestroyView() {
47         super.onDestroyView()

```

48	<code>_binding = null</code>
49	<code>}</code>
50	<code>}</code>

Tabel 24. Source Code Soal 1 Modul 4

File HomeFragment.kt	
1	<code>package com.example.recycler_view</code>
2	
3	<code>import android.content.Intent</code>
4	<code>import android.net.Uri</code>
5	<code>import android.os.Bundle</code>
6	<code>import android.util.Log</code>
7	<code>import android.view.LayoutInflater</code>
8	<code>import android.view.View</code>
9	<code>import android.view.ViewGroup</code>
10	<code>import androidx.fragment.app.Fragment</code>
11	<code>import androidx.fragment.app.viewModels</code>
12	<code>import androidx.lifecycle.lifecyleScope</code>
13	<code>import</code> <code>androidx.recyclerview.widget.LinearLayoutManager</code> <code>import</code>
14	<code>com.example.recycler_view.databinding.FragmentHomeBi</code> <code>nding</code>
15	<code>import kotlinx.coroutines.flow.collectLatest</code>
16	<code>import kotlinx.coroutines.launch</code>
17	
18	<code>class HomeFragment : Fragment() {</code>
19	
20	<code>private var _binding: FragmentHomeBinding? =</code> <code>null</code>

21	private val binding get() = _binding!!
22	private val viewModel: SeriesViewModel by
	viewModels { SeriesViewModelFactory() }
23	
24	private lateinit var seriesAdapter:
	ListSeriesAdapter
25	
26	override fun onCreateView(
27	inflater: LayoutInflater, container:
	ViewGroup?,
28	savedInstanceState: Bundle?
29	): View {
30	_binding =
	FragmentHomeBinding.inflate(inflater, container,
	false)
31	return binding.root
32	}
33	
34	override fun onViewCreated(view: View,
	savedInstanceState: Bundle?) {
35	super.onViewCreated(view,
	savedInstanceState)
36	
37	seriesAdapter = ListSeriesAdapter(
38	emptyList(),
39	onWikiClick = { link ->
40	Log.d("HomeFragment", "Tombol Intent
	(WeTV) diklik: \$link")
41	val uri = Uri.parse(link)
42	startActivity(Intent(Intent.ACTION_VIEW,

	uri))
43	},
44	onClick = { series ->
45	Log.d("HomeFragment", "Tombol Detail
	diklik: \${series.title}")
46	viewModel.selectSeries(series)
47	parentFragmentManager.beginTransaction()
48	.replace(R.id.frame_container,
	DetailFragment())
49	.addToBackStack(null)
50	.commit()
51	}
52	)
53	
54	binding.rvDrama.apply {
55	layoutManager =
	LinearLayoutManager(requireContext())
56	setHasFixedSize(true)
57	adapter = seriesAdapter
58	}
59	
60	lifecycleScope.launch {
61	viewModel.seriesList.collectLatest {
	updatedList ->
62	seriesAdapter.updateList(updatedList)
63	}
64	}
65	
66	if (viewModel.seriesList.value.isEmpty()) {
67	viewModel.setSeriesList(getSeriesList())

```

68         }
69     }
70
71     private fun getSeriesList(): List<Series> {
72         val dataTitle =
73             resources.getStringArray(R.array.data_name)
74         val dataLink =
75             resources.getStringArray(R.array.data_link)
76         val dataPhoto =
77             resources.obtainTypedArray(R.array.data_photo)
78         val dataPlot =
79             resources.getStringArray(R.array.data_plot)
80         val dataYear =
81             resources.getStringArray(R.array.data_year)
82         val dataCast =
83             resources.getStringArray(R.array.data_cast)
84
85         val list = ArrayList<Series>()
86         for (i in dataTitle.indices) {
87             val series = Series(
88                 dataTitle[i],
89                 dataLink[i],
90                 dataPhoto.getResourceId(i, -1),
91                 dataPlot[i],
92                 dataYear[i],
93                 dataCast[i]
94             )
95             list.add(series)
96         }
97         dataPhoto.recycle()

```

92	return list
93	}
94	
95	override fun onDestroyView() {
96	super.onDestroyView()
97	_binding = null
98	}
99	}

Tabel 25. Source Code Soal 1 Modul 4

File ListSeriesAdapter.kt	
1	package com.example.recycler_view
2	
3	import android.view.LayoutInflater
4	import android.view.View
5	import android.view.ViewGroup
6	import android.widget.Button
7	import android.widget.ImageView
8	import android.widget.TextView
9	import androidx.recyclerview.widget.RecyclerView
10	
11	class ListSeriesAdapter(
12	private var listSeries: List<Series>,
13	private val onWikiClick: (String) -> Unit,
14	private val onDetailClick: (Series) -> Unit
15	) :
	RecyclerView.Adapter<ListSeriesAdapter.ViewHolder>()
	{
16	
17	class ViewHolder(itemView: View) :

	RecyclerView.ViewHolder(itemView) {
18	val imgPhoto: ImageView =
	itemView.findViewById(R.id. <i>img_item_photo</i>)
19	val tvTitle: TextView =
	itemView.findViewById(R.id. <i>tv_item_name</i>)
20	val tvCast: TextView =
	itemView.findViewById(R.id. <i>tv_cast</i>)
21	val btnWiki: Button =
	itemView.findViewById(R.id. <i>btn_wiki</i>)
22	val btnDetail: Button =
	itemView.findViewById(R.id. <i>button_detail</i>)
23	}
24	
25	override fun onCreateViewHolder(parent:
	ViewGroup, viewType: Int): ViewHolder {
26	val view =
	LayoutInflater.from(parent. <i>context</i>).inflate(R.layout. <i>item_list</i> , parent, false)
27	return ViewHolder(view)
28	}
29	
30	override fun getItemCount(): Int =
	listSeries.size
31	
32	override fun onBindViewHolder(holder: ViewHolder,
	position: Int) {
33	val series = listSeries[position]
34	holder.tvTitle. <i>text</i> = series.title
35	holder.tvCast. <i>text</i> = series.cast
36	holder.imgPhoto.setImageResource(series.photo)

37	holder.btnWiki.setOnClickListener {
	onWikiClick(series.link) }
38	holder.btnDetail.setOnClickListener {
	onDetailClick(series) }
39	}
40	
41	fun updateList(newList: List<Series>) {
42	listSeries = newList
43	notifyDataSetChanged()
44	}
45	}

Tabel 26. Source Code Soal 1 Modul 4

File Series.kt	
1	package com.example.recycler_view
2	
3	import android.os.Parcelable
4	import kotlinx.parcelize.Parcelize
5	
6	@Parcelize
7	data class Series(
8	val title: String,
9	val link: String,
10	val photo: Int,
11	val plot: String,
12	val year: String,
13	val cast: String
14	) : Parcelable

Tabel 27. Source Code Soal 1 Modul 4

File SeriesViewModel.kt	
1	package com.example.recycler_view
2	
3	import android.util.Log
4	import androidx.lifecycle.ViewModel
5	import kotlinx.coroutines.flow.MutableStateFlow
6	import kotlinx.coroutines.flow.StateFlow
7	
8	class SeriesViewModel : ViewModel() {
9	
10	private val _seriesList =
	MutableStateFlow<List<Series>>(emptyList())
11	val seriesList: StateFlow<List<Series>> get() =
	_seriesList
12	
13	private val _selectedSeries =
	MutableStateFlow<Series?>(null)
14	val selectedSeries: StateFlow<Series?> get() =
	_selectedSeries
15	
16	fun setSeriesList(list: List<Series>) {
17	_seriesList.value = list
18	Log.d("SeriesViewModel", "Data item masuk ke
	list: \$list")
19	}
20	
21	fun selectSeries(series: Series) {
22	_selectedSeries.value = series
23	Log.d("SeriesViewModel", "Item dipilih:

	<code>\${series.title}")</code>
	<code> }</code>
24	<code>}</code>
25	

Tabel 28. Source Code Soal 1 Modul 4

File SeriesViewModelFactory.kt	
1	<code>package com.example.recycler_view</code>
2	
3	<code>import androidx.lifecycle.ViewModel</code>
4	<code>import androidx.lifecycle.ViewModelProvider</code>
5	
6	<code>class SeriesViewModelFactory :</code> <code>ViewModelProvider.Factory {</code>
7	<code> override fun <T : ViewModel> create(modelClass:</code> <code>Class<T>): T {</code>
8	<code> if</code> <code>(modelClass.isAssignableFrom(SeriesViewModel::class.j</code> <code>ava)) {</code>
9	<code> return SeriesViewModel() as T</code>
10	<code> }</code>
11	<code> throw IllegalArgumentException("Unknown</code> <code>ViewModel class")</code>
12	<code> }</code>
13	<code>}</code>

Tabel 29. Source Code Soal 1 Modul 4

File activity_main.xml	
1	<code><androidx.constraintlayout.widget.ConstraintLayout</code> <code>xmlns:android="http://schemas.android.com/apk/res/a</code>

	ndroid"
2	xmlns:app="http://schemas.android.com/apk/res-auto"
3	xmlns:tools="http://schemas.android.com/tools"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	tools:context=".MainActivity">
7	
8	<FrameLayout
9	android:id="@+id/frame_container"
10	android:layout_width="match_parent"
11	android:layout_height="match_parent" />
12	</androidx.constraintlayout.widget.ConstraintLayout
	>

Tabel 30. Source Code Soal 1 Modul 4

File fragment_detail.xml	
1	<?xml version="1.0" encoding="utf-8"?>
2	<ScrollView
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:fillViewport="true"
8	android:background="@drawable/bg"
9	tools:context=".DetailFragment">
10	

11	<androidx.constraintlayout.widget.ConstraintLayout
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"
14	android:paddingBottom="32dp">
15	
16	<ImageView
17	android:id="@+id/img_item_photo"
18	android:layout_width="341dp"
19	android:layout_height="458dp"
20	android:layout_marginTop="32dp"
21	android:scaleType="centerCrop"
22	app:layout_constraintEnd_toEndOf="parent"
23	app:layout_constraintStart_toStartOf="parent"
24	app:layout_constraintTop_toTopOf="parent"
25	tools:src="@drawable/antares" />
26	
27	<TextView
28	android:id="@+id/tv_name"
29	android:layout_width="wrap_content"
30	android:layout_height="wrap_content"
31	android:layout_marginTop="16dp"
32	android:fontFamily="serif"
33	android:textSize="25sp"
34	android:textStyle="bold"
35	app:layout_constraintEnd_toEndOf="parent"
36	app:layout_constraintStart_toStartOf="parent"
37	app:layout_constraintTop_toBottomOf="@id
	/img_item_photo"
38	tools:text="Judul Series" />
39	

40	<TextView
41	android:id="@+id/tv_year"
42	android:layout_width="wrap_content"
43	android:layout_height="wrap_content"
44	android:text="Tahun: "
45	android:fontFamily="serif"
46	android:textSize="18sp"
47	android:layout_marginTop="8dp"
48	app:layout_constraintTop_toBottomOf="@id
	/tv_name"
49	app:layout_constraintStart_toStartOf="parent"
50	app:layout_constraintEnd_toEndOf="parent"
	/>
51	
52	<TextView
53	android:id="@+id/tv_cast"
54	android:layout_width="0dp"
55	android:layout_height="wrap_content"
56	android:text="Pemeran: "
57	android:fontFamily="serif"
58	android:textSize="18sp"
59	android:layout_marginTop="8dp"
60	android:layout_marginHorizontal="16dp"
61	android:textAlignment="center"
62	app:layout_constraintTop_toBottomOf="@id
	/tv_year"
63	app:layout_constraintStart_toStartOf="parent"
64	app:layout_constraintEnd_toEndOf="parent"
	/>
65	

66	<TextView
67	android:id="@+id/tv_plot"
68	android:layout_width="0dp"
69	android:layout_height="wrap_content"
70	android:layout_margin="16dp"
71	android:fontFamily="serif"
72	android:text="Sinopsis: "
73	android:textAlignment="center"
74	android:textSize="18sp"
75	app:layout_constraintBottom_toBottomOf="parent"
76	app:layout_constraintEnd_toEndOf="parent"
77	app:layout_constraintStart_toStartOf="parent"
78	app:layout_constraintTop_toBottomOf="@+id/tv_cast"
	/>
79	
80	</androidx.constraintlayout.widget.ConstraintLayout>
81	</ScrollView>

Tabel 31. Source Code Soal 1 Modul 4

File fragment_home.xml	
1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:background="@drawable/bg2">

8	
9	<code><androidx.recyclerview.widget.RecyclerView</code>
10	<code> android:id="@+id/rvDrama"</code>
11	<code> android:layout_width="0dp"</code>
12	<code> android:layout_height="0dp"</code>
13	<code> app:layout_constraintTop_toTopOf="parent"</code>
14	<code> app:layout_constraintBottom_toBottomOf="parent"</code>
15	<code> app:layout_constraintStart_toStartOf="parent"</code>
16	<code> app:layout_constraintEnd_toEndOf="parent" /></code>
17	<code></androidx.constraintlayout.widget.ConstraintLayout></code>

Tabel 32. Source Code Soal 1 Modul 4

File item_list.xml	
1	<code><?xml version="1.0" encoding="utf-8"?></code>
2	<code><androidx.cardview.widget.CardView</code>
	<code> xmlns:android="http://schemas.android.com/apk/res/and</code>
	<code>roid"</code>
3	<code> xmlns:app="http://schemas.android.com/apk/res-</code>
	<code>auto"</code>
4	<code> android:layout_width="match_parent"</code>
5	<code> android:layout_height="wrap_content"</code>
6	<code> xmlns:tools="http://schemas.android.com/tools"</code>
7	<code> android:layout_margin="8dp"</code>
8	<code> app:cardCornerRadius="8dp"</code>
9	<code> app:cardElevation="4dp"></code>
10	
11	<code> <androidx.constraintlayout.widget.ConstraintLayout</code>
12	<code> android:layout_width="match_parent"</code>
13	<code> android:layout_height="wrap_content"</code>
14	<code> android:padding="8dp"</code>

15	tools:src="@drawable/bg2">
16	
17	<ImageView
18	android:id="@+id/img_item_photo"
19	android:layout_width="70dp"
20	android:layout_height="100dp"
21	android:scaleType="centerCrop"
22	app:layout_constraintStart_toStartOf="parent"
23	app:layout_constraintTop_toTopOf="parent"
24	app:layout_constraintBottom_toBottomOf="p
	arent"
25	tools:src="@drawable/antares" />
26	
27	<TextView
28	android:id="@+id/tv_item_name"
29	android:layout_width="0dp"
30	android:layout_height="wrap_content"
31	android:layout_marginStart="12dp"
32	android:textSize="18sp"
33	android:textStyle="bold"
34	android:fontFamily="serif"
35	android:textColor="#000000"
36	app:layout_constraintStart_toEndOf="@id/i
	mg_item_photo"
37	app:layout_constraintTop_toTopOf="parent"
38	app:layout_constraintEnd_toEndOf="parent"
39	tools:text="Antares" />
40	
41	<TextView
42	android:id="@+id/tv_item_plot"

43	android:layout_width="0dp"
44	android:layout_height="wrap_content"
45	android:layout_marginTop="4dp"
46	android:textSize="14sp"
47	android:fontFamily="serif"
48	app:layout_constraintTop_toBottomOf="@id/ tv_item_name"
49	app:layout_constraintStart_toStartOf="@id /tv_item_name"
50	app:layout_constraintEnd_toEndOf="parent"
51	tools:text="Cast: " />
52	
53	<Button
54	android:id="@+id/btn_wiki"
55	android:layout_width="wrap_content"
56	android:layout_height="wrap_content"
57	android:layout_marginStart="60dp"
58	android:layout_marginTop="12dp"
59	android:backgroundTint="#9bc6ec"
60	android:fontFamily="serif"
61	android:textStyle="bold"
62	android:text="WeTV"
63	app:layout_constraintStart_toStartOf="@id /tv_item_plot"
64	app:layout_constraintTop_toBottomOf="@id/ tv_item_plot" />
65	
66	<Button
67	android:id="@+id/button_detail"
68	android:layout_width="wrap_content"

69	android:layout_height="wrap_content"
70	android:layout_marginStart="40dp"
71	android:backgroundTint="#9bc6ec"
72	android:fontFamily="serif"
73	android:textStyle="bold"
74	android:text="Detail"
75	app:layout_constraintStart_toEndOf="@id/btn_wiki"
76	app:layout_constraintTop_toTopOf="@id/btn_wiki" />
77	
78	</androidx.constraintlayout.widget.ConstraintLayout>
79	</androidx.cardview.widget.CardView>

Tabel 33. Source Code Soal 1 Modul 4

File colors.xml	
1	<?xml version="1.0" encoding="utf-8"?>
2	<resources>
3	<color name="black">#FF000000</color>
4	<color name="white">#FFFFFFFF</color>
5	<color name="blue">#9bc6ec</color>
6	</resources>

Tabel 34. Source Code Soal 1 Modul 4

File strings.xml	
1	<resources>
2	<string name="app_name">Rycycler View</string>
3	<string name="btn_detail">Detail</string>
4	<string name="btn_wiki">Wiki</string>
5	

6	<string-array name="data_name">
7	<item>Antares</item>
8	<item>Harus Kawin</item>
9	<item>Kaget Nikah</item>
10	<item>Jodoh atau Bukan</item>
11	<item>Kisah untuk Geri</item>
12	<item>Mozachiko</item>
13	<item>5 Detik dan Rasa Rindu</item>
14	<item>17 Selamanya</item>
15	<item>Teluk Alaska</item>
16	<item>2 Wajah Arjuna</item>
17	</string-array>
18	
19	<string-array name="data_photo">
20	<item>@drawable/antares</item>
21	<item>@drawable/harus_kawin</item>
22	<item>@drawable/kaget_nikah</item>
23	<item>@drawable/jodoh_bukan</item>
24	<item>@drawable/kisah_geri</item>
25	<item>@drawable/mozachiko</item>
26	<item>@drawable/rasa_rindu</item>
27	<item>@drawable/selamanya</item>
28	<item>@drawable/teluk_alaska</item>
29	<item>@drawable/wajah_arjuna</item>
30	</string-array>
31	
32	<string-array name="data_link">
33	<item> https://wetv.vip/id/play/a41kjezlclozg0i/y0040sv1hu9-EP01A%3A_Antares </item>
34	<item> https://wetv.vip/id/play/hm0734w9etht4vn-

	Harus%20Kawin/w0047tjoaoi-EP01A%3A%20Harus%20Kawin</item>
35	<item>https://wetv.vip/id/play/yweofusr9c2jpy2/z0041b49t19-EP01A%3A_Kaget_Nikah</item>
36	<item>https://wetv.vip/id/play/k2cb8gl9t27lpbq-Jodoh%20atau%20Bukan/h0045cixsi1-EP01A%3A%20Jodoh%20atau%20Bukan</item>
37	<item>https://wetv.vip/id/play/ns0pvt3754wv3il-Kisah%20Untuk%20Geri/p0036f3bfbj-EP01%3A%20Kisah%20Untuk%20Geri</item>
38	<item>https://wetv.vip/id/play/3qoivvzv2pbi6wt-Mozachiko/w0046wfrhc5-EP01A%3A%20Mozachiko</item>
39	<item>https://wetv.vip/id/play/yfxtx6zpngrg25j-5%20Detik%20%26%20Rasa%20Rindu/b0047d2dh1h-EP01A%3A%205%20Detik%20%26%20Rasa%20Rindu</item>
40	<item>https://wetv.vip/id/play/2s6dyeoedxd6xc-17%20Selamanya/o00429wd0s1-EP01A%3A%2017%20Selamanya</item>
41	<item>https://wetv.vip/id/play/akwckof2lj9wjmn-Teluk%20Alaska/i0041ztcg2t-EP01A%3A%20Teluk%20Alaska</item>
42	<item>https://wetv.vip/id/play/ff3culwgrhb8q63-Dua%20Wajah%20Arjuna/k0047b3g69u-EP01A%3A%20Dua%20Wajah%20Arjuna</item>
43	</string-array>
44	
45	<string-array name="data_plot">
46	<item>Series ini mengisahkan Zea, siswi baru yang berusaha mengungkap kebenaran di balik kecelakaan kakaknya yang melibatkan geng motor

	bernama Calderioz, yang dipimpin oleh Antares.</item>
47	<item>Mengisahkan lima sahabat yang dipaksa menikah oleh keluarga mereka. Serial ini menyoroti dilema dan tekanan sosial yang mereka hadapi dalam menghadapi pernikahan yang tidak diinginkan.</item>
48	<item>Lalita mengalami kecelakaan yang menyebabkan kehilangan keperawanannya dan terpaksa menikah dengan Andre, pemuda yang menolongnya. Pernikahan ini dijalani tanpa cinta karena desakan orang tua.</item>
49	<item>Natalie, seorang wanita stylish, hidupnya berubah saat ibunya menjodohkannya dengan anak partner bisnisnya. Serial ini mengeksplorasi dilema antara cinta dan perjodohan.</item>
50	<item>Setelah keluarganya bangkrut, Dinda, yang dulunya populer, harus menghadapi realitas baru dan menjalin hubungan dengan Geri, mantan musuhnya, dalam sebuah kontrak palsu yang berubah menjadi cinta.</item>
51	<item>Moza, siswi culun, bertekad membuat Chiko, siswa populer, jatuh cinta padanya dalam 100 hari.</item>
52	<item>Via, seorang karyawan muda, mencari artis baru untuk duet dengan Drupadi. Gana pun ditunjuk sebagai teman duet, dan mereka merilis singel yang diadaptasi dari puisi karya Via.</item>
53	<item>Dawai, gadis polos yang disukai Putra, menyimpan rahasia besar terkait masa lalu mereka yang memengaruhi hubungan mereka.</item>

54	<code><item>Anastasia, gadis pendiam, bertemu kembali dengan Alister, teman masa kecilnya yang kini menjadi siswa populer.</item></code>
55	<code><item>Arjuna, pria dengan wajah biasa, menemukan spray misterius yang mengubah wajahnya menjadi tampan, mengubah kehidupannya secara drastis.</item></code>
56	<code></string-array></code>
57	
58	<code><string-array name="data_year"></code>
59	<code><item>30 Juli 2021</item></code>
60	<code><item>5 Januari 2024</item></code>
61	<code><item>23 Desember 2021</item></code>
62	<code><item>13 Januari 2023</item></code>
63	<code><item>5 Maret 2021</item></code>
64	<code><item>2 Juni 2023</item></code>
65	<code><item>8 September 2023</item></code>
66	<code><item>10 Maret 2022</item></code>
67	<code><item>5 November 2021</item></code>
68	<code><item>30 Oktober 2023</item></code>
69	<code></string-array></code>
70	
71	<code><string-array name="data_cast"></code>
72	<code><item>Angga Yunanda, Beby Tsabina, Irzan Faiq</item></code>
73	<code><item>Kevin Julio, Haico Van der Veken</item></code>
74	<code><item>Aurora Ribero, Kevin Julio, Steffi Zamora, Fero Walandouw</item></code>
75	<code><item>Megan Domani, Rayn Wijaya, Salshabilla</code>

	Adriani</item>
76	<item>Angga Yunanda, Syifa Hadju </item>
77	<item>Rebecca Kloppe, Junior Roberts, Yesaya Abraham</item>
78	<item>Prilly Latuconsina, Bryan Domani, Salshabilla Adriani</item>
79	<item>Syifa Hadju, Rizky Nazar, Steffi Zamora</item>
80	<item>Syifa Hadju, Devano Danendra, Kaneishia Yusuf</item>
81	<item>Yesaya Abraham, Beby Tsabina, Mikha Herman</item>
82	</string-array>
83	
84	<string name="hello_blank_fragment">Hello blank fragment</string>
85	</resources>

Tabel 35. Source Code Soal 1 Modul 4

File build.gradle.kts (:app)	
1	plugins {
2	id("com.android.application")
3	id("org.jetbrains.kotlin.android")
4	id("org.jetbrains.kotlin.plugin.parcelize")
5	}
6	
7	android {
8	namespace = "com.example.recycler_view"
9	compileSdk = 35
10	

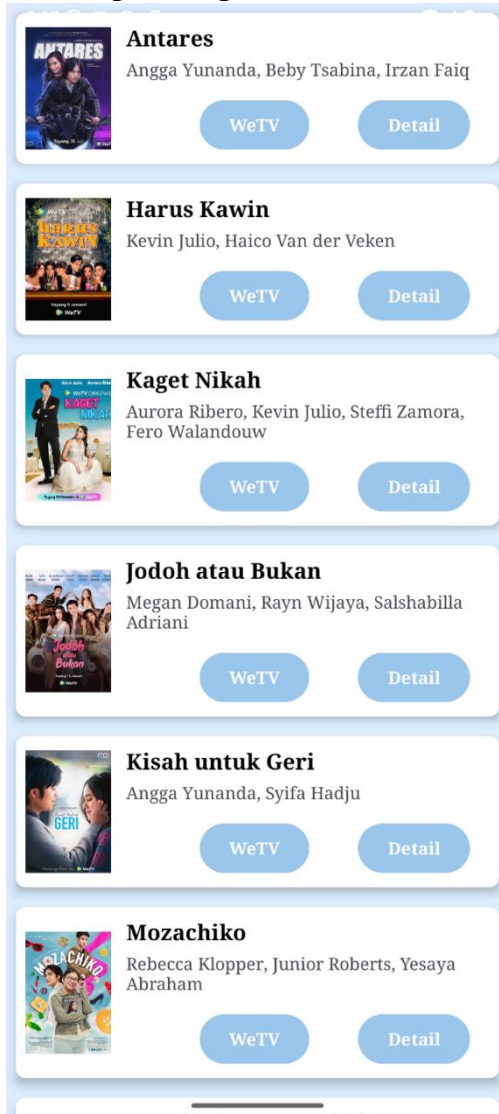

```

11     defaultConfig {
12         applicationId = "com.example.recycler_view"
13         minSdk = 30
14         targetSdk = 35
15         versionCode = 1
16         versionName = "1.0"
17
18         testInstrumentationRunner =
"androidx.test.runner.AndroidJUnitRunner"
19     }
20
21     buildTypes {
22         release {
23             isMinifyEnabled = false
24             proguardFiles(
25                 getDefaultProguardFile("proguard-
android-optimize.txt"),
26                 "proguard-rules.pro"
27             )
28         }
29     }
30     compileOptions {
31         sourceCompatibility = JavaVersion.VERSION_11
32         targetCompatibility = JavaVersion.VERSION_11
33     }
34     kotlinOptions {
35         jvmTarget = "11"
36     }
37     buildFeatures {
38         viewBinding = true

```

39	}
40	}
41	
42	dependencies {
43	
44	implementation(libs.androidx.core.ktx)
45	implementation(libs.androidx.appcompat)
46	implementation(libs.material)
47	implementation(libs.androidx.activity)
48	implementation("androidx.recyclerview:recyclerview:1.3.1")
49	implementation("androidx.constraintlayout:constraintlayout:2.1.4")
50	implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.6.1")
51	implementation("androidx.lifecycle:lifecycle-livedata-ktx:2.6.1")
52	implementation("androidx.fragment:fragment-ktx:1.5.7")
53	
54	testImplementation(libs.junit)
55	androidTestImplementation(libs.androidx.junit)
56	androidTestImplementation(libs.androidx.espresso.core)
57	}
58	}

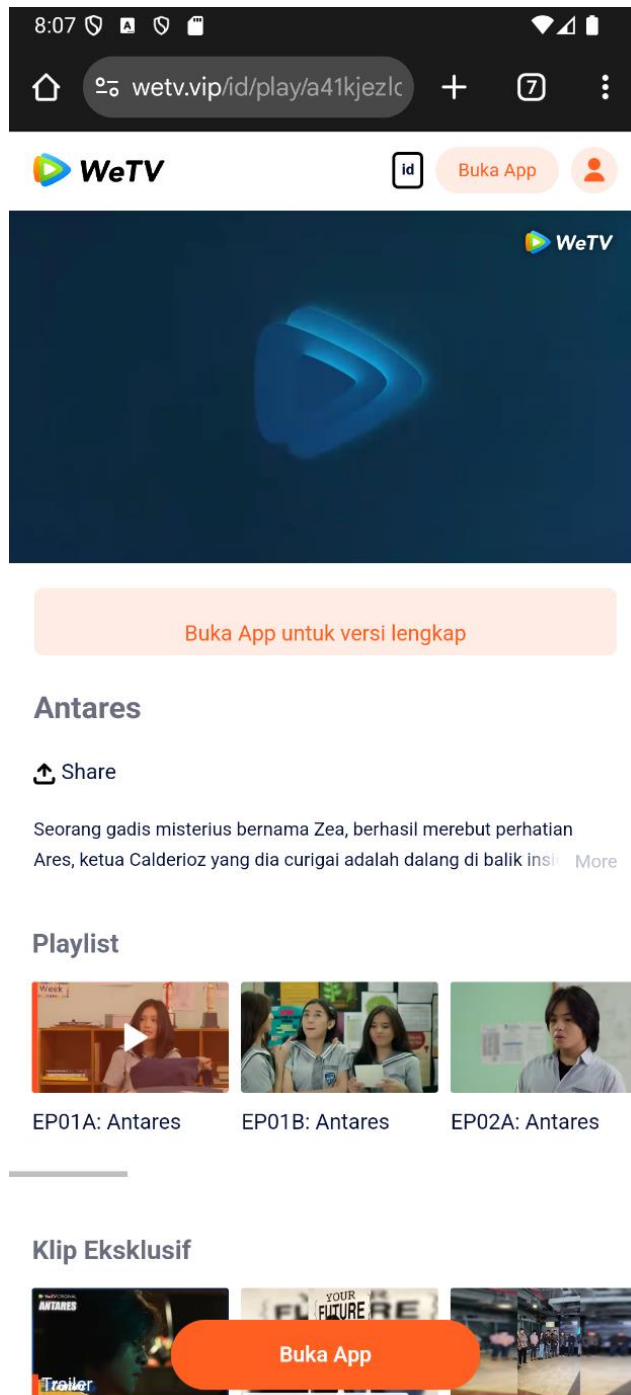
B. Output Program



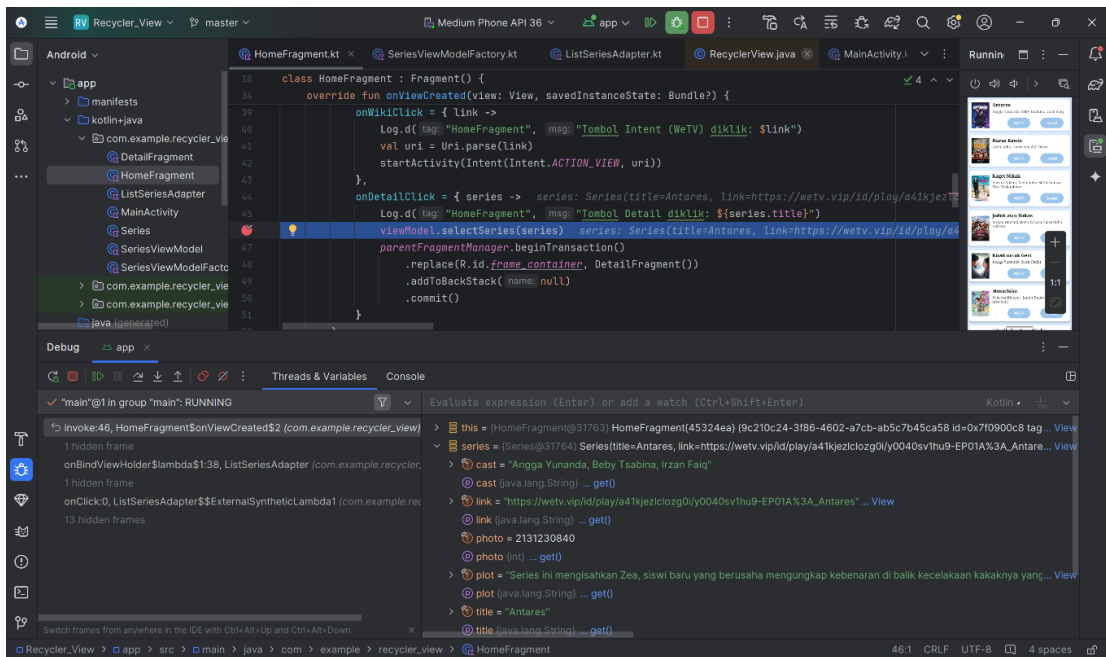
Gambar 24. Screenshot Hasil Tampilan Soal 1 Modul 4



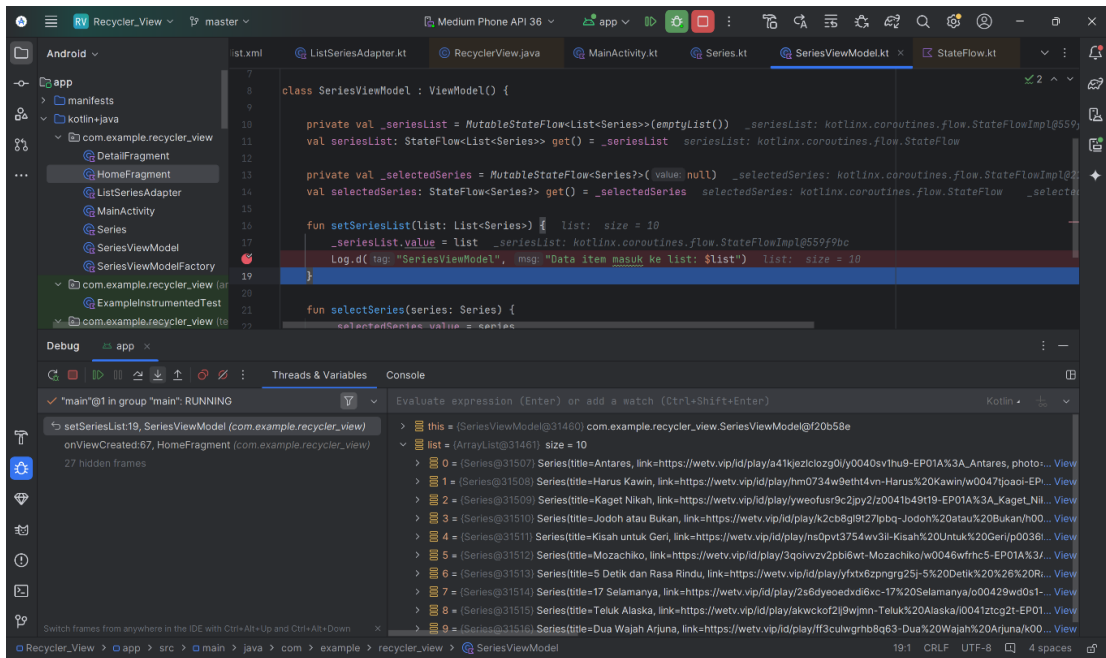
Gambar 25. Screenshot Hasil Tampilan Soal 1 Modul 4



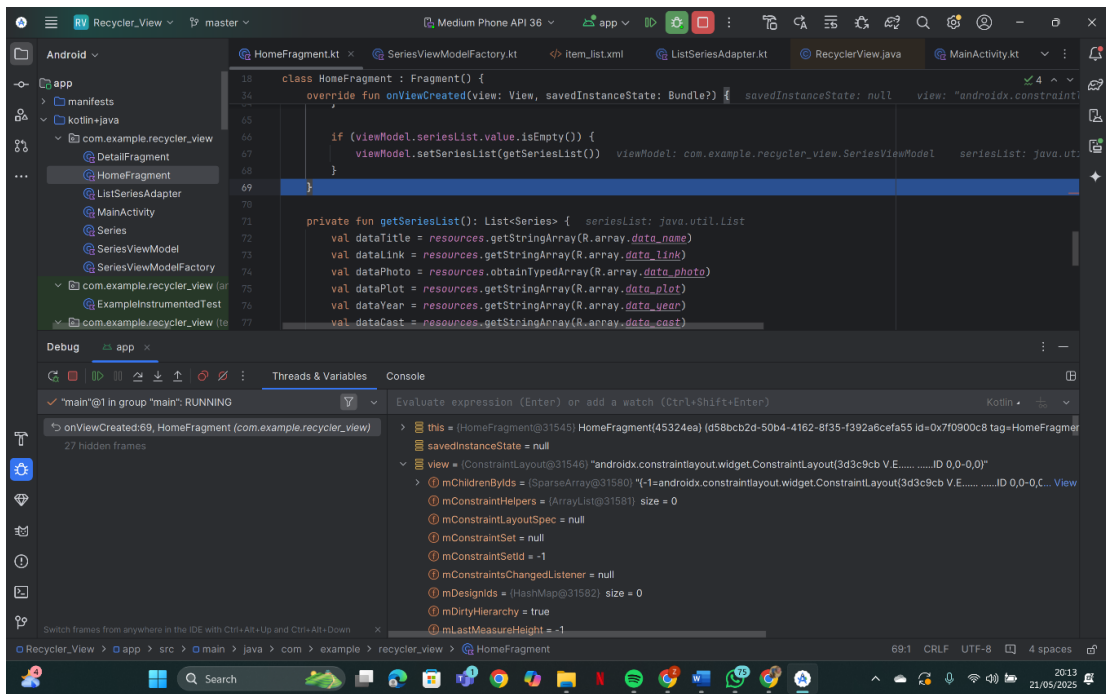
Gambar 26. Screenshot Hasil Tampilan Soal 1 Modul 4



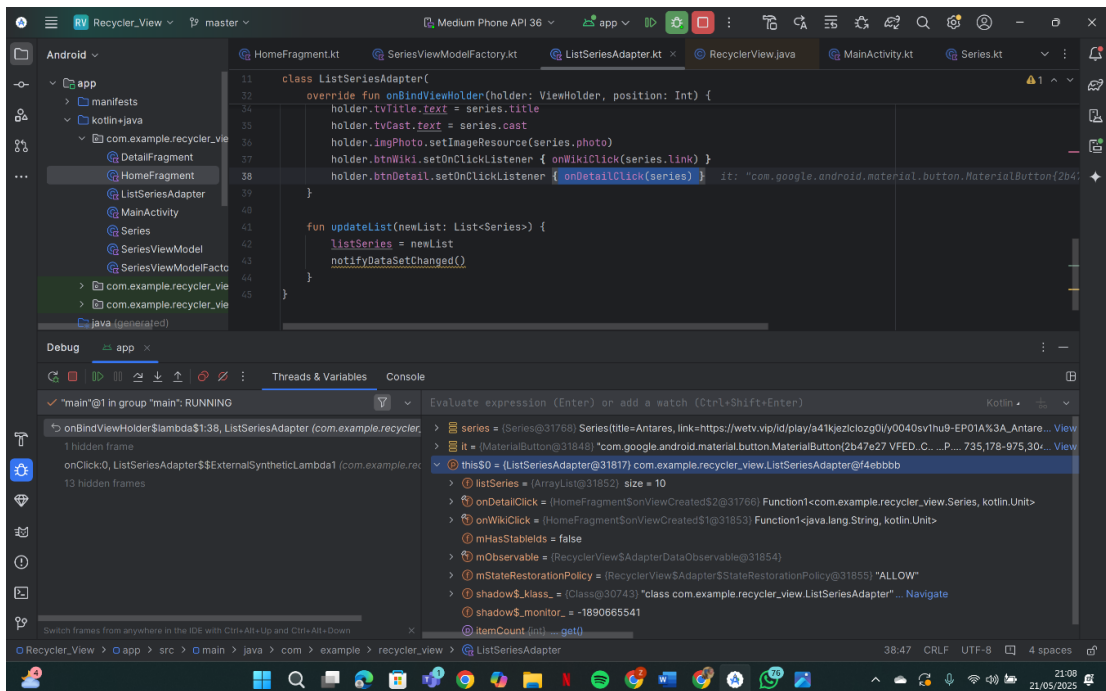
Gambar 27. Screenshot Hasil Tampilan Debugging Soal 1 Modul 4



Gambar 28. Screenshot Hasil Tampilan Debugging Step Into Soal 1 Modul 4



Gambar 29. Screenshot Hasil Tampilan Debugging Step Over Soal 1 Modul 4



Gambar 30. Screenshot Hasil Tampilan Debugging Step Out Soal 1 Modul 4

C. Pembahasan File MainActivity.kt

Kelas **MainActivity** mewarisi dari **AppCompatActivity**, yang merupakan superclass umum untuk aktivitas yang menggunakan fitur-fitur modern seperti **ActionBar**. Di dalam metode **onCreate()**, pertama-tama dipanggil **super.onCreate(savedInstanceState)** untuk memastikan bahwa logika inisialisasi dari superclass berjalan sebagaimana mestinya. Selanjutnya, **setContentView(R.layout.activity_main)** digunakan untuk menentukan layout XML yang akan digunakan sebagai tampilan utama dari aktivitas ini, dalam hal ini adalah **activity_main.xml**.

Setelah layout diatur, kode melanjutkan dengan inisialisasi **FragmentManager** melalui **supportFragmentManager**. **FragmentManager** ini digunakan untuk mengelola transaksi fragment seperti penambahan, penggantian, atau penghapusan fragment. Kemudian, dibuat sebuah *instance* dari **HomeFragment**, yaitu fragment yang akan ditampilkan dalam activity. Baris **val fragment = fragmentManager.findFragmentByTag(HomeFragment::class.java.simpleName)** digunakan untuk memeriksa apakah fragment dengan tag tertentu yang berdasarkan nama kelas **HomeFragment** sudah ada atau belum dalam manajer fragment.

Jika fragment yang ditemukan bukan *instance* dari **HomeFragment** (**fragment !is HomeFragment**), maka dilakukan transaksi untuk menambahkan fragment ke dalam container dengan ID **R.id.frame_container**. Metode **beginTransaction()** digunakan untuk memulai transaksi, kemudian **add()** menambahkan fragment ke dalam layout. Parameter **HomeFragment::class.java.simpleName** sebagai tag digunakan agar fragment ini bisa dikenali atau dicari kembali di kemudian hari. Terakhir, transaksi ini dikonfirmasi dan dijalankan dengan **commit()**.

File DetailFragment.kt

DetailFragment bertanggung jawab untuk menampilkan detail dari suatu series. **DetailFragment** mewarisi dari kelas **Fragment**, dan menggunakan *view binding* untuk mengakses elemen-elemen UI secara aman dan efisien tanpa perlu menggunakan **findViewById**.

View Binding dilakukan melalui objek **_binding** yang bertipe *nullable* dan diinisialisasi di dalam metode **onCreateView()** dengan memanggil **FragmentDetailBinding.inflate(...)**. Objek ini kemudian diakses menggunakan properti **binding** yang dijamin non-null selama siklus hidup view masih aktif.

Selain View Binding, **DetailFragment** juga terintegrasi dengan **SeriesViewModel**, yaitu sebuah ViewModel yang mengelola data series secara terpisah dari UI dan menyediakannya melalui **StateFlow** bernama **selectedSeries**. ViewModel ini dibuat menggunakan **SeriesViewModelFactory** untuk memastikan dependensi yang dibutuhkan tersedia. Ketika view fragment telah dibuat (**onViewCreated()**), fragment mulai mengobservasi **selectedSeries** menggunakan coroutine dalam **lifecycleScope.launch**. Penggunaan **collectLatest** memastikan bahwa setiap kali ada data series baru yang dipilih, UI akan diperbarui secara otomatis. Jika data yang diterima tidak null, maka informasi seperti **title**, **plot**, **year**, **cast**, dan **photo** dari series tersebut akan ditampilkan pada elemen-elemen UI yang sesuai menggunakan binding. Untuk mencegah terjadinya memory leak, fragment menerapkan praktik yang baik dengan menyetel **_binding** ke null di dalam metode **onDestroyView()**.

File HomeFragment.kt

Fragment ini memanfaatkan fitur View Binding melalui **FragmentHomeBinding**, yang memberikan akses langsung dan aman ke elemen-elemen antarmuka pengguna tanpa harus menggunakan

findViewById(). Dalam metode **onCreateView()**, layout fragment di-*inflate* dan dikembalikan sebagai root view. Kemudian, pada **onViewCreated()**, **RecyclerView** diatur menggunakan **LinearLayoutManager** untuk menampilkan daftar secara vertikal, dan dioptimalkan dengan **setHasFixedSize(true)** agar performanya lebih baik jika ukuran item tetap.

Sebuah adapter bernama **ListSeriesAdapter** dibuat dan dikaitkan dengan **RecyclerView**. Adapter ini menerima dua aksi **callback** melalui **onWikiClick**, yang akan membuka tautan WeTV dari sebuah series menggunakan **Intent** dan browser eksternal, serta **onDetailClick**, yang akan membuat instance **DetailFragment**. Pada saat detail diklik, data series yang dipilih dikirim ke **SeriesViewModel** melalui fungsi **selectSeries()**. Kemudian, **DetailFragment** ditampilkan menggunakan **FragmentTransaction**, dan ditambahkan ke back stack agar pengguna dapat kembali ke daftar sebelumnya.

Untuk mengisi data, **HomeFragment** mengambil informasi series dari resource XML melalui metode **getSeriesList()**. Data tersebut mencakup judul, tautan, gambar, alur cerita, tahun rilis, dan pemeran, yang dikemas ke dalam objek **Series**. Resource berupa **TypedArray** yang digunakan untuk gambar juga direcycle setelah selesai digunakan untuk menghindari kebocoran memori. Setelah data siap, jika **seriesList** pada **ViewModel** masih kosong, maka data dari resource akan dimasukkan ke dalam **ViewModel** menggunakan **setSeriesList()**.

Selain itu, fragment ini juga mengobservasi data dari **seriesList** di **ViewModel** menggunakan coroutine dalam **lifecycleScope.launch**. Setiap kali data diperbarui, adapter akan memanggil **updateList()** untuk menampilkan daftar yang baru. Terakhir, dalam **onDestroyView()**, binding disetel ke **null** untuk memastikan referensi ke tampilan dilepas ketika fragment dihancurkan, sehingga mencegah terjadinya memory leak.

File ListSeriesAdapter.kt

Kelas ini menampilkan daftar objek **Series** dalam bentuk daftar item. Adapter ini menerima tiga parameter utama dalam konstruktor sebuah **List<Series>** yang berisi data drama, sebuah lambda **onWikiClick** yang menangani aksi saat pengguna ingin membuka tautan WeTV, dan **onDetailClick** yang menangani aksi ketika pengguna ingin melihat detail drama.

Di dalam adapter terdapat kelas **ViewHolder**, yang bertanggung jawab untuk menampung dan menginisialisasi elemen-elemen UI dari setiap item, yaitu **ImageView** untuk **imgPhoto**, dua **TextView** untuk **tvTitle**, dan **tvPlot**, serta dua **Button** masing-masing untuk membuka **btnWiki** dan **btnDetail**. Layout setiap item dibuat dengan menggunakan file **item_list.xml** yang di-inflate di dalam metode **onCreateViewHolder()**. Metode ini mengembalikan objek **ViewHolder** yang siap digunakan.

Metode **getItemCount()** akan mengembalikan jumlah total item dalam daftar **listSeries**, yang menentukan seberapa banyak item yang akan ditampilkan di layar. Kemudian, pada **onBindViewHolder()**, adapter akan mengikat data ke elemen UI berdasarkan posisi item dalam daftar. Di sini, elemen-elemen seperti **title**, **photo**, dan **cast** dimasukkan ke tampilan masing-masing, dan tombol-tombol diatur dengan listener yang memanggil fungsi callback yang telah didefinisikan. Misalnya, **btnWiki** akan membuka link WeTV menggunakan fungsi **onWikiClick**, sedangkan **btnDetail** akan memicu **onDetailClick** yang akan membawa data drama ke tampilan detail.

Untuk mendukung pembaruan data, adapter menyediakan fungsi **updateList()**, yang akan mengganti isi **listSeries** dengan daftar baru, lalu memanggil **notifyDataSetChanged()** untuk memberi tahu **RecyclerView** bahwa data telah berubah dan tampilan harus diperbarui. Dengan desain seperti ini, **ListSeriesAdapter** mampu menampilkan dan mengelola daftar series secara dinamis dan interaktif dalam aplikasi.

File Series.kt

Kelas ini menggunakan anotasi **@Parcelize** dan mengimplementasikan antarmuka **Parcelable**, yang memungkinkan objek **Series** untuk dikirim antar komponen Android (seperti antar **Activity** atau **Fragment**) melalui **Bundle**.

Setiap instance dari **Series** memiliki enam property yaitu, **title** bertipe **String** untuk menyimpan judul serial, **link** untuk menyimpan URL untuk mengarah ke web WeTV, **photo** bertipe **Int** yang merupakan resource ID dari drawable, **plot** yang berisi sinopsis cerita, **year** untuk tahun rilis, dan **cast** yang memuat daftar pemeran.

Dengan penggunaan **@Parcelize**, proses pembuatan parcel (objek yang bisa dipaketkan) menjadi otomatis, sehingga tidak perlu menuliskan kode boilerplate seperti **writeToParcel()** atau **describeContents()**.

File SeriesViewModel.kt

Kelas **SeriesViewModel** adalah kelas turunan dari **ViewModel** yang bertanggung jawab untuk menyimpan dan mengelola data terkait daftar dan item **Series** yang dipilih dalam aplikasi. ViewModel ini memanfaatkan **StateFlow** dari Kotlin Coroutines untuk menyediakan data secara reaktif dan dapat diobservasi oleh komponen UI seperti **Fragment**. Properti **_seriesList** bertipe **MutableStateFlow<List<Series>>** menyimpan seluruh daftar series, sedangkan **seriesList** yang diekspos ke luar sebagai **StateFlow** digunakan agar data dapat diamati tanpa bisa diubah dari luar. Dengan pendekatan ini, data daftar series akan tetap konsisten meskipun konfigurasi ulang seperti rotasi layar terjadi.

Selain itu, ViewModel juga menyimpan state **Series** yang sedang dipilih dalam properti **_selectedSeries**, dan menyediakan versi publik **selectedSeries** yang bersifat read-only. Fungsi **setSeriesList()** digunakan untuk menetapkan atau memperbarui isi dari daftar series yang dimiliki ViewModel. Fungsi ini juga menuliskan log untuk membantu proses debugging.

Sementara itu, fungsi **selectSeries()** digunakan untuk menetapkan satu item **Series** yang sedang dipilih, dan juga mencatat informasi item yang dipilih ke dalam log. Pendekatan ini memungkinkan fragment seperti **HomeFragment** dan **DetailFragment** untuk saling berbagi data melalui ViewModel dengan cara yang aman, efisien, dan sesuai arsitektur Android modern.

File SeriesViewModelFactory.kt

Kelas **SeriesViewModelFactory** merupakan implementasi dari antarmuka **ViewModelProvider.Factory** yang digunakan untuk membuat instance dari **SeriesViewModel**. Factory ini diperlukan karena **ViewModel** tidak boleh langsung diinstansiasi menggunakan constructor biasa, terutama saat menggunakan mekanisme **by viewModel** di fragment atau activity, agar pembuatan **ViewModel** tetap terpisah dari komponen UI dan mengikuti prinsip dependency injection.

Metode **create()** di dalam factory ini akan dipanggil oleh sistem saat sebuah **ViewModel** dibutuhkan. Fungsi ini menerima parameter **modelClass**, yaitu kelas **ViewModel** yang diminta. Di dalamnya dilakukan pengecekan apakah **modelClass** sesuai dengan **SeriesViewModel**. Jika cocok, maka instance baru dari **SeriesViewModel** dibuat dan dikembalikan dengan casting ke tipe generik **T**. Jika tidak cocok, maka factory akan melemparkan **IllegalArgumentException** sebagai penanda bahwa kelas yang diminta tidak dikenali. Dengan menggunakan **SeriesViewModelFactory**, pembuatan **ViewModel** menjadi lebih fleksibel dan aman, serta mendukung penerapan arsitektur MVVM secara bersih.

File activity_main.xml

Kode XML tersebut merupakan layout utama dari **MainActivity** dalam aplikasi Android yang menggunakan **ConstraintLayout** sebagai root layout. Di

dalamnya terdapat satu komponen penting, yaitu **FrameLayout** dengan ID **@+id/frame_container**. Elemen ini berfungsi sebagai **wadah** untuk menampung dan menampilkan **Fragment**, misalnya **HomeFragment** atau **DetailFragment**, yang dimuat secara dinamis melalui transaksi fragment dari kode Kotlin.

Tag **ConstraintLayout** menyediakan kemampuan untuk menempatkan dan mengatur posisi elemen-elemen UI secara fleksibel menggunakan constraint antar komponen atau ke parent layout. Namun dalam kasus ini, hanya ada satu elemen di dalamnya, yaitu **FrameLayout**. Meskipun begitu, karena ukuran **FrameLayout** diatur ke **match_parent** baik untuk lebar maupun tinggi, maka elemen ini akan mengisi seluruh ruang yang tersedia dalam layar.

Atribut **tools:context=".MainActivity"** digunakan oleh Android Studio saat preview layout di editor, untuk memberi tahu bahwa layout ini digunakan dalam **MainActivity**.

File fragment_detail.xml

Layout ini menggunakan **ScrollView** sebagai root. Di dalam **ScrollView**, terdapat **ConstraintLayout** sebagai container utama untuk menata elemen-elemen UI secara fleksibel. Elemen pertama adalah **ImageView** dengan ID **img_item_photo**, yang menampilkan gambar serial dengan ukuran tetap (**341dp x 458dp**) dan **scaleType centerCrop** agar gambar terisi penuh namun tetap proporsional. Gambar ini ditampilkan di bagian atas layout dan diberi margin atas **32dp**. Terdapat empat buah **TextView** yaitu, **tv_name** untuk menampilkan nama atau judul serial. ukuran teks **25sp, bold**, dan menggunakan font **serif** agar tampak khas seperti judul buku atau film, **tv_year** untuk menampilkan tahun rilis dengan label “Tahun:”, **tv_cast** untuk menampilkan pemeran dari serial, diletakkan tepat di bawah tahun dan dipusatkan, **tv_plot** untuk menampilkan sinopsis atau alur cerita dari serial tersebut, juga dipusatkan dan diberi margin agar tampak rapi.

Semua elemen diposisikan menggunakan **ConstraintLayout**, yaitu dengan menentukan hubungan seperti **app:layout_constraintTop_toBottomOf**, **layout_constraintStart_toStartOf**, dan **layout_constraintEnd_toEndOf** agar konten tersusun secara vertikal dan simetris di tengah layar.

Atribut **android:background="@drawable/bg"** pada **ScrollView** menunjukkan bahwa latar belakang layar menggunakan gambar atau bentuk dari resource drawable dengan nama **bg**.

File fragment_home.xml

Kelas ini root layout-nya menggunakan **ConstraintLayout**, yang memberi fleksibilitas tinggi dalam menyusun elemen-elemen UI. Layout ini juga memiliki latar belakang bergambar melalui atribut **android:background="@drawable/bg2"**, yang memberikan estetika visual yang lebih menarik dibandingkan latar belakang polos.

Satu-satunya elemen yang dimuat dalam **ConstraintLayout** ini adalah **RecyclerView** dengan ID **rvDrama**. Komponen ini digunakan untuk menampilkan daftar series secara efisien, karena hanya memuat tampilan yang terlihat di layar dan mendaur ulang elemen lainnya. **RecyclerView** diatur agar mengisi seluruh area layar dengan menggunakan constraint ke setiap **Top**, **Bottom**, **Start**, **End**, serta lebar dan tinggi diset ke **0dp** yang berarti akan mengikuti constraint yang telah didefinisikan.

Layout ini bekerja bersama dengan **DramaAdapter** dan data dari **Series** untuk menampilkan daftar item yang interaktif, di mana pengguna bisa menekan tombol untuk membuka halaman detail atau tautan WeTV dari serial tersebut.

File item_list.xml

Root elemen dari layout ini adalah **CardView**, yang memberikan tampilan kartu dengan sudut melengkung dengan menggunakan **cardCornerRadius="8dp"** dan bayangan dengan menggunakan **cardElevation="4dp"**, menciptakan tampilan modern dan terangkat dari latar belakang.

Di dalam **CardView**, terdapat **ConstraintLayout** sebagai wadah utama untuk mengatur tata letak elemen UI seperti gambar, teks, dan tombol dengan posisi yang fleksibel dan presisi. Bagian kiri layout menampilkan **ImageView** (**@+id/img_item_photo**) berukuran **70x100dp** yang menampilkan gambar sampul serial dengan skala **centerCrop**, sehingga gambar akan memenuhi area tampilan secara proporsional.

Di samping gambar, terdapat dua **TextView**: satu untuk nama series dengan menggunakan **@+id/tv_item_name** dengan ukuran teks besar dan gaya tebal untuk menonjolkan judul, dan satu lagi di bawahnya yaitu **@+id/tv_item_plot** yang menampilkan informasi singkat seperti nama pemeran. Kedua teks ini menggunakan font **serif**, memberi nuansa klasik dan rapi. Di bagian bawah, terdapat dua tombol **Button** yang berfungsi sebagai aksi untuk pengguna. **btn_wiki** diberi label "WeTV" dan kemungkinan digunakan untuk membuka link eksternal yaitu WeTV. Tombol kedua yaitu **button_detail** membuka halaman detail serial. Kedua tombol ini didesain dengan **backgroundTint** berwarna biru muda dengan kode warna **#9bc6ec**, sehingga tampak serasi.

File colors.xml

Dalam kelas ini terdapat beberapa warna yang didefinisikan untuk digunakan secara konsisten di seluruh aplikasi, yaitu:

- **black** – Diberi nilai **#FF000000**, yang merupakan warna hitam sepenuhnya.
- **white** – Diberi nilai **#FFFFFFFF**, warna putih dengan opasitas penuh.

- **blue** – Diberi nilai **#9bc6ec**, warna biru muda yang sering dipakai dalam elemen UI seperti latar belakang tombol (**backgroundTint**) agar terlihat segar dan menarik secara visual.

File strings.xml

Terdapat tiga string tunggal yaitu **app_name**, **btn_detail**, dan **btn_wiki** yang masing-masing digunakan sebagai nama aplikasi, label tombol detail, dan tombol WeTV. Selanjutnya, file ini berisi beberapa **string-array** yang digunakan untuk menampilkan data dinamis dalam daftar seperti **RecyclerView**. **data_name** berisi daftar judul drama, **data_plot** berisi sinopsis singkat tiap drama, **data_year** mencatat tanggal rilisnya, **data_cast** menampilkan nama-nama pemeran utama, dan **data_photo** untuk menampilkan poster tiap series. Masing-masing elemen dalam array ini disusun secara paralel, artinya urutannya harus konsisten satu sama lain agar data bisa ditampilkan dengan benar.

File build.gradle.kts (:app)

Pada file **build.gradle.kts** (:app) yang ditampilkan merupakan berkas konfigurasi utama untuk aplikasi Android berbasis Kotlin yang menggunakan DSL (*Domain Specific Language*) Kotlin. File ini berperan dalam mendefinisikan proses build aplikasi, termasuk plugin yang digunakan, konfigurasi SDK Android, fitur build, serta daftar dependensi. Di bagian atas file, terdapat deklarasi tiga plugin utama:

```
com.android.application,
org.jetbrains.kotlin.android,
org.jetbrains.kotlin.plugin.parcelize.
```

dan
Plugin

com.android.application digunakan untuk menyatakan bahwa modul ini adalah aplikasi Android. Plugin **kotlin.android** memungkinkan penggunaan bahasa Kotlin dalam pengembangan Android, sementara plugin **kotlin-parcelize** memungkinkan penggunaan anotasi **@Parcelize** untuk mempermudah implementasi **Parcelable**.

Blok **android** memuat konfigurasi inti proyek. Properti **namespace** diatur ke "**com.example.recycler_view**", yang berfungsi sebagai ruang nama unik aplikasi. Versi **compileSdk**, **targetSdk**, dan **minSdk** masing-masing ditetapkan ke 35, 35, dan 30. Ini menandakan bahwa aplikasi dikembangkan menggunakan API level 35 dan dapat dijalankan pada perangkat Android dengan API level 30 ke atas (Android 11+). Bagian **defaultConfig** juga mendefinisikan **applicationId**, **versionCode**, dan **versionName**, yang diperlukan dalam distribusi dan pembaruan aplikasi di Google Play. **testInstrumentationRunner** disetel ke **androidx.test.runner.AndroidJUnitRunner** untuk keperluan pengujian instrumentasi.

Dalam blok **buildTypes**, terdapat konfigurasi **release** yang menonaktifkan proses **minifyEnabled**, artinya kode tidak akan dimampatkan saat proses rilis, dan **proguardFiles** menunjukkan penggunaan konfigurasi default ProGuard serta file **proguard-rules.pro** untuk aturan kustom. Kemudian, **compileOptions** dan **kotlinOptions** menetapkan bahwa proyek ini menggunakan Java 11 baik sebagai target maupun sumber kompatibilitas, serta mengarahkan Kotlin untuk menargetkan JVM versi 11.

Selanjutnya, fitur **viewBinding** diaktifkan melalui **buildFeatures**, yang memungkinkan pengaksesan langsung ke elemen UI tanpa menggunakan **findViewById**, sehingga meningkatkan keamanan dan efisiensi pengembangan.

Pada bagian **dependencies**, berbagai pustaka penting ditambahkan untuk menunjang pengembangan. Beberapa dependensi dideklarasikan menggunakan **Version Catalog**, seperti **libs.androidx.core.ktx**, **libs.androidx.appcompat**, **libs.material**, dan **libs.androidx.activity**, yang mengacu pada konfigurasi versi di file **libs.versions.toml**. Sementara itu, pustaka **RecyclerView**, **ConstraintLayout**, serta pustaka untuk **lifecycle-viewmodel-ktx**, **lifecycle-livedata-ktx**, dan **fragment-ktx** ditambahkan secara

eksplisit. Untuk mendukung pengujian, digunakan **junit** untuk unit testing dan **androidx.junit** serta **espresso.core** untuk instrumented testing.

Debugging merupakan proses identifikasi dan perbaikan kesalahan (bug) dalam sebuah program. Pada Android Studio, pengguna dapat menetapkan breakpoint pada baris kode tertentu untuk menghentikan sementara eksekusi program, sehingga memungkinkan pemeriksaan nilai variabel dan alur eksekusi secara detail. Terdapat tiga fitur utama dalam proses debugging yang sangat penting, yaitu *Step Over*, *Step Into*, dan *Step Out*. *Step Over* berfungsi untuk melewati pemanggilan fungsi tanpa memasuki isi fungsi tersebut, yang berguna ketika pengguna hanya ingin mengetahui bahwa fungsi telah dipanggil tanpa perlu memeriksa implementasinya secara rinci. Sebagai contoh, saat menekan *Step Over* pada baris pemanggilan fungsi, debugger akan langsung melanjutkan ke baris berikutnya tanpa memasuki fungsi yang dipanggil. Sebaliknya, *Step Into* memungkinkan pengguna untuk masuk ke dalam fungsi yang sedang dipanggil sehingga setiap baris kode di dalam fungsi tersebut dapat ditelusuri secara detail. Apabila pengguna sudah berada dalam sebuah fungsi dan ingin kembali ke fungsi pemanggil, fitur *Step Out* digunakan untuk keluar dari fungsi saat ini dan melanjutkan eksekusi di luar fungsi tersebut. Shortcut keyboard yang umum digunakan untuk ketiga fitur ini adalah **F8** untuk *Step Over*, **F7** untuk *Step Into*, dan **Shift + F8** untuk *Step Out* pada sistem operasi Windows maupun Mac

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

A. Pembahasan

Dalam arsitektur aplikasi Android, kelas **Application** adalah komponen penting yang berfungsi sebagai titik masuk global untuk status dan konfigurasi aplikasi. Kelas ini merupakan subclass dari **android.app.Application** dan hanya ada satu instance **Application** yang dibuat oleh sistem saat proses aplikasi dimulai, sebelum aktivitas, layanan, atau komponen lain diinisialisasi. Fungsi Utama *Application Class*:

1. Inisialisasi Global

Kelas **Application** ideal digunakan untuk melakukan setup global yang dibutuhkan oleh seluruh aplikasi, seperti:

- Inisialisasi dependency injection (misalnya Hilt, Dagger, Koin)
- Inisialisasi pustaka pihak ketiga (misalnya Firebase, Timber, Retrofit)
- Menentukan konfigurasi logging atau analitik
- Membuat instance singleton atau cache global

2. Menyediakan Konteks Aplikasi

Kelas ini menyediakan **applicationContext**, yaitu konteks global yang tidak terkait dengan lifecycle aktivitas atau fragment mana pun. Ini berguna untuk komponen yang butuh konteks, tetapi tidak terikat pada UI.

3. Monitoring Lifecycle Aplikasi

Sejak API 14 (Android 4.0), **Application** dapat memonitor lifecycle aktivitas menggunakan **Application.ActivityLifecycleCallbacks**. Ini memungkinkan pengembang melacak kapan aktivitas dimulai, dihentikan, atau dihancurkan — berguna untuk analitik, pelacakan penggunaan, atau manajemen status global.

4. Pengelolaan Resource Global

Application digunakan untuk menyimpan data global (misalnya token autentikasi, pengaturan pengguna) yang bisa diakses dari mana saja

selama proses aplikasi berjalan. Namun, perlu hati-hati agar tidak menyebabkan memory leak atau pelanggaran prinsip modularitas.

MODUL 5: Connect to the Internet

SOAL 1

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- Gunakan KotlinX Serialization sebagai library JSON.
- Gunakan library seperti Coil atau Glide untuk image loading.
- API yang digunakan pada modul ini bebas, contoh API gratis The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started>
- Implementasikan konsep data persistence (misalnya offline-first app, pengaturan dark/light mode, fitur favorite, dll)
- Gunakan caching strategy pada Room.
- Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya

A. Source Code

Tabel 36. Source Code Soal 1 Modul 5

File AndroidManifest.xml	
1	<?xml version="1.0" encoding="utf-8"?>
2	<manifest
	xmlns:android="http://schemas.android.com/apk/res/and
	roid"
3	xmlns:tools="http://schemas.android.com/tools">
4	

5	<uses-permission
	android:name="android.permission.INTERNET" />
6	
7	<application
8	android:allowBackup="true"
9	android:dataExtractionRules="@xml/data_extraction_rules"
10	android:fullBackupContent="@xml/backup_rules"
11	android:icon="@mipmap/ic_launcher"
12	android:label="@string/app_name"
13	android:roundIcon="@mipmap/ic_launcher_round"
14	android:supportsRtl="true"
15	android:theme="@style/Theme.MovieList"
16	tools:targetApi="31">
17	<activity
18	android:name=".presentation.ui.activity.MainActivity"
19	android:exported="true">
20	<intent-filter>
21	<action
	android:name="android.intent.action.MAIN" />
22	<category
	android:name="android.intent.category.LAUNCHER" />
23	</intent-filter>
24	</activity>
25	
26	<activity
27	android:name=".presentation.ui.activity.DetailActivity"
28	android:exported="false">
29	</activity>

30	
31	</application>
32	
33	</manifest>

Tabel 37. Source Code Soal 1 Modul 5

File MainActivity.kt	
1	package
	com.example.movielist.presentation.ui.activity
2	
3	import android.content.Intent
4	import android.os.Bundle
5	import android.view.View
6	import android.widget.Toast
7	import androidx.activity.viewModels
8	import androidx.appcompat.app.AppCompatActivity
9	import androidx.appcompat.app.AppCompatActivity
10	import androidx.lifecycle.Observer
11	import
	androidx.recyclerview.widget.LinearLayoutManager
12	import androidx.room.Room
13	import
	com.example.movielist.data.local.MovieAppPreference
	s
14	import
	com.example.movielist.data.local.database.AppDataaba
	se
15	import
	com.example.movielist.data.remote.api.RetrofitClien
	t

16	import com.example.movielist.data.repository.MovieRepositoryImpl
17	import com.example.movielist.databinding.ActivityMainBinding
18	import com.example.movielist.domain.usecase.GetPopularMoviesUseCase
19	import com.example.movielist.presentation.ui.adapter.MovieAdapter
20	import com.example.movielist.presentation.viewmodel.MovieViewModel
21	import com.example.movielist.presentation.viewmodel.ViewModelFactory
22	import com.example.movielist.utils.Result
23	
24	class MainActivity : AppCompatActivity() {
25	
26	private lateinit var binding: ActivityMainBinding
27	private lateinit var movieAdapter: MovieAdapter
28	private lateinit var movieAppPreferences: MovieAppPreferences
29	
30	private val movieViewModel: MovieViewModel by viewModels {

31	val apiService =
	RetrofitClient.tmdbApiService
32	val database = Room.databaseBuilder(
33	applicationContext,
34	AppDatabase::class.java,
35	AppDatabase.DATABASE_NAME
36	).build()
37	val movieDao = database.movieDao()
38	
39	val tmdbApiKey =
	"71819bfeac768c2a5b9a32b26e50cael"
40	movieAppPreferences.saveApiKey(tmdbApiKey)
41	
42	val movieRepositoryImpl =
	MovieRepositoryImpl(apiService, movieDao,
	tmdbApiKey)
43	val getPopularMoviesUseCase =
	GetPopularMoviesUseCase(movieRepositoryImpl)
44	ViewModelFactory(getPopularMoviesUseCase)
45	}
46	
47	override fun onCreate(savedInstanceState:
	Bundle?) {
48	super.onCreate(savedInstanceState)
49	binding =
	ActivityMainBinding.inflate(layoutInflater)
50	setContentView(binding.root)
51	
52	movieAppPreferences =
	MovieAppPreferences(this)

53	
54	setupRecyclerView()
55	observeViewModel()
56	setupDarkModeToggle()
57	
58	binding.btnRetry.setOnClickListener {
59	movieViewModel.fetchPopularMovies()
60	}
61	}
62	
63	private fun setupRecyclerView() {
64	movieAdapter = MovieAdapter()
65	binding.rvMovies.apply {
66	layoutManager =
67	LinearLayoutManager(this@MainActivity)
68	adapter = movieAdapter
69	}
70	movieAdapter.onItemClick = { movie ->
71	val intent = Intent(this,
72	DetailActivity::class.java).apply {
73	putExtra(DetailActivity.EXTRA_MOVIE,
74	movie)
75	}
76	startActivity(intent)
77	}
78	private fun observeViewModel() {
79	movieViewModel.popularMovies.observe(this,

	<i>Observer { result -></i>
80	<i>when (result) {</i>
81	<i>is Result.Loading -> {</i>
82	<i>binding.progressBar.visibility</i>
	<i>= View.VISIBLE</i>
83	<i>binding.tvError.visibility =</i>
	<i>View.GONE</i>
84	<i>binding.btnRetry.visibility =</i>
	<i>View.GONE</i>
85	<i>binding.rvMovies.visibility =</i>
	<i>View.GONE</i>
86	<i>}</i>
87	<i>is Result.Success -> {</i>
88	<i>binding.progressBar.visibility</i>
	<i>= View.GONE</i>
89	<i>binding.tvError.visibility =</i>
	<i>View.GONE</i>
90	<i>binding.btnRetry.visibility =</i>
	<i>View.GONE</i>
91	<i>binding.rvMovies.visibility =</i>
	<i>View.VISIBLE</i>
92	<i>movieAdapter.submitList(result.data)</i>
93	<i>}</i>
94	<i>is Result.Error -> {</i>
95	<i>binding.progressBar.visibility</i>
	<i>= View.GONE</i>
96	<i>binding.rvMovies.visibility =</i>
	<i>View.GONE</i>
97	<i>binding.tvError.visibility =</i>
	<i>View.VISIBLE</i>

98	binding.btnRetry.visibility =
	View.VISIBLE
99	binding.tvError.text = "Error:
	\${result.exception.message}"
100	Toast.makeText(this, "Error:
	\${result.exception.message}",
	Toast.LENGTH_LONG).show()
101	}
102	}
103	})
104	}
105	
106	private fun setupDarkModeToggle() {
107	binding.switchDarkMode.isChecked =
	movieAppPreferences.getDarkModeState()
108	
109	applyTheme(movieAppPreferences.getDarkModeState())
110	
111	binding.switchDarkMode.setOnCheckedChangeListener
	{ _, isChecked ->
112	movieAppPreferences.saveDarkModeState(isChecked)
113	applyTheme(isChecked)
114	}
115	}
116	
117	private fun applyTheme(isDarkMode: Boolean) {
118	if (isDarkMode) {
119	AppCompatDelegate.setDefaultNightMode(AppCompatDele
	gate.MODE_NIGHT_YES)
120	} else {

121	AppCompatActivity.setDefaultNightMode (AppCompatActivity.MODE_NIGHT_NO)
122	}
123	}
124	}

Tabel 38. Source Code Soal 1 Modul 5

File MovieDao.kt	
1	package com.example.movielist.data.local.dao
2	
3	import androidx.room.Dao
4	import androidx.room.Insert
5	import androidx.room.OnConflictStrategy
6	import androidx.room.Query
7	import
	com.example.movielist.data.local.entities.MovieEntity
8	
9	@Dao
10	interface MovieDao {
11	@Insert(onConflict = OnConflictStrategy.REPLACE)
12	suspend fun insertAllMovies (movies:
	List<MovieEntity>)
13	
14	@Query("SELECT * FROM movies ORDER BY popularity
	DESC")
15	suspend fun getAllMovies(): List<MovieEntity>
16	
17	@Query("DELETE FROM movies")
18	suspend fun clearAllMovies()
19	}

Tabel 39. Source Code Soal 1 Modul 5

File AppDatabase.kt	
1	Package com.example.movielist.data.local.database
2	
3	import androidx.room.Database
4	import androidx.room.RoomDatabase
5	import com.example.movielist.data.local.dao.MovieDao
6	import
	com.example.movielist.data.local.entities.MovieEntity
7	
8	@Database(entities = [MovieEntity::class], version =
	1, exportSchema = false)
9	abstract class AppDatabase : RoomDatabase() {
10	abstract fun movieDao(): MovieDao
11	
12	companion object {
13	const val DATABASE_NAME = "tmdb_app_db"
14	}
15	}

Tabel 40. Source Code Soal 1 Modul 5

File MovieEntity.kt	
1	package com.example.movielist.data.local.entities
2	
3	import androidx.room.Entity
4	import androidx.room.PrimaryKey
5	import com.example.movielist.domain.model.Movie
6	
7	@Entity(tableName = "movies")

```

8  data class MovieEntity(
9      @PrimaryKey
10     val id: Int,
11     val title: String,
12     val overview: String,
13     val posterPath: String?,
14     val releaseDate: String,
15     val voteAverage: Double,
16     val popularity: Double
17 ) {
18     fun toDomainMovie(): Movie {
19         return Movie(
20             id = id,
21             title = title,
22             overview = overview,
23             posterPath = posterPath,
24             releaseDate = releaseDate,
25             voteAverage = voteAverage
26         )
27     }
28
29     companion object {
30         fun fromDomainMovie(movie: Movie, popularity:
31 Double): MovieEntity {
32             return MovieEntity(
33                 id = movie.id,
34                 title = movie.title,
35                 overview = movie.overview,
36                 posterPath = movie.posterPath,
37                 releaseDate = movie.releaseDate,

```

37	voteAverage = movie.voteAverage,
38	popularity = popularity
39	)
40	}
41	}
42	}

Tabel 41. Source Code Soal 1 Modul 5

File MovieAppPreferences.kt	
1	package com.example.movielist.data.local
2	
3	import android.content.Context
4	import android.content.SharedPreferences
5	
6	class MovieAppPreferences(context: Context) {
7	
8	private val sharedPreferences: SharedPreferences
	=
9	context.getSharedPreferences("tmdb_app_prefs",
	Context.MODE_PRIVATE)
10	
11	companion object {
12	private const val KEY_API_KEY = "api_key"
13	private const val KEY_DARK_MODE =
	"dark_mode"
14	}
15	
16	fun saveApiKey(apiKey: String) {
17	sharedPreferences.edit().putString(KEY_API_KEY,

	apiKey).apply()
18	}
19	
20	fun getApiKey(): String? {
21	return
	sharedPreferences.getString(KEY_API_KEY, null)
22	}
23	
24	fun saveDarkModeState(isDarkMode: Boolean) {
25	sharedPreferences.edit().putBoolean(KEY_DARK_MODE,
	isDarkMode).apply()
26	}
27	
28	fun getDarkModeState(): Boolean {
29	return
	sharedPreferences.getBoolean(KEY_DARK_MODE, false)
30	}
31	}

Tabel 42. Source Code Soal 1 Modul 5

File RetrofitClient.kt	
1	package com.example.movielist.data.remote.api
2	
3	import
	com.jakewharton.retrofit2.converter.kotlinx.serialization.asConverterFactory
4	import kotlinx.serialization.json.Json
5	import okhttp3.MediaType.Companion.toMediaType
6	import okhttp3.OkHttpClient
7	import okhttp3.logging.HttpLoggingInterceptor

```

8 import retrofit2.Retrofit
9 import java.util.concurrent.TimeUnit
10
11 object RetrofitClient {
12
13     private const val BASE_URL =
14         "https://api.themoviedb.org/3/"
15
16     private val json = Json {
17         ignoreUnknownKeys = true
18         prettyPrint = true
19     }
20
21     private val okHttpClient: OkHttpClient by lazy {
22         val logging = HttpLoggingInterceptor()
23         logging.setLevel(HttpLoggingInterceptor.Level.BODY)
24
25         OkHttpClient.Builder()
26             .addInterceptor(logging)
27             .connectTimeout(30, TimeUnit.SECONDS)
28             .readTimeout(30, TimeUnit.SECONDS)
29             .writeTimeout(30, TimeUnit.SECONDS)
30             .build()
31     }
32
33     val tmdbApiService: TmdbApiService by lazy {
34         Retrofit.Builder()
35             .baseUrl(BASE_URL)
36             .client(okHttpClient)
37             .addConverterFactory(json.asConverterFactory("applica

```

	tion/json".toMediaType()))
37	.build()
38	.create(TmdbApiService::class.java)
39	}
40	}

Tabel 43. Source Code Soal 1 Modul 5

File TmdbApiService.xml	
1	package com.example.movielist.data.remote.api
2	
3	import
	com.example.movielist.data.remote.models.MovieListResponse
4	import retrofit2.Response
5	import retrofit2.http.GET
6	import retrofit2.http.Query
7	
8	interface TmdbApiService {
9	
10	@GET("movie/popular")
11	suspend fun getPopularMovies(
12	@Query("api_key") apiKey: String,
13	@Query("language") language: String = "en-US",
14	@Query("page") page: Int = 1
15	): Response<MovieListResponse>
16	}

Tabel 44. Source Code Soal 1 Modul 5

File MovieDto.kt

1	package com.example.movielist.data.remote.models
2	
3	import kotlinx.serialization.SerialName
4	import kotlinx.serialization.Serializable
5	
6	@Serializable
7	data class MovieDto(8 val adult: Boolean, 9 @SerialName("backdrop_path") 10 val backdropPath: String?, 11 @SerialName("genre_ids") 12 val genreIds: List<Int>, 13 val id: Int, 14 @SerialName("original_language") 15 val originalLanguage: String, 16 @SerialName("original_title") 17 val originalTitle: String, 18 val overview: String, 19 val popularity: Double, 20 @SerialName("poster_path") 21 val posterPath: String?, 22 @SerialName("release_date") 23 val releaseDate: String, 24 val title: String, 25 val video: Boolean, 26 @SerialName("vote_average") 27 val voteAverage: Double, 28 @SerialName("vote_count") 29 val voteCount: Int 30)

Tabel 45. Source Code Soal 1 Modul 5

File MovieDtoExtension.kt	
1	package com.example.movielist.data.remote.models
2	
3	import com.example.movielist.domain.model.Movie
4	import
	com.example.movielist.data.local.entities.MovieEntity
5	
6	fun MovieDto.toDomainMovie(): Movie {
7	return Movie(
8	id = id,
9	title = title,
10	overview = overview,
11	posterPath = posterPath,
12	releaseDate = releaseDate,
13	voteAverage = voteAverage
14	)
15	}
16	
17	fun MovieDto.toMovieEntity(): MovieEntity {
18	return MovieEntity(
19	id = id,
20	title = title,
21	overview = overview,
22	posterPath = posterPath,
23	releaseDate = releaseDate,
24	voteAverage = voteAverage,
25	popularity = popularity
26	)
27	}

Tabel 46. Source Code Soal 1 Modul 5

File MovieListResponse.kt	
1	package com.example.movieslist.data.remote.models
2	
3	import kotlinx.serialization.SerialName
4	import kotlinx.serialization.Serializable
5	
6	@Serializable
7	data class MovieListResponse(
8	val page: Int,
9	val results: List<MovieDto>,
10	@SerialName("total_pages")
11	val totalPages: Int,
12	@SerialName("total_results")
13	val totalResults: Int
14	)

Tabel 47. Source Code Soal 1 Modul 5

File MovieRepository.kt	
1	package com.example.movieslist.data.repository
2	
3	import com.example.movieslist.data.local.dao.MovieDao
4	import
	com.example.movieslist.data.remote.api.TmdbApiService
5	import
	com.example.movieslist.data.remote.models.toDomainMovie
	e
6	import
	com.example.movieslist.data.remote.models.toMovieEntity
	y

```

7  import com.example.movielist.domain.model.Movie
8  import com.example.movielist.utils.Result
9  import kotlinx.coroutines.flow.Flow
10 import kotlinx.coroutines.flow.flow
11 import retrofit2.HttpException
12 import java.io.IOException
13
14 interface MovieRepository {
15     fun getPopularMovies(): Flow<Result<List<Movie>>>
16 }
17
18 class MovieRepositoryImpl(
19     private val apiService: TmdbApiService,
20     private val movieDao: MovieDao,
21     private val apiKey: String
22 ) : MovieRepository {
23
24     override fun getPopularMovies():
25         Flow<Result<List<Movie>>> = flow {
26         emit(Result.Loading)
27
28         val cachedMovies =
29             movieDao.getAllMovies().map { it.toDomainMovie() }
30         if (cachedMovies.isNotEmpty()) {
31             emit(Result.Success(cachedMovies))
32         }
33
34         try {
35             val response =
36                 apiService.getPopularMovies(apiKey = apiKey)

```

34	if (response.isSuccessful) {
35	val movieDtos =
	response.body()?.results ?: emptyList()
36	val domainMovies = movieDtos.map {
	it.toDomainMovie() }
37	
38	movieDao.clearAllMovies()
39	movieDao.insertAllMovies(movieDtos.map
	{ it.toMovieEntity() })
40	
41	emit(Result.Success(domainMovies))
42	} else {
43	emit(Result.Error(Exception("API
	Error: \${response.code()} \${response.message()}"))
44	}
45	} catch (e: HttpException) {
46	emit(Result.Error(Exception("Network
	Error (HTTP \${e.code()}): \${e.message()}"))
47	} catch (e: IOException) {
48	emit(Result.Error(Exception("No Internet
	Connection or API Timeout: \${e.message()}"))
49	} catch (e: Exception) {
50	emit(Result.Error(Exception("An
	unexpected error occurred: \${e.localizedMessage()}"))
51	}
	}
52	}
53	

Tabel 48. Source Code Soal 1 Modul 5

File Movie.kt	
1	package com.example.movielist.domain.model
2	
3	import android.os.Parcelable
4	import kotlinx.parcelize.Parcelize
5	
6	@Parcelize
7	data class Movie(
8	val id: Int,
9	val title: String,
10	val overview: String,
11	val posterPath: String?,
12	val releaseDate: String,
13	val voteAverage: Double
14	) : Parcelable

Tabel 49. Source Code Soal 1 Modul 5

File GetPopularMoviesUseCase.kt	
1	package com.example.movielist.domain.usecase
2	
3	import com.example.movielist.domain.model.Movie
4	import
	com.example.movielist.data.repository.MovieRepositoryI
	mpl
5	import com.example.movielist.utils.Result
6	import kotlinx.coroutines.flow.Flow
7	
8	class GetPopularMoviesUseCase (
9	private val movieRepository: MovieRepositoryImpl

10	) {
11	operator fun invoke(): Flow<Result<List<Movie>>> {
12	return movieRepository.getPopularMovies()
13	}
14	}

Tabel 50. Source Code Soal 1 Modul 5

File DetailActivity.kt	
1	package com.example.movielist.presentation.ui.activity
2	
3	import android.os.Build
4	import android.os.Bundle
5	import android.view.MenuItem
6	import android.widget.Toast
7	import androidx.appcompat.app.AppCompatActivity
8	import com.bumptech.glide.Glide
9	import
	com.example.movielist.databinding.ActivityDetailBindin
	g
10	import com.example.movielist.domain.model.Movie
11	
12	class DetailActivity : AppCompatActivity() {
13	
14	private lateinit var binding:
	ActivityDetailBinding
15	
16	companion object {
17	const val EXTRA_MOVIE = "extra_movie"
18	}
19	

```

20     override fun onCreate(savedInstanceState: Bundle?)
21     {
22         super.onCreate(savedInstanceState)
23         binding =
24             ActivityDetailBinding.inflate(layoutInflater)
25         setContentView(binding.root)
26
27         supportActionBar?.setDisplayHomeAsUpEnabled(true)
28
29         val movie = if (Build.VERSION.SDK_INT >=
30             Build.VERSION_CODES.TIRAMISU) {
31             intent.getParcelableExtra(EXTRA_MOVIE,
32                 Movie::class.java)
33         } else {
34             @Suppress("DEPRECATION")
35             intent.getParcelableExtra(EXTRA_MOVIE)
36         }
37
38         movie?.let {
39
40             supportActionBar?.title = it.title
41
42             binding.apply {
43                 tvDetailTitle.text = it.title
44                 tvDetailReleaseDate.text = "Release
45                 Date: {it.releaseDate}"
46                 tvDetailVoteAverage.text = "Rating:
47                 {String.format("%.1f", it.voteAverage)}"
48                 tvDetailOverview.text = it.overview
49

```

44	val imageUrl =
	"https://image.tmdb.org/t/p/w500\${it.posterPath}"
45	Glide.with(this@DetailActivity)
46	.load(imageUrl)
47	.centerCrop()
48	.into(ivDetailPoster)
49	}
50	} ?: run {
51	Toast.makeText(this, "Film tidak
	ditemukan.", Toast.LENGTH_SHORT).show()
52	finish()
53	}
54	}
55	
56	override fun onOptionsItemSelected(item:
	MenuItem): Boolean {
57	if (item.itemId == android.R.id.home) {
58	onBackPressedDispatcher.onBackPressed()
59	return true
60	}
61	return super.onOptionsItemSelected(item)
62	}
63	}

Tabel 51. Source Code Soal 1 Modul 5

File MovieAdapter.kt	
1	package com.example.movielist.presentation.ui.adapter
2	
3	import android.view.LayoutInflater
4	import android.view.ViewGroup

```

5 import androidx.recyclerview.widget.DiffUtil
6 import androidx.recyclerview.widget.ListAdapter
7 import androidx.recyclerview.widget.RecyclerView
8 import com.bumptech.glide.Glide
9 import
    com.example.movielist.databinding.ItemMovieBinding
10 import com.example.movielist.domain.model.Movie
11
12 class MovieAdapter : ListAdapter<Movie,
    MovieAdapter.MovieViewHolder>(MovieDiffCallback()) {
13
14     var onItemClick: ((Movie) -> Unit)? = null
15
16     override fun onCreateViewHolder(parent: ViewGroup,
    viewType: Int): MovieViewHolder {
17         val binding =
    ItemMovieBinding.inflate(LayoutInflater.from(parent.co
    ntext), parent, false)
18         return MovieViewHolder(binding)
19     }
20
21     override fun onBindViewHolder(holder:
    MovieViewHolder, position: Int) {
22         val movie = getItem(position)
23         holder.bind(movie)
24     }
25
26     inner class MovieViewHolder(private val binding:
    ItemMovieBinding) :
27         RecyclerView.ViewHolder(binding.root) {

```

```

28
29         init {
30             binding.btnDetail.setOnClickListener {
31                 onItemClick?.invoke(getItem(adapterPosition))
32             }
33         }
34
35         fun bind(movie: Movie) {
36             binding.apply {
37                 tvMovieTitle.text = movie.title
38                 tvReleaseDate.text = "Release Date:
39                 ${movie.releaseDate}"
40                 tvVoteAverage.text = "Rating:
41                 ${String.format("%.1f", movie.voteAverage)}"
42                 tvOverview.text = movie.overview
43
44                 val imageUrl =
45                 "https://image.tmdb.org/t/p/w500${movie.posterPath}"
46
47                 Glide.with(itemView.context)
48                     .load(imageUrl)
49                     .centerCrop()
50                     .into(ivPoster)
51             }
52         }
53
54         class MovieDiffCallback :
55         DiffUtil.ItemCallback<Movie>() {
56             override fun areItemsTheSame(oldItem: Movie,

```

	newItem: Movie): Boolean {
54	return oldItem.id == newItem.id
55	}
56	
57	override fun areContentsTheSame(oldItem:
	Movie, newItem: Movie): Boolean {
58	return oldItem == newItem
59	}
60	}
61	}

Tabel 52. Source Code Soal 1 Modul 5

File MovieViewModel.kt	
1	package com.example.movielist.presentation.viewmodel
2	
3	import androidx.lifecycle.LiveData
4	import androidx.lifecycle.MutableLiveData
5	import androidx.lifecycle.ViewModel
6	import androidx.lifecycle.viewModelScope
7	import com.example.movielist.domain.model.Movie
8	import
	com.example.movielist.domain.usecase.GetPopularMoviesU
	seCase
9	import com.example.movielist.utils.Result
10	import kotlinx.coroutines.launch
11	
12	class MovieViewModel (
13	private val getPopularMoviesUseCase:
	GetPopularMoviesUseCase
14	) : ViewModel() {

15	
16	private val _popularMovies =
	MutableLiveData<Result<List<Movie>>>()>>()
17	val popularMovies: LiveData<Result<List<Movie>>> =
	_popularMovies
18	
19	init {
20	fetchPopularMovies()
21	}
22	
23	fun fetchPopularMovies() {
24	viewModelScope.launch {
25	getPopularMoviesUseCase().collect { result
	->
26	_popularMovies.value = result
27	}
28	}
29	}
30	}

Tabel 53. Source Code Jawaban Soal 1 Modul 5

File ViewModelFactory.kt	
1	package com.example.movielist.presentation.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	import
	com.example.movielist.domain.usecase.GetPopularMoviesU
	seCase
6	

7	class ViewModelFactory(
8	private val getPopularMoviesUseCase:
	GetPopularMoviesUseCase
9	) : ViewModelProvider.Factory {
10	
11	override fun <T : ViewModel> create(modelClass:
	Class<T>): T {
12	if
	(modelClass.isAssignableFrom(MovieViewModel::class.java)) {
13	@Suppress("UNCHECKED_CAST")
14	return
	MovieViewModel(getPopularMoviesUseCase) as T
15	}
16	throw IllegalArgumentException("Unknown
	ViewModel class")
17	}
18	}

Tabel 54. Source Code Soal 1 Modul 5

File Result.kt	
1	package com.example.movielist.utils
2	
3	sealed class Result<out T> {
4	object Loading : Result<Nothing>()
5	data class Success<out T>(val data: T) :
	Result<T>()
6	data class Error(val exception: Exception) :
	Result<Nothing>()
7	}

Tabel 55. Source Code Soal 1 Modul 5

File Activity_detail.xml	
1	<?xml version="1.0" encoding="utf-8"?>
2	<ScrollView
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	tools:context=".presentation.ui.activity.DetailActivity">
8	
9	<androidx.constraintlayout.widget.ConstraintLayout
10	android:layout_width="match_parent"
11	android:layout_height="wrap_content"
12	android:padding="16dp">
13	
14	<ImageView
15	android:id="@+id/iv_detail_poster"
16	android:layout_width="0dp"
17	android:layout_height="300dp"
18	android:scaleType="centerCrop"
19	app:layout_constraintEnd_toEndOf="parent"
20	app:layout_constraintStart_toStartOf="parent"
21	app:layout_constraintTop_toTopOf="parent"
22	tools:src="@tools:sample/avatars" />
23	

24	<TextView
25	android:id="@+id/tv_detail_title"
26	android:layout_width="0dp"
27	android:layout_height="wrap_content"
28	android:layout_marginTop="16dp"
29	android:textSize="24sp"
30	android:textStyle="bold"
31	app:layout_constraintEnd_toEndOf="parent"
32	app:layout_constraintStart_toStartOf="parent"
33	app:layout_constraintTop_toBottomOf="@id/iv_detail_poster"
34	tools:text="Movie Title on Detail Page" />
35	
36	<TextView
37	android:id="@+id/tv_detail_release_date"
38	android:layout_width="0dp"
39	android:layout_height="wrap_content"
40	android:layout_marginTop="8dp"
41	android:textSize="16sp"
42	app:layout_constraintEnd_toEndOf="parent"
43	app:layout_constraintStart_toStartOf="parent"
44	app:layout_constraintTop_toBottomOf="@id/tv_detail_title"
45	tools:text="Release Date: 2023-01-01" />
46	
47	<TextView
48	android:id="@+id/tv_detail_vote_average"
49	android:layout_width="0dp"
50	android:layout_height="wrap_content"
51	android:layout_marginTop="4dp"

52	android:textSize="16sp"
53	app:layout_constraintEnd_toEndOf="parent"
54	app:layout_constraintStart_toStartOf="parent"
55	app:layout_constraintTop_toBottomOf="@id/tv_detail_release_date"
56	tools:text="Rating: 7.5" />
57	
58	<TextView
59	android:id="@+id/tv_detail_overview_label"
60	android:layout_width="0dp"
61	android:layout_height="wrap_content"
62	android:layout_marginTop="16dp"
63	android:text="Overview:"
64	android:textSize="18sp"
65	android:textStyle="bold"
66	app:layout_constraintEnd_toEndOf="parent"
67	app:layout_constraintStart_toStartOf="parent"
68	app:layout_constraintTop_toBottomOf="@id/tv_detail_vote_average" />
69	
70	<TextView
71	android:id="@+id/tv_detail_overview"
72	android:layout_width="0dp"
73	android:layout_height="wrap_content"
74	android:layout_marginTop="8dp"
75	android:textSize="16sp"
76	app:layout_constraintEnd_toEndOf="parent"
77	app:layout_constraintStart_toStartOf="parent"
78	app:layout_constraintTop_toBottomOf="@id/tv_detail_overview_label"

79	tools:text="This is a very long and detailed overview of the movie. It covers the plot, characters, themes, and critical reception." />
80	
81	</androidx.constraintlayout.widget.ConstraintLayout>
82	</ScrollView>

Tabel 56. Source Code Soal 1 Modul 5

File Activity_main.xml	
1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/andr oid"
3	xmlns:app="http://schemas.android.com/apk/res- auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	tools:context=".presentation.ui.activity.MainActivity"
8	>
9	<TextView
10	android:id="@+id/tv_title"
11	android:layout_width="wrap_content"
12	android:layout_height="wrap_content"
13	android:layout_marginTop="8dp"
14	android:text="Popular Movies"
15	android:textSize="24sp"
16	android:textStyle="bold"
17	app:layout_constraintHorizontal_bias="0.454"

18	app:layout_constraintStart_toStartOf="parent"
19	app:layout_constraintTop_toTopOf="parent" />
20	
21	<ProgressBar
22	android:id="@+id/progress_bar"
23	android:layout_width="wrap_content"
24	android:layout_height="wrap_content"
25	android:visibility="gone"
26	app:layout_constraintTop_toBottomOf="@id/tv_title"
27	app:layout_constraintStart_toStartOf="parent"
28	app:layout_constraintEnd_toEndOf="parent"
29	app:layout_constraintBottom_toBottomOf="parent"
	/>
30	
31	<TextView
32	android:id="@+id/tv_error"
33	android:layout_width="wrap_content"
34	android:layout_height="wrap_content"
35	android:text="Error: Could not load movies."
36	android:textColor="@android:color/holo_red_dark"
37	android:visibility="gone"
38	android:gravity="center"
39	app:layout_constraintTop_toBottomOf="@id/tv_title"
40	app:layout_constraintStart_toStartOf="parent"
41	app:layout_constraintEnd_toEndOf="parent"
42	app:layout_constraintBottom_toBottomOf="parent"
	/>
43	
44	<Button
45	android:id="@+id/btn_retry"

```

46         android:layout_width="wrap_content"
47         android:layout_height="wrap_content"
48         android:text="Retry"
49         android:visibility="gone"
50         android:layout_marginTop="8dp"
51     app:layout_constraintTop_toBottomOf="@id/tv_error"
52     app:layout_constraintStart_toStartOf="parent"
53     app:layout_constraintEnd_toEndOf="parent"
54     app:layout_constraintBottom_toBottomOf="parent"
55 />
56     <androidx.recyclerview.widget.RecyclerView
57         android:id="@+id/rv_movies"
58         android:layout_width="0dp"
59         android:layout_height="0dp"
60         android:layout_marginTop="16dp"
61         app:layout_constraintBottom_toBottomOf="parent"
62         app:layout_constraintEnd_toEndOf="parent"
63         app:layout_constraintStart_toStartOf="parent"
64         app:layout_constraintTop_toBottomOf="@+id/tv_title"
65         tools:listitem="@layout/item_movie" />
66
67     <com.google.android.material.switchmaterial.SwitchMate
68         android:id="@+id/switch_dark_mode"
69         android:layout_width="wrap_content"
70         android:layout_height="wrap_content"
71         android:layout_marginEnd="8dp"
72         android:layout_marginBottom="8dp"
73         android:text="Dark Mode"

```

74	app:layout_constraintEnd_toEndOf="parent"
75	app:layout_constraintTop_toTopOf="parent" />
76	
77	</androidx.constraintlayout.widget.ConstraintLayout>

Tabel 57. Source Code Soal 1 Modul 5

File item_movie.xml	
1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.cardview.widget.CardView
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:layout_width="match_parent"
6	android:layout_height="wrap_content"
7	android:layout_margin="8dp"
8	app:cardCornerRadius="8dp"
9	app:cardElevation="4dp">
10	
11	<androidx.constraintlayout.widget.ConstraintLayout
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"
14	android:padding="16dp">
15	
16	<ImageView
17	android:id="@+id/iv_poster"
18	android:layout_width="550dp"
19	android:layout_height="150dp"
20	android:scaleType="centerCrop"

21	app:layout_constraintStart_toStartOf="parent"
22	app:layout_constraintTop_toTopOf="parent"
23	tools:src="@tools:sample/avatars" />
24	
25	<TextView
26	android:id="@+id/tv_movie_title"
27	android:layout_width="0dp"
28	android:layout_height="wrap_content"
29	android:layout_marginStart="16dp"
30	android:textStyle="bold"
31	android:textSize="18sp"
32	app:layout_constraintEnd_toEndOf="parent"
33	app:layout_constraintStart_toEndOf="@id/iv_poster"
34	app:layout_constraintTop_toTopOf="@id/iv_poster"
35	tools:text="Movie Title" />
36	
37	<TextView
38	android:id="@+id/tv_release_date"
39	android:layout_width="0dp"
40	android:layout_height="wrap_content"
41	android:layout_marginStart="16dp"
42	android:layout_marginTop="4dp"
43	android:textSize="14sp"
44	app:layout_constraintEnd_toEndOf="parent"
45	app:layout_constraintStart_toEndOf="@id/iv_poster"
46	app:layout_constraintTop_toBottomOf="@id/tv_movie_titl
	e"
47	tools:text="Release Date: 2023-01-01" />
48	
49	<TextView

50	android:id="@+id/tv_vote_average"
51	android:layout_width="0dp"
52	android:layout_height="wrap_content"
53	android:layout_marginStart="16dp"
54	android:layout_marginTop="4dp"
55	android:textSize="14sp"
56	app:layout_constraintEnd_toEndOf="parent"
57	app:layout_constraintStart_toEndOf="@id/iv_poster"
58	app:layout_constraintTop_toBottomOf="@id/tv_release_date"
59	tools:text="Rating: 7.5" />
60	
61	<TextView
62	android:id="@+id/tv_overview"
63	android:layout_width="0dp"
64	android:layout_height="wrap_content"
65	android:layout_marginTop="8dp"
66	android:maxLines="3"
67	android:ellipsize="end"
68	app:layout_constraintEnd_toEndOf="parent"
69	app:layout_constraintStart_toStartOf="parent"
70	app:layout_constraintTop_toBottomOf="@id/iv_poster"
71	tools:text="This is a short overview of
	the movie. It talks about the plot and characters..."
	/>
72	
73	<Button
74	android:id="@+id/btn_detail"
75	android:layout_width="wrap_content"
76	android:layout_height="wrap_content"

77	android:layout_marginTop="16dp"
78	android:text="Lihat Detail"
79	app:layout_constraintEnd_toEndOf="parent"
80	app:layout_constraintTop_toBottomOf="@id/tv_overview"
	/>
81	
82	</androidx.constraintlayout.widget.ConstraintLayout>
83	</androidx.cardview.widget.CardView>

Tabel 58. Source Code Soal 1 Modul 5

File build.gradle.kts (Module:app)	
1	plugins {
2	alias(libs.plugins.android.application)
3	alias(libs.plugins.kotlin.android)
4	id("kotlin-parcelize")
5	id("kotlin-kapt")
6	kotlin("plugin.serialization") version "2.0.21"
7	}
8	
9	android {
10	namespace = "com.example.movielist"
11	compileSdk = 35
12	
13	defaultConfig {
14	applicationId = "com.example.movielist"
15	minSdk = 24
16	targetSdk = 35
17	versionCode = 1
18	versionName = "1.0"
19	

```

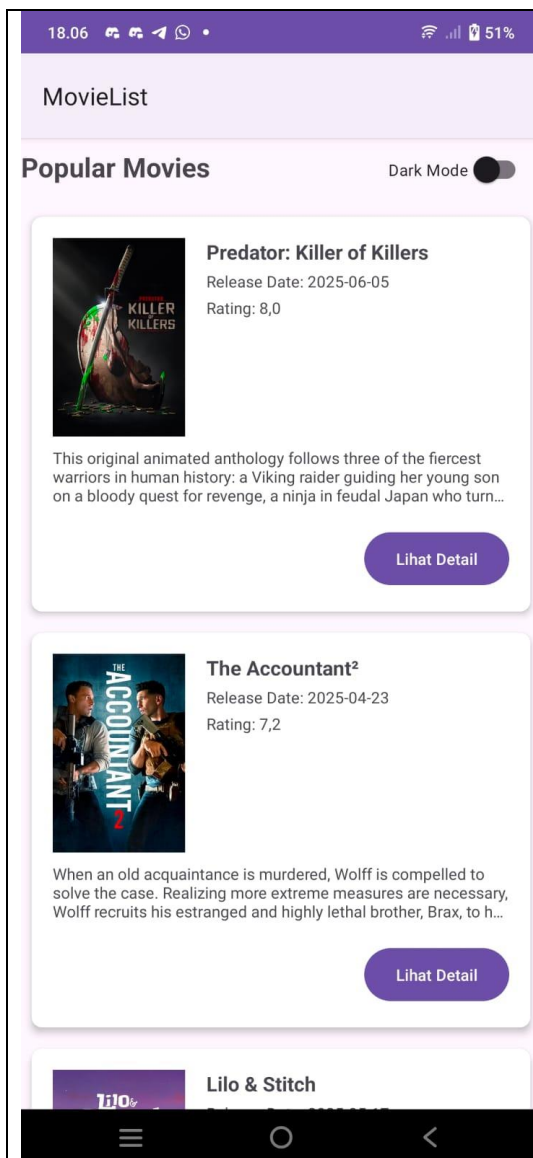
20         testInstrumentationRunner =
21         "androidx.test.runner.AndroidJUnitRunner"
22
23     buildTypes {
24         release {
25             isMinifyEnabled = false
26             proguardFiles(
27                 getDefaultProguardFile("proguard-
28 android-optimize.txt"),
29                 "proguard-rules.pro"
30             )
31         }
32     compileOptions {
33         sourceCompatibility = JavaVersion.VERSION_11
34         targetCompatibility = JavaVersion.VERSION_11
35     }
36     kotlinOptions {
37         jvmTarget = "11"
38     }
39     buildFeatures {
40         viewBinding = true
41     }
42 }
43
44 dependencies {
45
46     implementation(libs.androidx.core.ktx)
47     implementation(libs.androidx.appcompat)

```

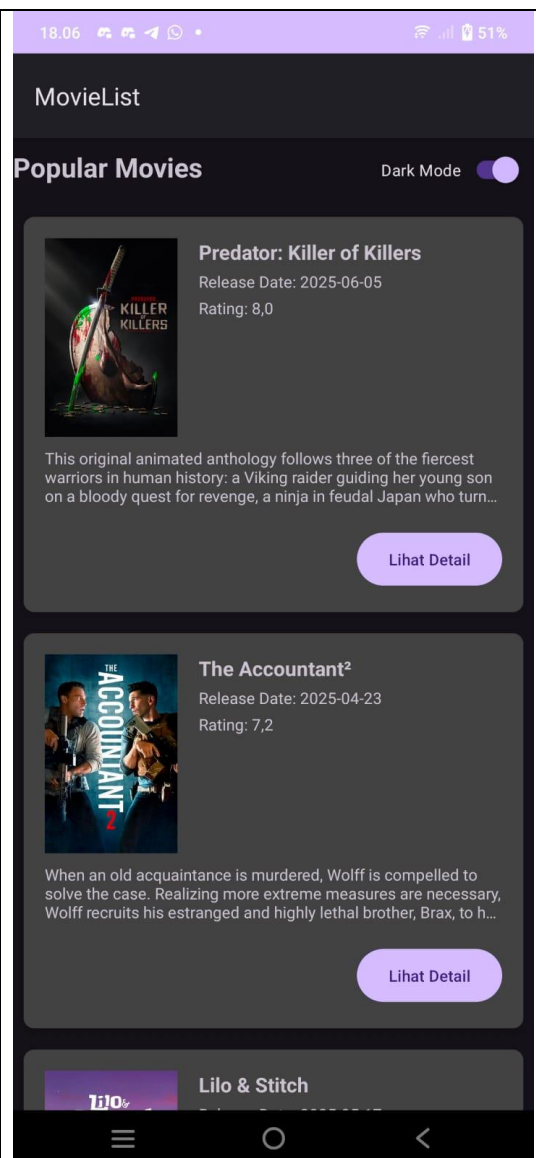
48	<i>implementation(libs.material)</i>
49	<i>implementation(libs.androidx.activity)</i>
50	<i>implementation(libs.androidx.constraintlayout)</i>
51	<i>testImplementation(libs.junit)</i>
52	<i>androidTestImplementation(libs.androidx.junit)</i>
53	<i>androidTestImplementation(libs.androidx.espresso.core)</i>
54	
55	<i>// Room</i>
56	<i>implementation("androidx.room:room-runtime:2.6.1")</i>
57	<i>implementation("androidx.room:room-ktx:2.6.1")</i>
58	<i>kapt("androidx.room:room-compiler:2.6.1")</i>
59	
60	<i>// Glide</i>
61	<i>implementation</i>
	<i>("com.github.bumptech.glide:glide:4.16.0")</i>
62	
63	<i>// Retrofit & Kotlinx Serialization</i>
64	<i>implementation("com.squareup.retrofit2:retrofit:2.9.0"</i>
	<i>)</i>
65	<i>implementation("com.jakewharton.retrofit:retrofit2-</i>
	<i>kotlinx-serialization-converter:0.8.0")</i>
66	<i>implementation("org.jetbrains.kotlinx:kotlinx-</i>
	<i>serialization-json:1.6.3")</i>
67	<i>implementation("com.squareup.okhttp3:okhttp:4.9.3")</i>
68	<i>implementation("com.squareup.okhttp3:logging-</i>
	<i>interceptor:4.9.3")</i>
69	<i>implementation("com.jakewharton.retrofit:retrofit2-</i>
	<i>kotlinx-serialization-converter:1.0.0")</i>
70	
71	<i>// Lifecycle</i>

72	<code>implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.7.0")</code>
73	<code>implementation("androidx.lifecycle:lifecycle-livedata-ktx:2.7.0")</code>
74	
75	<code>// Coroutine</code>
76	<code>implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3")</code>
77	
78	<code>// Navigation</code>
79	<code>val nav_version = "2.8.9"</code>
80	<code>implementation("androidx.navigation:navigation-fragment:\$nav_version")</code>
81	<code>implementation("androidx.navigation:navigation-ui:\$nav_version")</code>
82	<code>}</code>

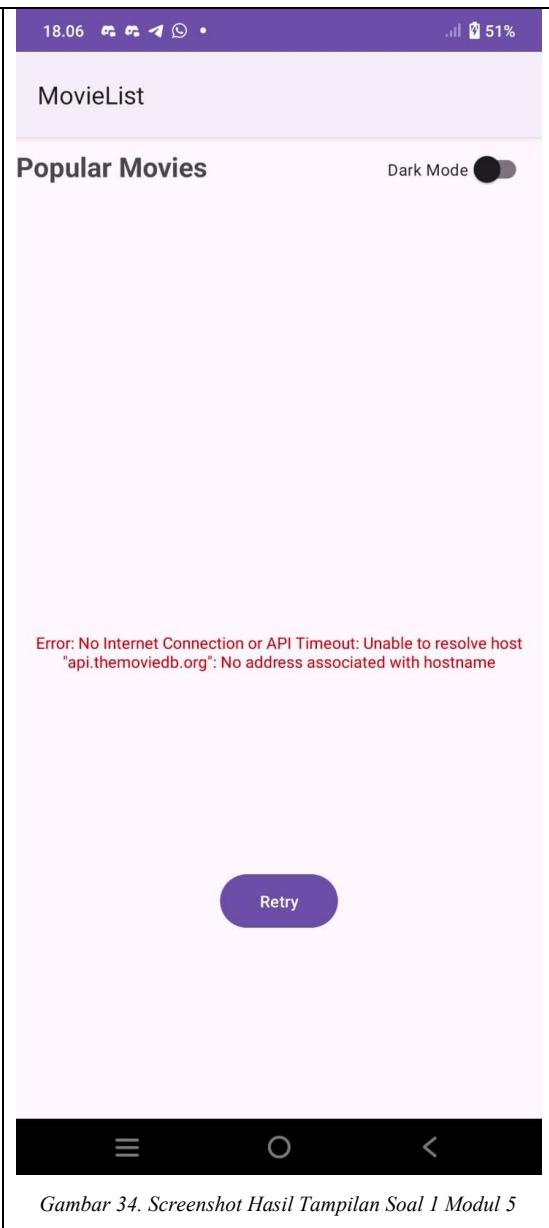
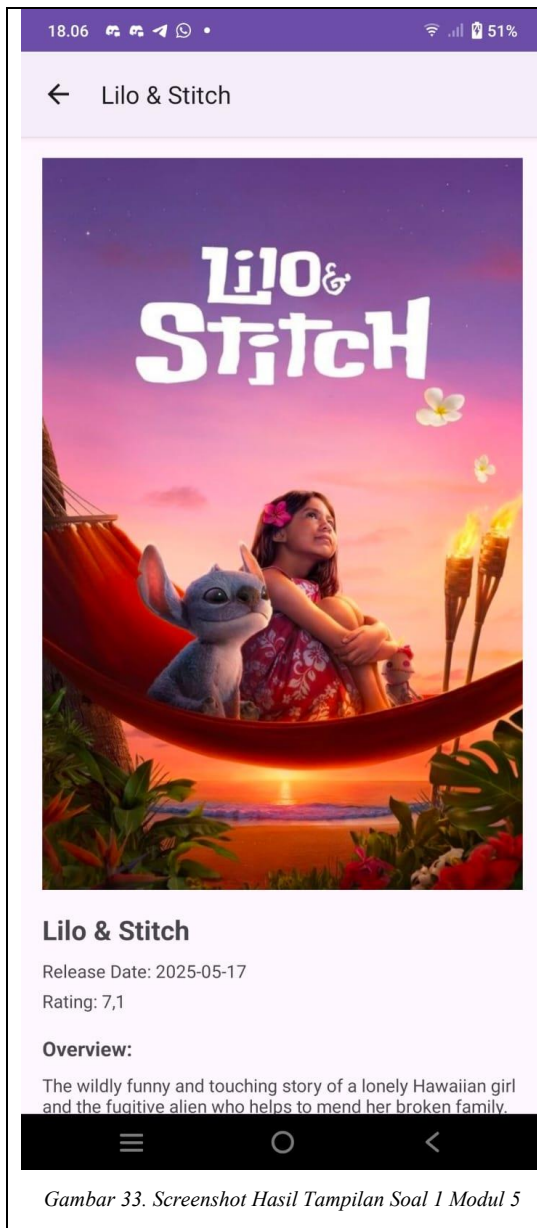
B. Output Program



Gambar 31. Screenshot Hasil Tampilan Soal 1 Modul 5



Gambar 32. Screenshot Hasil Tampilan Soal 1 Modul 5



C. Pembahasan

File AndroidManifest.xml

File **AndroidManifest.xml** di atas adalah komponen penting dalam aplikasi Android karena mendeklarasikan izin, aktivitas, dan konfigurasi penting aplikasi kepada sistem Android. Pada bagian atas, terdapat deklarasi XML standar dengan encoding UTF-8, serta dua namespace: **xmlns:android** untuk atribut

Android standar, dan **xmlns:tools** untuk konfigurasi yang digunakan oleh Android tools seperti Android Studio.

Bagian pertama dari isi manifest adalah deklarasi izin aplikasi dengan **<uses-permission android:name="android.permission.INTERNET" />**, yang menyatakan bahwa aplikasi memerlukan akses ke internet. Izin ini sangat penting, terutama jika aplikasi menggunakan API eksternal, seperti The Movie Database (TMDB), untuk mengambil data secara online.

Selanjutnya adalah elemen **<application>** yang menyimpan konfigurasi inti dari aplikasi, seperti ikon (**android:icon** dan **android:roundIcon**), label aplikasi (**android:label**), tema (**android:theme**), serta pengaturan pencadangan seperti **android:allowBackup**, **android:dataExtractionRules**, dan **android:fullBackupContent**, yang berkaitan dengan bagaimana data aplikasi dicadangkan dan dipulihkan di perangkat Android yang mendukungnya. Atribut **tools:targetApi="31"** menunjukkan bahwa alat pengembangan (tools) seperti lint akan menargetkan API level 31 untuk pemeriksaan dan validasi.

Di dalam elemen **<application>** terdapat dua **<activity>** yang dideklarasikan. Aktivitas utama (**MainActivity**) didefinisikan dengan **android:exported="true"** yang artinya aktivitas ini dapat diakses dari luar aplikasi, dan di dalamnya terdapat **<intent-filter>** dengan aksi **MAIN** dan kategori **LAUNCHER**, yang menjadikan **MainActivity** sebagai titik masuk utama aplikasi ketika ikon aplikasi diklik di layar utama perangkat. Aktivitas kedua adalah **DetailActivity**, yang dideklarasikan dengan **android:exported="false"**, artinya hanya bisa diakses secara internal dari dalam aplikasi, bukan dari aplikasi atau komponen eksternal lain. Ini sesuai dengan praktik keamanan Android yang membatasi akses aktivitas sensitif atau tidak diperlukan dari luar aplikasi.

File MainActivity.kt

Kelas **MainActivity** pada kode di atas merupakan *entry point* utama dalam aplikasi Android "MovieList" yang menggunakan arsitektur modern dengan *MVVM* (*Model-View-ViewModel*), *ViewBinding*, dan berbagai komponen arsitektur Android seperti Room, Retrofit, dan LiveData. Kelas ini berperan sebagai lapisan *presentation* yang menghubungkan tampilan antarmuka pengguna dengan logika aplikasi.

Pada awal kelas, terdapat beberapa deklarasi penting. Objek **binding** digunakan untuk mengakses elemen UI melalui ViewBinding tanpa menggunakan **findViewById**, sehingga meningkatkan keamanan dan efisiensi. **movieAdapter** adalah adapter untuk **RecyclerView** yang digunakan menampilkan daftar film, dan **movieAppPreferences** adalah wrapper untuk **SharedPreferences** yang digunakan menyimpan preferensi pengguna seperti dark mode dan API key.

ViewModel **movieViewModel** dibuat menggunakan delegasi by **viewModels** dan menyertakan custom **ViewModelFactory**. Dalam blok inisialisasi ViewModel tersebut, dilakukan beberapa hal: menginisialisasi Retrofit untuk API TMDb, membuat instansi Room untuk database lokal, menyimpan API key ke dalam **SharedPreferences**, dan membuat repository untuk menyatukan data dari API dan database.

Di dalam **onCreate()**, **binding** diinisialisasi dan root view diset ke konten tampilan. **movieAppPreferences** kemudian diinisialisasi dengan konteks saat ini. Fungsi **setupRecyclerView()** dipanggil untuk mengatur tampilan daftar film menggunakan **LinearLayoutManager** dan **MovieAdapter**, serta menyediakan aksi klik pada item untuk membuka **DetailActivity** dengan intent dan data film terpilih. Fungsi **observeViewModel()** mengamati perubahan LiveData **popularMovies** dari ViewModel dan menyesuaikan tampilan UI sesuai status **Result** (*Loading*, *Success*, *Error*). Saat berhasil, data film

ditampilkan di RecyclerView; jika gagal, pesan error ditampilkan bersama tombol `retry`.

Fitur dark mode diatur melalui `setupDarkModeToggle()`, yang membaca preferensi sebelumnya dan mengatur status **Switch** serta menerapkan tema sesuai keadaan. Listener `setOnCheckedChangeListener` disiapkan untuk menyimpan preferensi baru saat switch diklik dan menerapkan tema yang sesuai menggunakan `AppCompatActivity`.

File MovieDao.kt

File `MovieDao.kt` merupakan komponen penting dalam arsitektur lokal aplikasi Android yang menggunakan Room Database. DAO (*Data Access Object*) ini berfungsi sebagai antarmuka untuk mengakses dan memanipulasi data film yang tersimpan secara lokal di tabel **movies**. Antarmuka ini ditandai dengan anotasi `@Dao`, yang memberi tahu Room bahwa ini adalah komponen yang menyediakan perintah SQL untuk entitas **MovieEntity**. Terdapat tiga fungsi utama di dalamnya. Pertama, fungsi `insertAllMovies` memungkinkan penyisipan daftar film ke dalam database dengan strategi konflik **REPLACE**, artinya data lama akan digantikan jika terjadi duplikasi berdasarkan primary key—fitur ini berguna untuk memastikan data film yang ditampilkan selalu yang terbaru saat diambil dari API. Kedua, fungsi `getAllMovies` digunakan untuk mengambil seluruh data film dari tabel, yang kemudian diurutkan berdasarkan tingkat popularitas secara menurun. Fungsi ini bersifat **suspend**, yang berarti dijalankan secara asynchronous menggunakan coroutine, sehingga tidak membebani thread utama. Ketiga, fungsi `clearAllMovies` menghapus seluruh data dari tabel **movies**, yang biasanya digunakan saat penyegaran data atau pembersihan lokal cache.

File AppDatabase.kt

File `AppDatabase.kt` adalah komponen utama yang mendefinisikan *Room Database* dalam aplikasi Android "MovieList". Kelas ini ditandai dengan anotasi

@Database, yang memberi tahu sistem bahwa ini adalah kelas database dan mengatur konfigurasi penting seperti entitas yang digunakan (**MovieEntity**), versi database (**version = 1**), dan pengaturan **exportSchema = false** untuk mencegah ekspor skema database ke file JSON, yang biasanya dilakukan untuk tujuan dokumentasi atau pengujian. **MovieEntity** sendiri merupakan representasi tabel **movies** dalam database lokal.

Kelas **AppDatabase** diturunkan dari **RoomDatabase**, dan bersifat abstrak karena berfungsi sebagai kerangka kerja yang akan digunakan Room untuk menghasilkan implementasi konkret di balik layar. Di dalamnya terdapat satu fungsi abstrak **movieDao()**, yang digunakan untuk mengakses DAO (*Data Access Object*) **MovieDao**. Fungsi ini menjadi pintu utama untuk menjalankan operasi database seperti menyisipkan, mengambil, dan menghapus data film.

Selain itu, terdapat **companion object** yang mendeklarasikan konstanta **DATABASE_NAME** dengan nilai **"tmdb_app_db"**, yang digunakan sebagai nama file database yang disimpan di perangkat pengguna. Penamaan ini penting untuk identifikasi dan pengelolaan database, terutama saat terjadi pembaruan versi atau migrasi skema.

File **MovieEntity.kt**

File **MovieEntity.kt** mendefinisikan *entitas* Room untuk menyimpan data film di database lokal aplikasi "MovieList". Kelas data **MovieEntity** ditandai dengan anotasi **@Entity**, yang menunjukkan bahwa kelas ini merepresentasikan sebuah tabel bernama **"movies"** di database Room. Setiap properti dalam kelas ini menjadi kolom pada tabel tersebut, di mana **id** berfungsi sebagai **@PrimaryKey**, artinya merupakan identifikasi unik bagi setiap entri film. Properti lainnya seperti **title**, **overview**, **posterPath**, **releaseDate**, **voteAverage**, dan **popularity** berisi informasi detail mengenai film yang disimpan.

Selain menyimpan data, **MovieEntity** juga menyediakan dua fungsi penting untuk konversi data antar lapisan: fungsi **toDomainMovie()** dan

fromDomainMovie(...). Fungsi **toDomainMovie()** digunakan untuk mengubah **MovieEntity** menjadi objek **Movie** dari lapisan domain, yaitu representasi logika bisnis yang digunakan ViewModel dan UI. Sebaliknya, fungsi **fromDomainMovie()** digunakan untuk membentuk **MovieEntity** dari objek **Movie** dengan tambahan nilai **popularity**, sehingga data dari domain dapat disimpan kembali ke database. Konversi dua arah ini mencerminkan prinsip *separation of concerns*, di mana data yang digunakan di database (*data layer*) dipisahkan dari data yang digunakan di antarmuka dan logika aplikasi (*domain layer*).

File **MovieAppPreferences.kt**

File **MovieAppPreferences.kt** berfungsi sebagai kelas utilitas untuk mengelola penyimpanan data sederhana secara lokal menggunakan **SharedPreferences** pada aplikasi Android "MovieList". **SharedPreferences** sendiri adalah mekanisme bawaan Android yang memungkinkan penyimpanan data *key-value* secara permanen di perangkat, cocok untuk data ringan seperti preferensi pengguna atau konfigurasi aplikasi.

Dalam kelas ini, objek **SharedPreferences** diinisialisasi menggunakan nama **"tmdb_app_prefs"** dengan mode **Context.MODE_PRIVATE**, yang memastikan bahwa data hanya bisa diakses oleh aplikasi itu sendiri. Terdapat dua kunci utama yang dikelola dalam kelas ini, yaitu **KEY_API_KEY** untuk menyimpan API key TMDB, dan **KEY_DARK_MODE** untuk menyimpan status mode gelap (*dark mode*).

Fungsi **saveApiKey(apiKey: String)** dan **getApiKey()** digunakan untuk menyimpan dan mengambil nilai API key yang diperlukan untuk autentikasi saat melakukan permintaan ke server TMDB. Sementara itu, fungsi **saveDarkModeState(isDarkMode: Boolean)** dan **getDarkModeState()** digunakan untuk menyimpan dan membaca preferensi pengguna terhadap penggunaan mode gelap. Dengan menyimpan status ini, aplikasi

dapat mempertahankan tema yang dipilih pengguna meskipun aplikasi ditutup atau perangkat di-restart.

File **RetrofitClient.kt**

File **RetrofitClient.kt** berperan sebagai *singleton object* yang mengatur dan menyediakan instance Retrofit untuk melakukan komunikasi jaringan dengan API TMDB (*The Movie Database*) dalam aplikasi "MovieList". Kelas ini mendefinisikan konfigurasi dasar dan penting untuk memastikan proses permintaan dan pengambilan data dari server berjalan dengan efisien, andal, dan mudah untuk di-*debug*.

Pertama, konstanta **BASE_URL** menyimpan alamat dasar endpoint API TMDB versi 3, yang akan menjadi dasar semua permintaan jaringan. Kemudian, **Json** dikonfigurasi dari *KotlinX Serialization* untuk parsing JSON, dengan opsi **ignoreUnknownKeys = true** agar proses deserialisasi tidak gagal jika ada kunci asing yang tidak dikenali dalam respons API. **prettyPrint = true** digunakan untuk memformat hasil JSON agar lebih mudah dibaca, meskipun ini lebih berdampak saat melakukan *debugging*.

Untuk lapisan HTTP, digunakan **OkHttpClient**, yang dikonfigurasi secara *lazy* untuk efisiensi. Salah satu fitur penting yang ditambahkan adalah **HttpLoggingInterceptor** yang mencetak log isi permintaan dan respons HTTP ke logcat, sangat berguna untuk keperluan debugging saat mengembangkan aplikasi. Selain itu, waktu tunggu koneksi (**connectTimeout**, **readTimeout**, dan **writeTimeout**) disetel ke 30 detik untuk menangani kondisi jaringan yang lambat.

Akhirnya, objek **tmdbApiService** dibuat secara *lazy* menggunakan **Retrofit.Builder**, yang menghubungkan semua konfigurasi: **baseUrl**, **okHttpClient**, dan **converterFactory** untuk JSON. Hasilnya adalah instance dari **TmdbApiService**, yaitu antarmuka yang berisi definisi endpoint API yang akan digunakan oleh aplikasi.

File TmdbApiService.kt

File **TmdbApiService.kt** mendefinisikan antarmuka Retrofit yang berisi deklarasi endpoint untuk melakukan permintaan data ke API *The Movie Database* (TMDB). Antarmuka ini menjadi kontrak antara aplikasi dan layanan web TMDB, yang mendeskripsikan bagaimana permintaan dibuat dan respons apa yang diharapkan.

Dalam antarmuka ini terdapat satu fungsi utama, **getPopularMovies()**, yang ditandai dengan anotasi **@GET("movie/popular")**. Anotasi ini menunjukkan bahwa fungsi akan melakukan permintaan HTTP GET ke endpoint **movie/popular**, yang merupakan bagian dari TMDB API untuk mengambil daftar film populer.

Fungsi **getPopularMovies()** bersifat **suspend**, artinya dapat dipanggil dalam korutin sehingga mendukung pemrograman *asynchronous* tanpa memblokir *main thread*, sangat penting untuk menjaga responsivitas UI. Fungsi ini menerima tiga parameter: **apiKey** untuk otentikasi permintaan ke TMDB, **language** untuk menentukan bahasa hasil (secara default diatur ke **"en-US"**), dan **page** untuk mendukung pagination data (default halaman 1). Ketiganya akan secara otomatis dikonversi menjadi parameter URL melalui anotasi **@Query**.

Hasil dari fungsi ini adalah objek **Response<MovieListResponse>**, di mana **MovieListResponse** merupakan model data yang merepresentasikan struktur respons JSON dari TMDB untuk daftar film populer. Dengan membungkus respons dalam kelas **Response<>**, aplikasi dapat memeriksa status HTTP dan isi respons secara eksplisit, seperti sukses atau error, sebelum memproses datanya lebih lanjut.

File MovieDto.kt

File **MovieDto.kt** merupakan bagian dari lapisan data pada aplikasi Android yang berfungsi sebagai *Data Transfer Object* (DTO) untuk memetakan respons

JSON dari API *The Movie Database* (TMDB) ke dalam objek Kotlin. Kelas ini diberi anotasi **@Serializable** dari pustaka Kotlinx Serialization, yang memungkinkan proses serialisasi dan deserialisasi data JSON secara otomatis ketika berkomunikasi dengan jaringan menggunakan Retrofit. Setiap properti di dalam **MovieDto** merepresentasikan elemen informasi dari sebuah film, seperti **id**, **title**, **overview**, **popularity**, dan **releaseDate**. Beberapa properti diberi anotasi **@SerializedName** untuk menyelaraskan nama-nama variabel Kotlin dengan kunci pada JSON, misalnya **posterPath** disesuaikan dengan **poster_path** dari API. Selain itu, atribut seperti **genreIds**, **originalLanguage**, dan **video** menambah informasi tambahan mengenai kategori, bahasa asli, dan jenis konten. Tujuan utama dari **MovieDto** adalah menjadi representasi awal data film yang diambil dari API, sebelum nantinya dikonversi ke dalam model domain internal (seperti **Movie**) guna memisahkan struktur eksternal dari logika bisnis aplikasi. Hal ini membantu menjaga arsitektur aplikasi tetap bersih, modular, dan mudah untuk dikembangkan atau diperbarui ketika struktur API berubah.

File MovieDtoExtension.kt

File **MovieDtoExtension.kt** berisi dua buah *extension function* pada objek **MovieDto** yang berfungsi untuk mengonversi data hasil dari respons API (yang berbentuk objek **MovieDto**) ke dua bentuk lain dalam arsitektur aplikasi, yaitu ke dalam model domain (**Movie**) dan ke dalam entitas lokal (**MovieEntity**) yang disimpan di database. Fungsi **toDomainMovie()** mengonversi **MovieDto** menjadi objek **Movie**, yaitu representasi data film yang digunakan di lapisan domain atau logika bisnis aplikasi. Proses ini melibatkan pemetaan properti penting seperti **id**, **title**, **overview**, **posterPath**, **releaseDate**, dan **voteAverage**, tanpa membawa informasi yang hanya relevan untuk penyimpanan atau transport, seperti **popularity**. Sementara itu, fungsi **toMovieEntity()** mengonversi **MovieDto** menjadi **MovieEntity**, yaitu representasi data film yang akan disimpan secara lokal menggunakan Room. Fungsi ini mencakup semua

atribut penting termasuk **popularity**, karena nilai ini digunakan untuk proses penyortiran di database lokal. Dengan adanya dua fungsi konversi ini, aplikasi dapat memisahkan dan menjaga integritas data antara lapisan data, domain, dan penyimpanan, sesuai dengan prinsip *Clean Architecture*, sekaligus memudahkan pemeliharaan dan pengembangan aplikasi di masa depan.

File **MovieListResponse.kt**

File **MovieListResponse.kt** merupakan representasi dari struktur respons JSON yang diterima dari endpoint API populer TMDB yang mengembalikan daftar film. Kelas ini diberi anotasi **@Serializable** dari pustaka **kotlinx.serialization**, yang memudahkan proses konversi dari data JSON ke objek Kotlin saat menerima respons jaringan melalui Retrofit. Di dalam kelas ini terdapat empat properti utama yang sesuai dengan struktur data dari respons TMDB: **page** menunjukkan nomor halaman saat ini, **results** berisi daftar objek **MovieDto** yang merupakan detail film individual, **totalPages** adalah jumlah total halaman yang tersedia untuk permintaan tersebut, dan **totalResults** mencerminkan jumlah keseluruhan data film yang tersedia. Anotasi **@SerializedName** digunakan untuk menyesuaikan nama properti Kotlin (**totalPages**, **totalResults**) dengan nama yang digunakan di dalam JSON (**total_pages**, **total_results**), karena notasi pada API biasanya menggunakan gaya *snake_case*. Kelas ini memiliki peran penting dalam menangkap data yang dikirim oleh server dan mempermudah akses ke daftar film dalam bentuk list, sehingga bisa diproses lebih lanjut, ditampilkan ke pengguna, atau disimpan ke database lokal.

File **MovieRepository.kt**

File **MovieRepositoryImpl.kt** berfungsi sebagai implementasi konkret dari antarmuka **MovieRepository**, yang bertanggung jawab mengelola pengambilan data film populer dari sumber data jarak jauh (*remote*) maupun lokal (*local cache*). Kelas ini mengintegrasikan dua komponen utama, yaitu

TmdbApiService sebagai API client untuk berkomunikasi dengan TMDB dan **MovieDao** untuk operasi basis data lokal melalui Room. Ia juga menerima **apiKey** sebagai kunci akses ke layanan TMDB.

Dalam implementasinya, fungsi **getPopularMovies()** mengembalikan sebuah aliran data (**Flow**) bertipe **Result<List<Movie>>**. Pertama, data dari database lokal diambil dan diubah menjadi domain model menggunakan fungsi ekstensi **toDomainMovie()**, lalu dikirim sebagai **Result.Success** jika tidak kosong. Ini memastikan pengguna tetap bisa melihat data meskipun belum ada jaringan.

Kemudian, repository mencoba memuat data terbaru dari API. Jika berhasil (**isSuccessful**), daftar film (**MovieDto**) dikonversi ke domain model, lalu di-cache ke database lokal setelah menghapus data sebelumnya. Hasil ini kembali dikirim sebagai **Result.Success**. Namun, bila terjadi kegagalan, seperti **HttpException**, **IOException**, atau error lainnya, error yang sesuai dibungkus dalam **Result.Error** dengan pesan deskriptif.

File Movie.kt

File **Movie.kt** dalam paket **domain.model** mendefinisikan struktur data **Movie** sebagai representasi entitas film dalam lapisan domain aplikasi. Kelas ini ditandai dengan anotasi **@Parcelize**, yang memungkinkan objek **Movie** untuk di-serialize secara otomatis ke dalam bentuk **Parcelable**. Hal ini sangat penting dalam konteks Android karena memungkinkan objek **Movie** dikirim antar komponen seperti **Activity** atau **Fragment** melalui **Intent** atau **Bundle** dengan efisien dan aman.

Struktur **Movie** terdiri dari beberapa properti inti, yaitu **id** sebagai identitas unik film, **title** sebagai judul film, **overview** sebagai ringkasan cerita, **posterPath** sebagai jalur gambar poster (yang bersifat opsional), **releaseDate** untuk tanggal rilis, serta **voteAverage** sebagai nilai rating rata-rata dari pengguna.

File GetPopularMoviesUseCase.kt

File **GetPopularMoviesUseCase.kt** merupakan bagian dari lapisan domain dalam arsitektur aplikasi dan berfungsi sebagai use case atau *interactor* yang menghubungkan logika bisnis dengan data yang diperlukan oleh UI. Kelas **GetPopularMoviesUseCase** memiliki tanggung jawab untuk menjalankan proses pengambilan data film populer dengan memanfaatkan fungsi yang disediakan oleh **MovieRepositoryImpl.kt**.

Dengan menggunakan operator **invoke()**, kelas ini memungkinkan *instance*-nya dipanggil seperti fungsi, sehingga memudahkan penggunaannya di kelas lain seperti **ViewModel**. Fungsi **invoke()** mengembalikan objek **Flow** dari **Result<List<Movie>>**, yang berarti aliran data film populer akan dipancarkan dalam bentuk hasil yang dapat berupa **Loading**, **Success**, atau **Error**. Hal ini selaras dengan pendekatan reaktif menggunakan Kotlin Coroutines dan **Flow**, sehingga proses pemanggilan data tetap responsif dan efisien terhadap perubahan data maupun kondisi jaringan.

File GetPopularMoviesUseCase.kt

File **GetPopularMoviesUseCase.kt** merupakan bagian dari lapisan domain dalam arsitektur aplikasi dan berfungsi sebagai use case atau *interactor* yang menghubungkan logika bisnis dengan data yang diperlukan oleh UI. Kelas **GetPopularMoviesUseCase** memiliki tanggung jawab untuk menjalankan proses pengambilan data film populer dengan memanfaatkan fungsi yang disediakan oleh **MovieRepositoryImpl.kt**.

Dengan menggunakan operator **invoke()**, kelas ini memungkinkan *instance*-nya dipanggil seperti fungsi, sehingga memudahkan penggunaannya di kelas lain seperti **ViewModel**. Fungsi **invoke()** mengembalikan objek **Flow** dari **Result<List<Movie>>**, yang berarti aliran data film populer akan dipancarkan dalam bentuk hasil yang dapat berupa **Loading**, **Success**, atau **Error**. Hal ini

selaras dengan pendekatan reaktif menggunakan Kotlin Coroutines dan **Flow**, sehingga proses pemanggilan data tetap responsif dan efisien terhadap perubahan data maupun kondisi jaringan.

File DetailActivity.kt

File **DetailActivity** adalah bagian dari lapisan presentasi dalam aplikasi dan bertanggung jawab untuk menampilkan detail dari sebuah objek **Movie** yang dipilih oleh pengguna. Aktivitas ini menggunakan **View Binding** melalui **ActivityDetailBinding** untuk mengakses elemen UI secara langsung dan aman tanpa perlu memanggil **findViewById**. Saat **onCreate()** dijalankan, **DetailActivity** akan mencoba mengambil data **Movie** yang dikirim melalui **Intent** menggunakan kunci **EXTRA_MOVIE**. Untuk memastikan kompatibilitas dengan berbagai versi Android, pengecekan dilakukan menggunakan **Build.VERSION.SDK_INT**, di mana metode pemanggilan **getParcelableExtra** akan menyesuaikan berdasarkan versi SDK.

Jika data **Movie** berhasil ditemukan, maka informasi seperti judul, tanggal rilis, rating, dan sinopsis akan ditampilkan pada tampilan yang sesuai. Gambar poster film juga dimuat menggunakan library **Glide** dari path URL **https://image.tmdb.org/t/p/w500**, yang merupakan standar path poster dari API TMDB. Judul **ActionBar** akan diubah agar sesuai dengan judul film yang ditampilkan. Namun jika objek **Movie** bernilai **null**, pengguna akan diberi notifikasi berupa **Toast** yang menyatakan bahwa film tidak ditemukan, dan aktivitas langsung ditutup menggunakan **finish()**.

Selain itu, aktivitas ini juga menangani event saat tombol back di **ActionBar** ditekan dengan menimpa fungsi **onOptionsItemSelected**, yang memanggil **onBackPressedDispatcher.onBackPressed()** agar pengguna dapat kembali ke layar sebelumnya dengan cara yang sesuai dengan sistem.

File MovieAdapter.kt

File **MovieAdapter** merupakan implementasi dari **ListAdapter** yang dirancang untuk menampilkan daftar film (**Movie**) dalam komponen **RecyclerView**. Dengan menggunakan **DiffUtil.ItemCallback**, adapter ini secara efisien membandingkan perbedaan antar item dalam daftar, sehingga hanya elemen yang berubah saja yang akan diperbarui pada UI—meningkatkan performa terutama saat data diperbarui secara dinamis.

Adapter ini memiliki fungsi **onItemClick**, yaitu lambda opsional yang dapat diatur untuk menangani event ketika pengguna menekan tombol detail pada setiap item film. Pada metode **onCreateViewHolder**, layout item untuk tiap film diinflasi dari **ItemMovieBinding**, yang kemudian digunakan dalam **MovieViewHolder**. Dalam metode **onBindViewHolder**, data film pada posisi tertentu di-bind ke **ViewHolder** dengan memanggil fungsi **bind()**.

Kelas **MovieViewHolder** bertanggung jawab mengisi data ke elemen UI pada tiap item film, seperti judul, tanggal rilis, rating, dan sinopsis. Selain itu, poster film dimuat menggunakan library **Glide** dari URL **<https://image.tmdb.org/t/p/w500>**, yang merupakan path standar gambar dari API TMDb. Pada bagian **init**, tombol detail disiapkan untuk *trigger* fungsi **onItemClick** saat ditekan, sehingga event klik bisa ditangani di luar adapter, misalnya untuk navigasi ke halaman detail.

File **MovieViewModel.kt**

File **MovieViewModel** adalah bagian dari lapisan presentasi dalam arsitektur MVVM (Model-View-ViewModel) yang bertanggung jawab untuk mengelola data film populer dan menyediakannya ke antarmuka pengguna (**UI**). ViewModel ini menerima dependensi berupa **GetPopularMoviesUseCase**, yang berfungsi untuk menjalankan logika bisnis dalam mengambil data film dari repository.

Properti **_popularMovies** adalah objek **MutableLiveData** bertipe **Result<List<Movie>>**, yang menampung status data film populer, baik itu dalam bentuk **Loading**, **Success**, maupun **Error**. Data ini dibungkus

menggunakan **LiveData** publik (**popularMovies**) agar dapat diamati oleh UI tanpa memberi akses untuk memodifikasi nilainya secara langsung, menjaga prinsip encapsulation.

Saat **ViewModel** diinisialisasi, fungsi **fetchPopularMovies()** akan langsung dijalankan. Fungsi ini memanfaatkan coroutine pada **viewModelScope** untuk mengeksekusi use case secara asynchronous, lalu mengumpulkan (**collect**) hasilnya. Setiap perubahan hasil (loading, data berhasil, atau error) akan diperbarui ke **LiveData _popularMovies**, yang nantinya akan diterima oleh observer di UI seperti **Activity** atau **Fragment**.

File ViewModelFactory.kt

File **ViewModelFactory** pada kode tersebut berfungsi sebagai pabrik pembuat objek **ViewModel**, khususnya **MovieViewModel**, yang membutuhkan dependensi eksternal yaitu **GetPopularMoviesUseCase**. Karena **ViewModel** tidak secara langsung mendukung konstruktor berparameter, maka **ViewModelProvider.Factory** digunakan untuk menyuntikkan dependensi secara manual saat **ViewModel** dibuat.

Metode **create()** dalam **ViewModelFactory** akan dipanggil oleh **ViewModelProvider** saat sistem membutuhkan instance **MovieViewModel**. Ia memeriksa apakah **modelClass** merupakan turunan dari **MovieViewModel** menggunakan **isAssignableFrom**. Jika cocok, maka instance **MovieViewModel** dikembalikan dengan cara casting ke tipe generik **T**. Jika tidak, akan dilempar **IllegalArgumentException** karena **ViewModelFactory** ini hanya bisa membuat **MovieViewModel**.

File Result.kt

File **Result** dalam kode tersebut merupakan sebuah *sealed class* yang dirancang untuk merepresentasikan status dari suatu operasi asynchronous, seperti

pengambilan data dari API atau database, dalam arsitektur aplikasi berbasis MVVM. Kelas ini memiliki tiga kemungkinan status, yaitu **Loading**, **Success**, dan **Error**. Status **Loading** digunakan untuk menunjukkan bahwa proses sedang berlangsung, biasanya dimanfaatkan untuk menampilkan indikator pemuatan di antarmuka pengguna. Status **Success** membawa data hasil dari operasi yang berhasil, di mana data tersebut akan diteruskan ke UI untuk ditampilkan, misalnya daftar film populer dalam bentuk **List<Movie>**. Sedangkan status **Error** digunakan untuk menangani kondisi ketika operasi gagal, misalnya karena kesalahan jaringan, kesalahan parsing, atau kondisi tak terduga lainnya, dengan membawa objek **Exception** yang memuat pesan error.

File activity_detail.xml

Kode **Activity_Detail.xml** merupakan layout untuk **DetailActivity** pada aplikasi Android yang menampilkan detail sebuah film. Layout ini menggunakan **ScrollView** sebagai elemen induk agar kontennya dapat digulir jika melebihi tinggi layar perangkat. Di dalam **ScrollView**, terdapat **ConstraintLayout** yang mengatur posisi elemen-elemen UI secara fleksibel berdasarkan batasan (constraints).

Elemen pertama adalah **ImageView** dengan ID **iv_detail_poster** yang digunakan untuk menampilkan poster film dalam ukuran besar dan memenuhi lebar layar. Setelah itu, ada beberapa **TextView** yang menyajikan informasi detail film, seperti judul (**tv_detail_title**), tanggal rilis (**tv_detail_release_date**), dan rating (**tv_detail_vote_average**). Di bagian bawah, terdapat label "Overview" (**tv_detail_overview_label**) dan **TextView** (**tv_detail_overview**) yang berfungsi untuk menampilkan deskripsi lengkap atau sinopsis film. Setiap elemen diberi margin dan diatur dengan constraint agar rapi dan responsif terhadap berbagai ukuran layar.

File activity_main.xml

Kode **activity_main.xml** merupakan **MainActivity** yang menampilkan daftar film populer. Layout ini menggunakan **ConstraintLayout** sebagai kontainer utama agar setiap komponen UI dapat diatur secara fleksibel dan responsif. Di bagian atas terdapat **TextView** dengan ID **tv_title** yang berfungsi sebagai judul halaman dengan teks "Popular Movies" dan tampilan tebal berukuran besar. Tepat di bawahnya, terdapat elemen **ProgressBar** dengan ID **progress_bar** yang secara default disembunyikan (**visibility="gone"**), dan digunakan untuk menandakan bahwa data sedang dimuat.

Jika terjadi kesalahan saat mengambil data, maka **TextView** dengan ID **tv_error** akan ditampilkan bersama **Button** dengan ID **btn_retry**, yang memungkinkan pengguna untuk mencoba memuat ulang data. Komponen utama dari halaman ini adalah **RecyclerView** dengan ID **rv_movies** yang akan menampilkan daftar film dalam bentuk list, dan posisinya berada di bawah judul dan mengisi sebagian besar area layar. Di pojok kanan atas, terdapat **SwitchMaterial** dari *Material Components* dengan ID **switch_dark_mode** yang memungkinkan pengguna untuk mengaktifkan atau menonaktifkan mode gelap (*dark mode*).

File item_movie.xml

Kode **item_movie.xml** merupakan layout item untuk **RecyclerView** yang digunakan pada daftar film populer. Komponen utamanya adalah **CardView**, yang memberikan tampilan berbentuk kartu dengan sudut melengkung (**cardCornerRadius="8dp"**) dan efek bayangan (**cardElevation="4dp"**) agar terlihat menonjol dari latar belakang. Di dalam **CardView** terdapat **ConstraintLayout** yang mengatur posisi setiap elemen secara fleksibel.

Setiap item terdiri dari sebuah **ImageView** dengan ID **iv_poster** di sisi kiri yang menampilkan gambar poster film. Di sebelah kanannya terdapat tiga **TextView** berturut-turut: **tv_movie_title** untuk menampilkan judul film

dengan ukuran dan gaya tulisan yang menonjol, **tv_release_date** untuk menampilkan tanggal rilis, dan **tv_vote_average** yang menampilkan rating film. Di bawah poster, terdapat **TextView** lain dengan ID **tv_overview** yang menampilkan sinopsis singkat film, dibatasi maksimal 3 baris dan dipotong (**ellipsize="end"**) jika terlalu panjang agar tetap ringkas. Terakhir, ada sebuah tombol **Button** dengan teks "**Lihat Detail**" yang memungkinkan pengguna untuk melihat informasi lebih lengkap dari film yang bersangkutan.

File build.gradle.kts (Module:app)

Kode **build.gradle.kts** merupakan konfigurasi utama untuk aplikasi Android **MovieList** yang ditulis menggunakan Kotlin DSL. Plugin yang digunakan mencakup plugin Android aplikasi, Kotlin Android, **kotlin-parcelize** untuk mendukung parcelable class, **kotlin-kapt** untuk annotation processing (misalnya untuk Room), serta plugin Kotlin serialization untuk pengolahan data JSON secara efisien menggunakan Kotlinx Serialization.

Bagian **android** mengatur informasi penting seperti namespace, versi SDK (**compileSdk**, **minSdk**, dan **targetSdk**), serta konfigurasi build default seperti **applicationId**, **versionCode**, dan **versionName**. Selain itu, juga disetel opsi build khusus untuk rilis seperti penggunaan ProGuard, konfigurasi **Java 11**, dan aktivasi fitur **viewBinding** untuk mempermudah akses ke komponen UI dari XML.

Di bagian **dependencies**, tercantum berbagai pustaka penting. Untuk antarmuka dasar, digunakan AndroidX Core, AppCompatActivity, Material Design, Activity, dan ConstraintLayout. Pengujian didukung oleh JUnit, Espresso, dan library pengujian AndroidX. Untuk penyimpanan lokal, digunakan Room (termasuk **room-runtime**, **room-ktx**, dan compiler dengan **kapt**). Untuk pemuatan gambar, digunakan Glide. Pengambilan data dari API dilakukan melalui Retrofit, dikombinasikan dengan Kotlinx Serialization dan OkHttp sebagai klien HTTP. Library **lifecycle** dipakai untuk arsitektur berbasis ViewModel dan LiveData.

Coroutine juga digunakan untuk pengelolaan thread asynchronous yang efisien. Terakhir, terdapat dukungan navigasi menggunakan **androidx.navigation** untuk berpindah antar-fragment secara aman.

LINK GITHUB

Berikut adalah tautan untuk semua source code yang telah dibuat:

<https://github.com/biilaaa/Pemrograman-Mobile>